

NUS-ISS

*Pattern Recognition using
Machine Learning System*



Day 4(I) - Case studies on using convolutional neural networks for machine learning systems

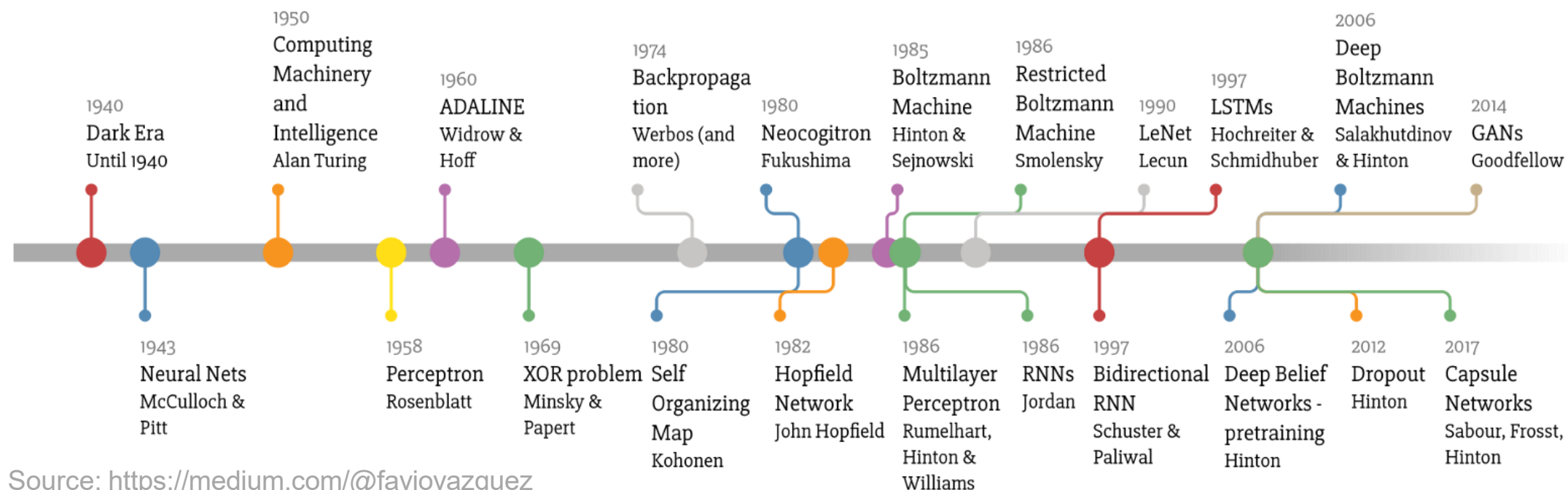
by Dr. Gary Leung

The long march to the deepest network

The deeper the tougher

Breaking the winter

- The deeper the network, the better it can perform
- But the deeper the network, the harder it is to train
- Why?
- Why did deep learning emerge so late?

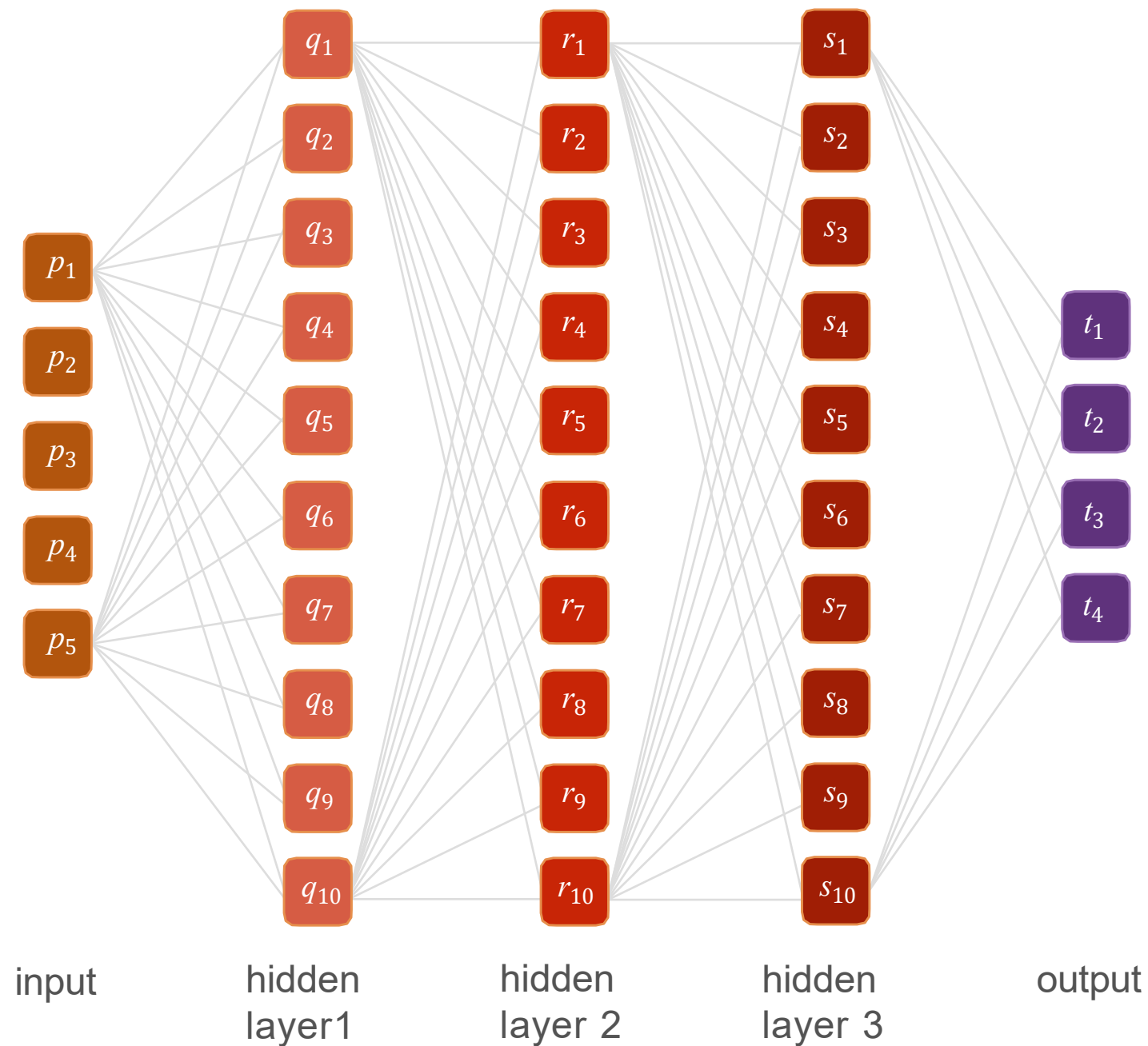


Source: <https://medium.com/@favio vazquez>

Backpropagation

A simple example

- What is propagated backward during backpropagation?



Backpropagation

From the last layer

- For an input to the network, we obtain an output. Together with the corresponding training label and loss function, we can calculate the output-layer error signals, i.e. gradients of the loss w.r.t. the output pre-activations

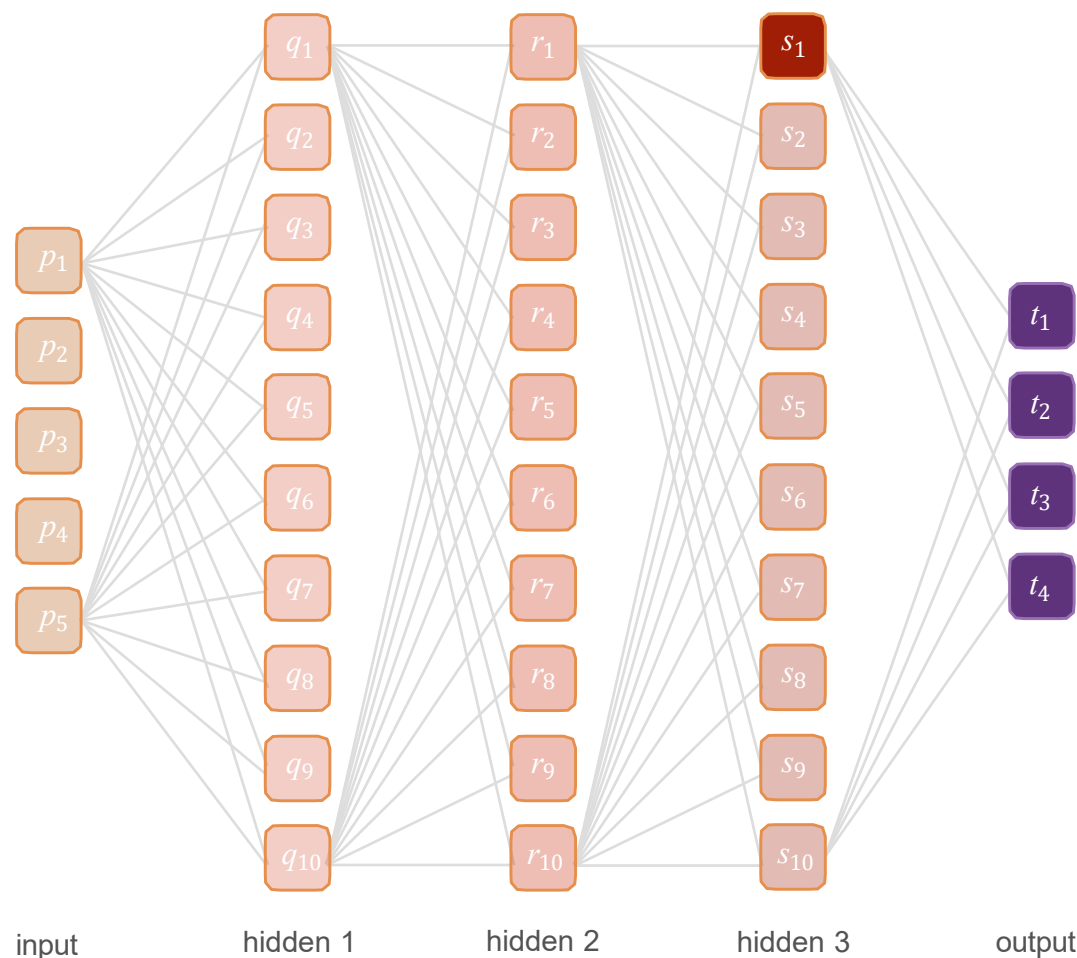
- Assume the error signals at the output layer are

$$\delta_{t_1}, \delta_{t_2}, \delta_{t_3}, \delta_{t_4}$$

where $\delta_{t_i} = \frac{\partial L}{\partial z_{t_i}}$

and z_{t_i} is the pre-activation value of output unit t_i , before applying the output activation function

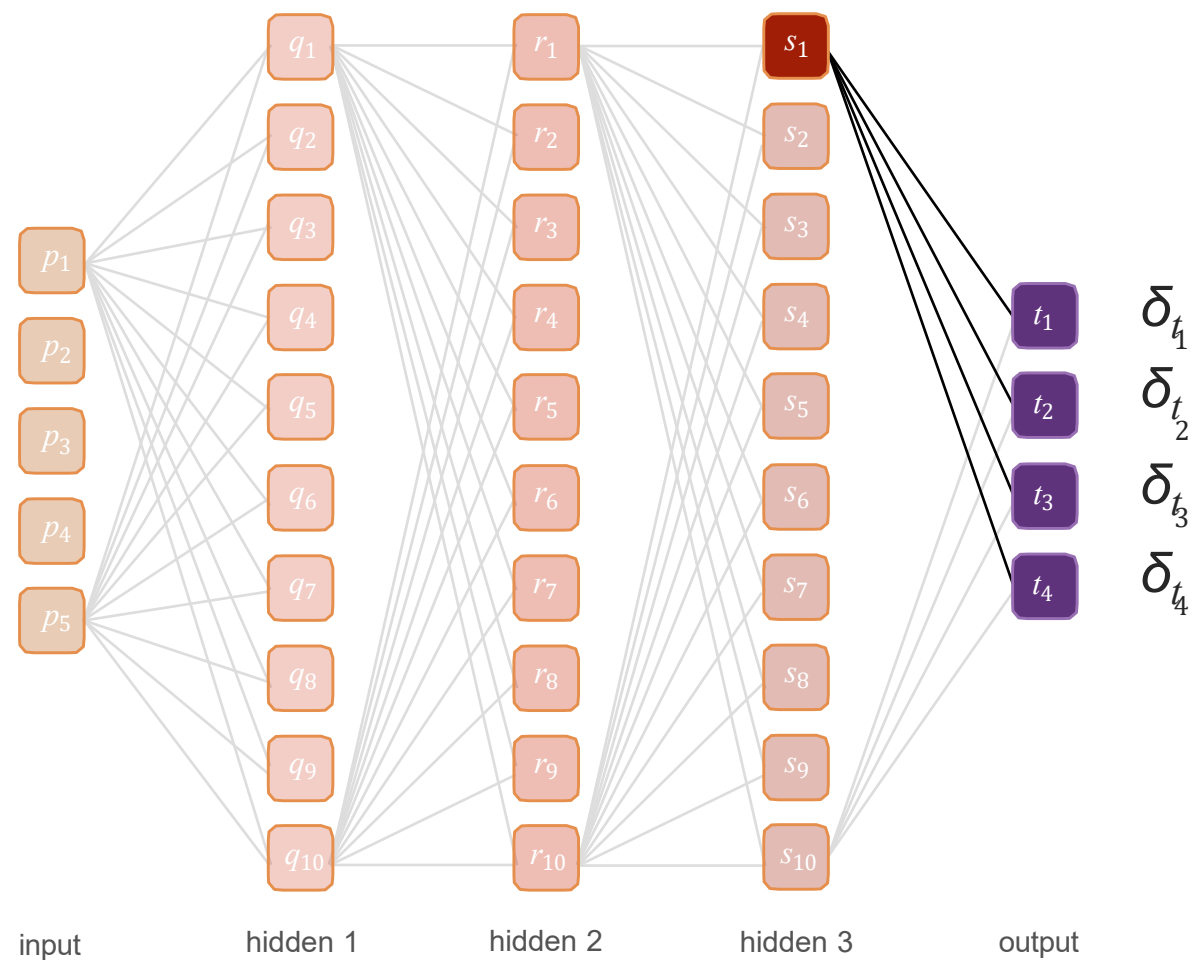
- What is the error signal at s_1 ?



Backpropagation

Weight connections at the last layer

- $w_{s_1 t_1}$: Weight connecting neuron s_1 and t_1
- $w_{s_1 t_2}$: Weight connecting neuron s_1 and t_2
- $w_{s_1 t_3}$: Weight connecting neuron s_1 and t_3
- $w_{s_1 t_4}$: Weight connecting neuron s_1 and t_4



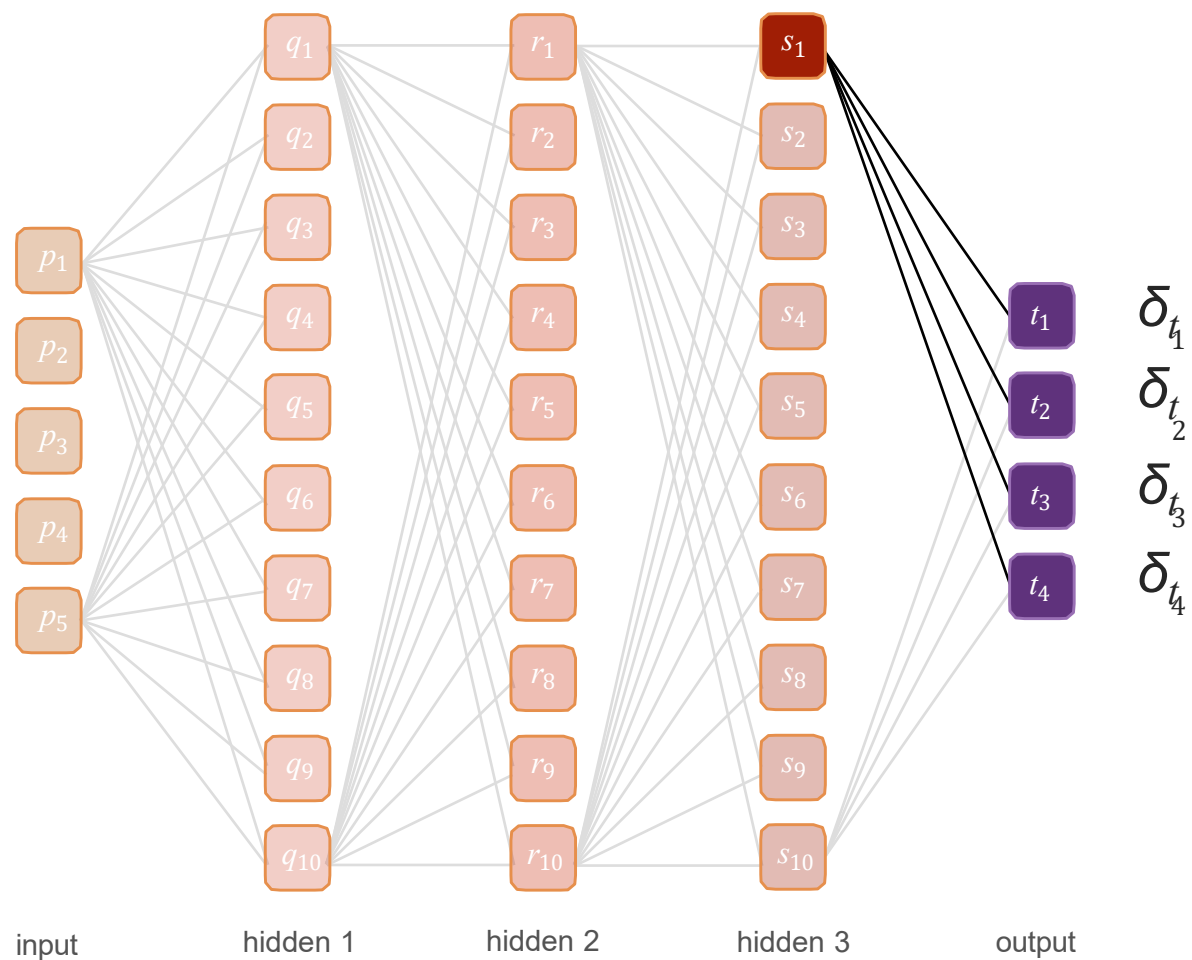
Backpropagation

The error signal at s_1

- The error signal at s_1

$$\delta_{s_1} = f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \delta_{t_2} w_{s_1 t_2} \cdots + \delta_{t_4} w_{s_1 t_4}]$$

- f is the chosen activation function
 f' is its derivative function, and
 z_{s_1} is the pre-activation output of neuron s_1



Backpropagation

The error signal at s_2

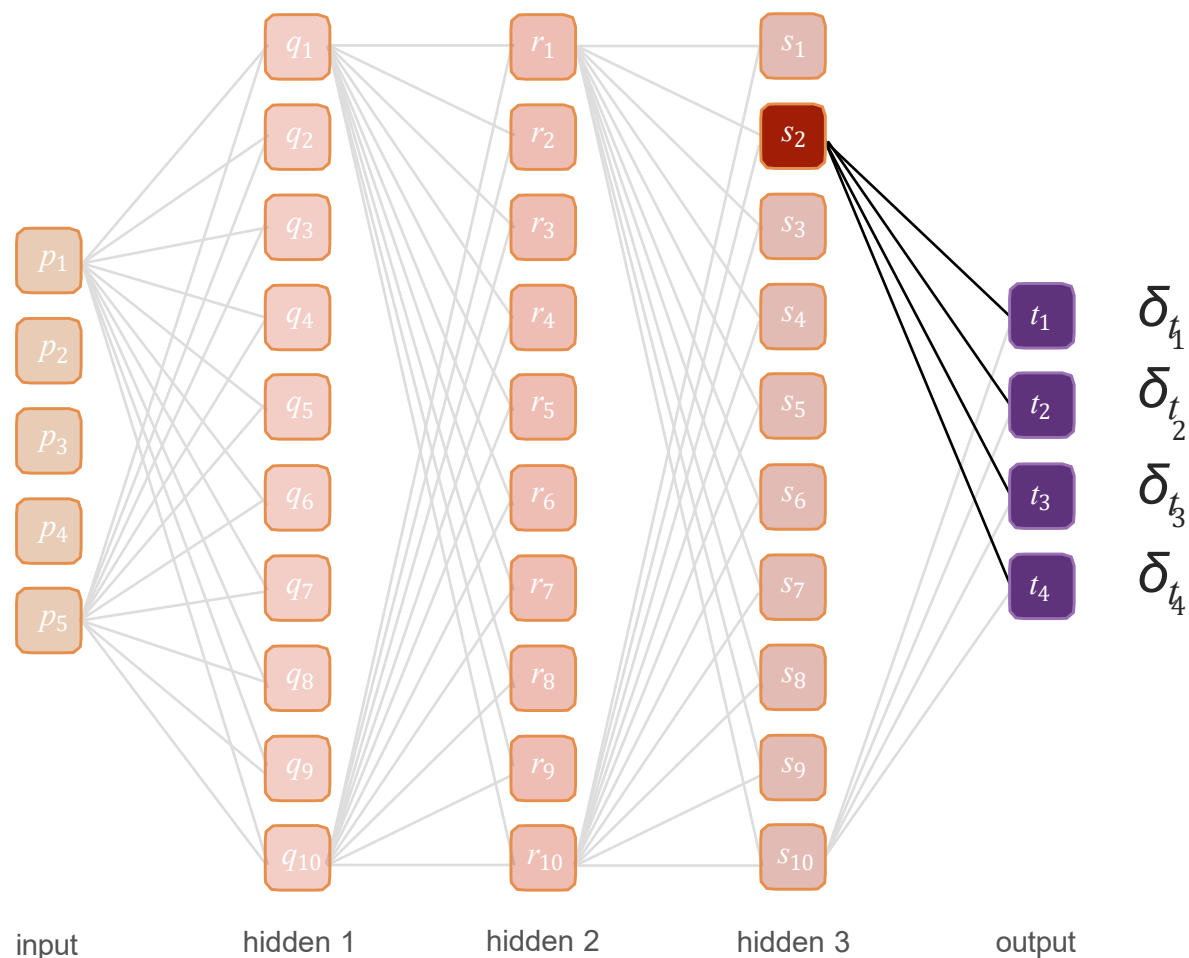
- The error signal at s_1

$$\delta_{s_1} = f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \delta_{t_2} w_{s_1 t_2} \cdots + \delta_{t_4} w_{s_1 t_4}]$$

- f is the chosen activation function
 f' is its derivative function, and
 z_{s_1} is the pre-activation output of neuron s_1

- Similarly, the error signal at s_2 is

$$\delta_{s_2} = f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \delta_{t_2} w_{s_2 t_2} \cdots + \delta_{t_4} w_{s_2 t_4}]$$



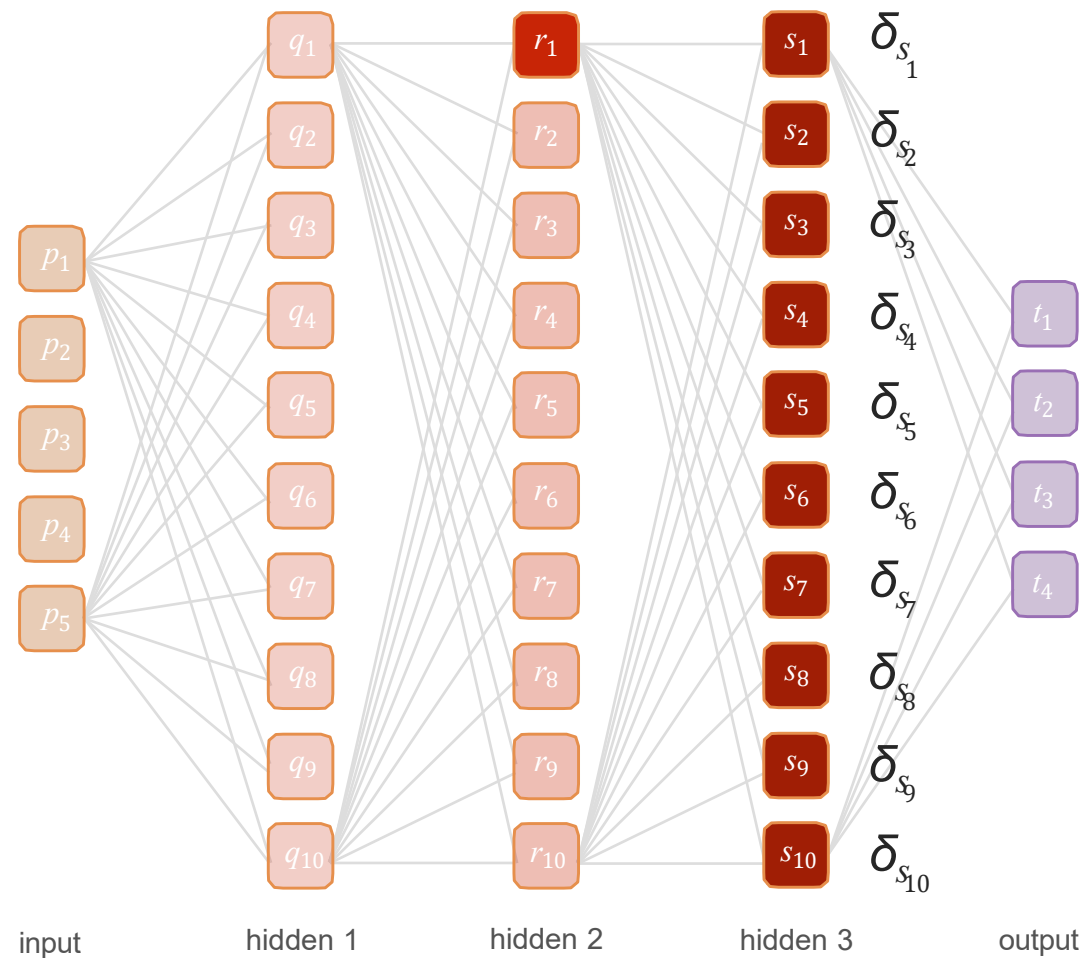
Backpropagation

The error at r_1

- What about the error signal at r_1 ?

$$\delta_{r_1} = f'(z_{r_1}) [\delta_{s_1} w_{r_1 s_1} + \delta_{s_2} w_{r_1 s_2} \cdots + \delta_{s_{10}} w_{r_1 s_{10}}]$$

- We know the expressions for the error signals at $s_1, s_2, \dots, s_{10}$. Substituting them into the expression above gives



$$\begin{aligned} \delta_{r_1} = & f'(z_{r_1}) \underbrace{f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \cdots + \delta_{t_4} w_{s_1 t_4}]}_{\delta_{s_1}} w_{r_1 s_1} \\ & + f'(z_{r_1}) \underbrace{f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \cdots + \delta_{t_4} w_{s_2 t_4}]}_{\delta_{s_2}} w_{r_1 s_2} \\ & \vdots \\ & + f'(z_{r_1}) \underbrace{f'(z_{s_{10}}) [\delta_{t_1} w_{s_{10} t_1} + \cdots + \delta_{t_4} w_{s_{10} t_4}]}_{\delta_{s_{10}}} w_{r_1 s_{10}} \end{aligned}$$

Backpropagation

The error at q_1

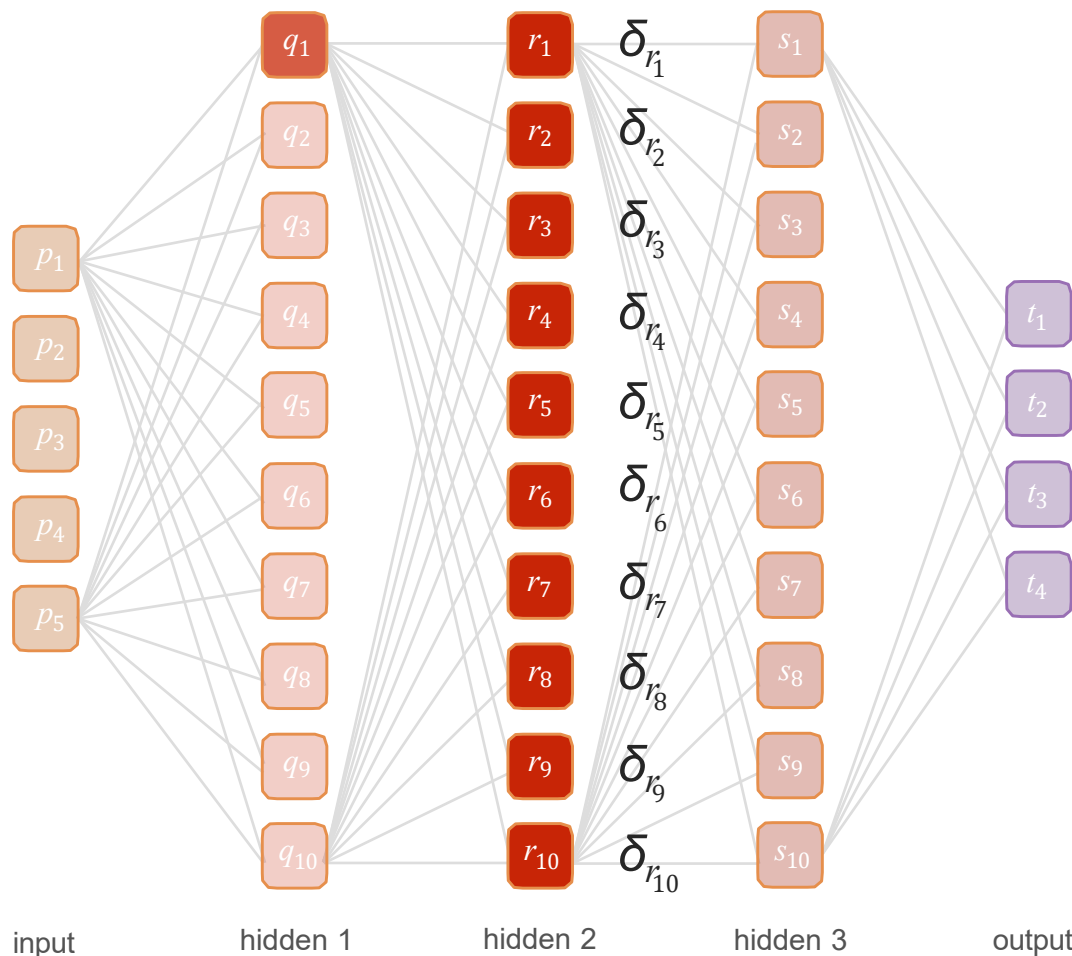
- How about the error signal at q_1 ?

$$\delta_{q_1} = f'(z_{q_1}) [\delta_{r_1} w_{q_1 r_1} + \delta_{r_2} w_{q_1 r_2} \cdots + \delta_{r_{10}} w_{q_1 r_{10}}]$$

- We know the error signals at $r_1, r_2, \dots, r_{10}$. However, if we substitute them into the expression above, the equation will be too long to be displayed here. Therefore, let us look at the first term

$$\begin{aligned} f'(z_{q_1}) \delta_{r_1} w_{q_1 r_1} &= f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \cdots \delta_{t_4} w_{s_1 t_4}] w_{r_1 s_1} w_{q_1 r_1} \\ &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \cdots \delta_{t_4} w_{s_2 t_4}] w_{r_1 s_2} w_{q_1 r_1} \\ &\vdots \\ &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_{10}}) [\delta_{t_1} w_{s_{10} t_1} + \cdots \delta_{t_4} w_{s_{10} t_4}] w_{r_1 s_{10}} w_{q_1 r_1} \end{aligned}$$

- What observations can we make about each item in the expression above?

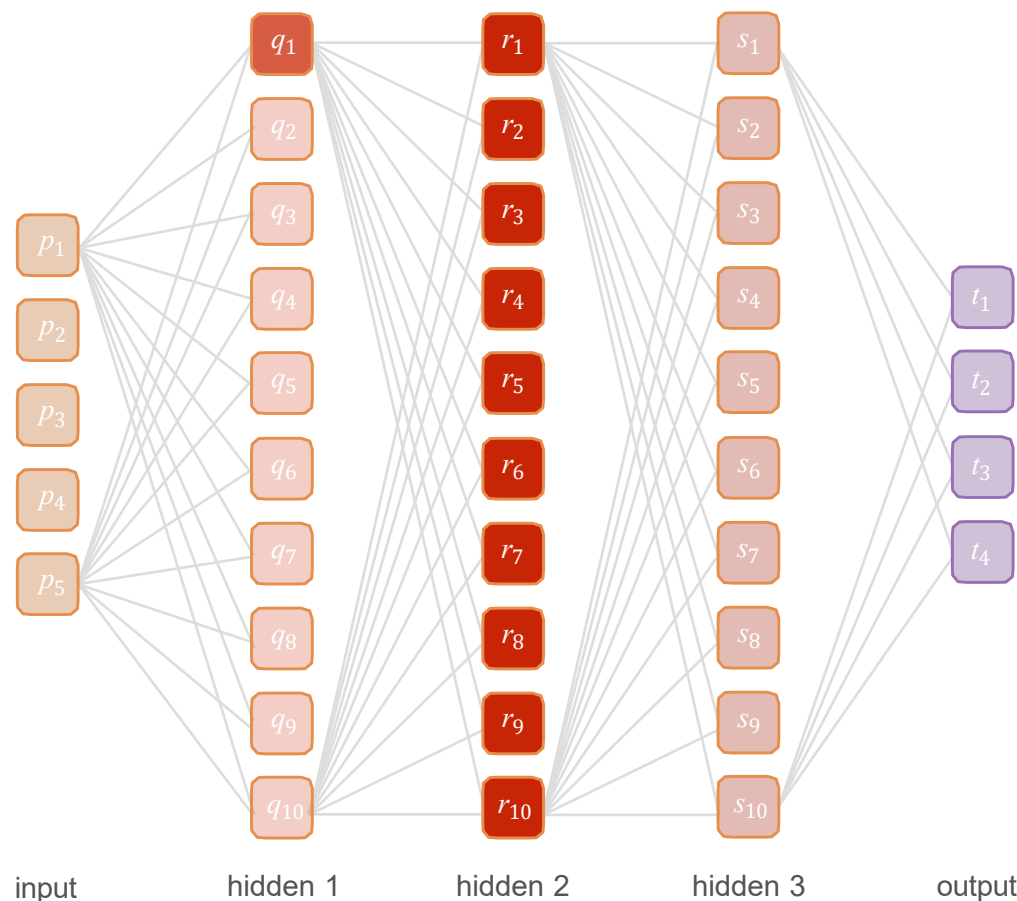


Backpropagation

Observations

$$\begin{aligned}
 f'(z_{q_1}) \delta_{r_1} w_{q_1 r_1} &= f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \dots \delta_{t_4} w_{s_1 t_4}] w_{r_1 s_1} w_{q_1 r_1} \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \dots \delta_{t_4} w_{s_2 t_4}] w_{r_1 s_2} w_{q_1 r_1} \\
 &\vdots \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_{10}}) [\delta_{t_1} w_{s_{10} t_1} + \dots \delta_{t_4} w_{s_{10} t_4}] w_{r_1 s_{10}} w_{q_1 r_1}
 \end{aligned}$$

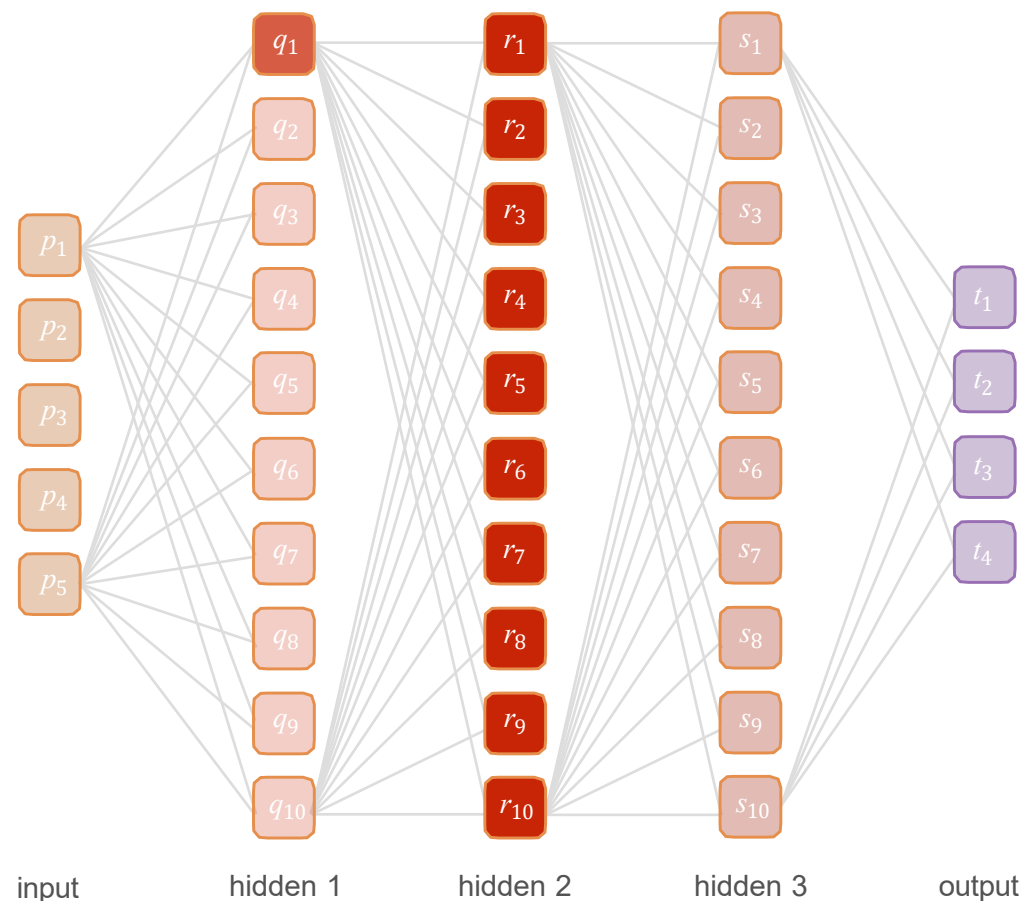
- Many multiplications are involved in the calculation of each item
- In particular, there are repeated multiplications of derivative of activation functions
- At hidden layer 1, each term contains a product of 3 activation-function derivatives
- If the network is deeper than this example, earlier layers will involve even more multiplications of activation-function derivatives in each item of the equation
- Note: *The multiplication of many derivatives happens in all deep networks, including CNNs*



Backpropagation

Implication

$$\begin{aligned}
 f'(z_{q_1}) \delta_{r_1} w_{q_1 r_1} &= f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \dots \delta_{t_4} w_{s_1 t_4}] w_{r_1 s_1} w_{q_1 r_1} \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \dots \delta_{t_4} w_{s_2 t_4}] w_{r_1 s_2} w_{q_1 r_1} \\
 &\vdots \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_{10}}) [\delta_{t_1} w_{s_{10} t_1} + \dots \delta_{t_4} w_{s_{10} t_4}] w_{r_1 s_{10}} w_{q_1 r_1}
 \end{aligned}$$

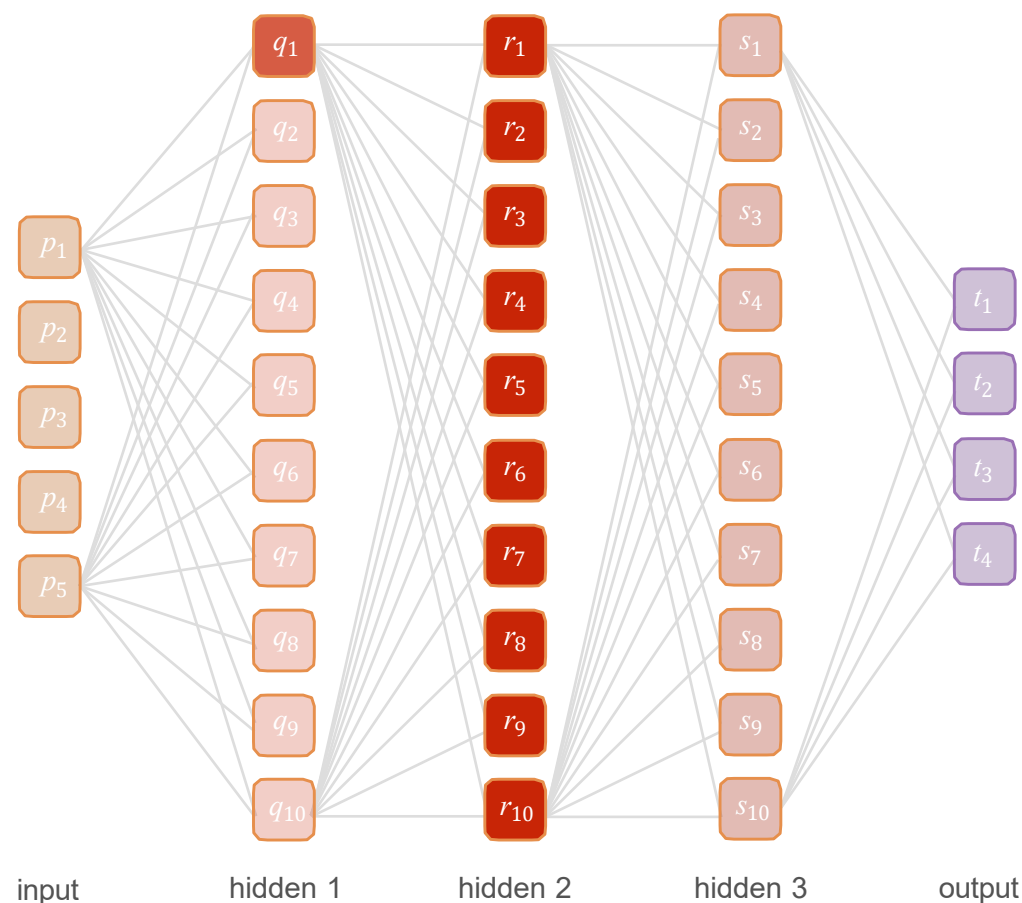


- In backpropagation, the update applied to each weight is proportional to the error signal
- If the chosen activation function has derivatives that are often very small, the error signals in earlier layers can also become very small
- Consequently, earlier layers learn very slowly

The importance of earlier layers

$$\begin{aligned}
 f'(z_{q_1}) \delta_{r_1} w_{q_1 r_1} &= f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_1}) [\delta_{t_1} w_{s_1 t_1} + \dots \delta_{t_4} w_{s_1 t_4}] w_{r_1 s_1} w_{q_1 r_1} \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_2}) [\delta_{t_1} w_{s_2 t_1} + \dots \delta_{t_4} w_{s_2 t_4}] w_{r_1 s_2} w_{q_1 r_1} \\
 &\vdots \\
 &+ f'(z_{q_1}) f'(z_{r_1}) f'(z_{s_{10}}) [\delta_{t_1} w_{s_{10} t_1} + \dots \delta_{t_4} w_{s_{10} t_4}] w_{r_1 s_{10}} w_{q_1 r_1}
 \end{aligned}$$

- Earlier layers are key to good performance
- Earlier layers can detect simple patterns in the input
- Earlier layers serve as feature extractors
- If the feature extractors fail, the classifier receives poor-quality input



How can we train the earlier layers?

3 main strategies to start

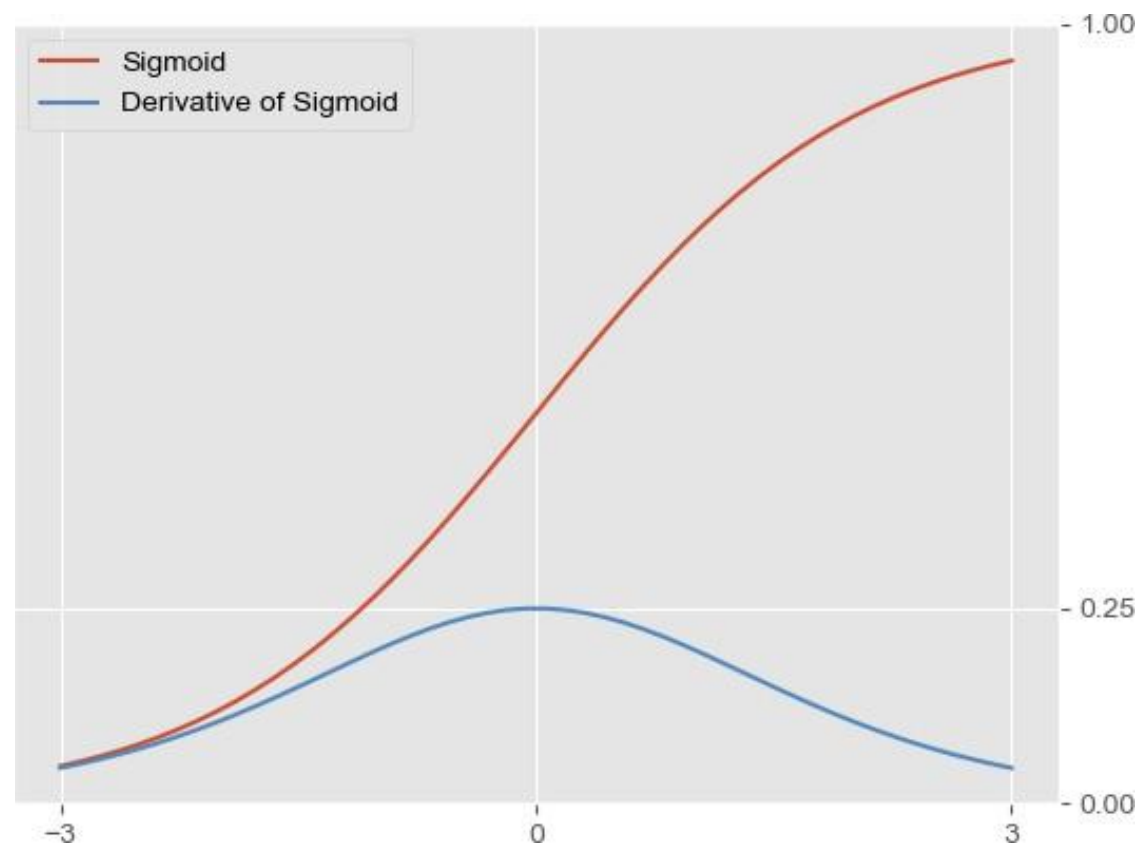
- First, choose the right activation function
- Second, initialize the weights properly
- Third, constrain the weights

First, choose the right activation function

The problem with Sigmoid

Small gradients

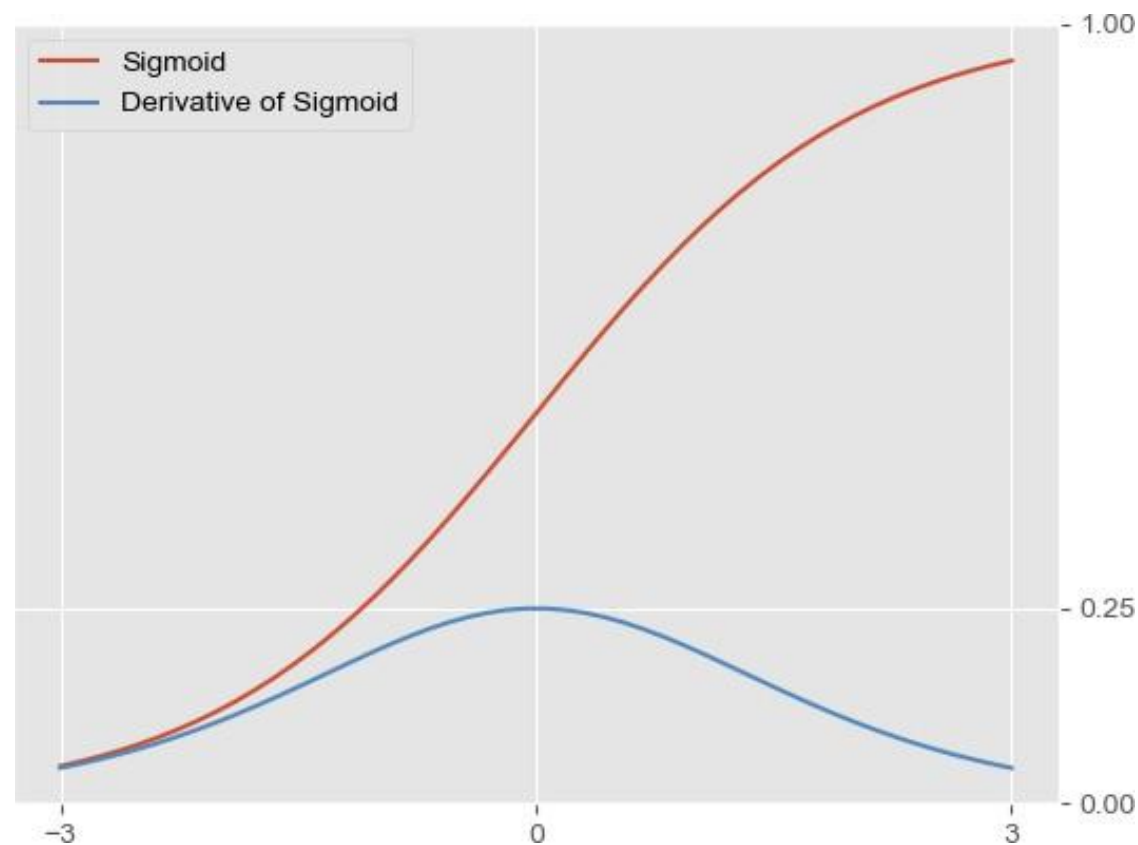
- When sigmoid outputs are close to 0 or 1, the derivative becomes very small
- This can cause vanishing gradients and poor learning
- Poor weight initialization can make this worse, especially with large positive or negative values



The problem with Sigmoid

Small gradients

- Even when weights are initialized properly, the largest derivative value is only around 0.25
- During backpropagation, multiplying these small values repeatedly can lead to vanishing gradients
- For example, $0.25 \times 0.25 \times 0.25 \times 0.25 = 0.004$, which is very small
- As a result, earlier layers may receive very small updates and learn slowly



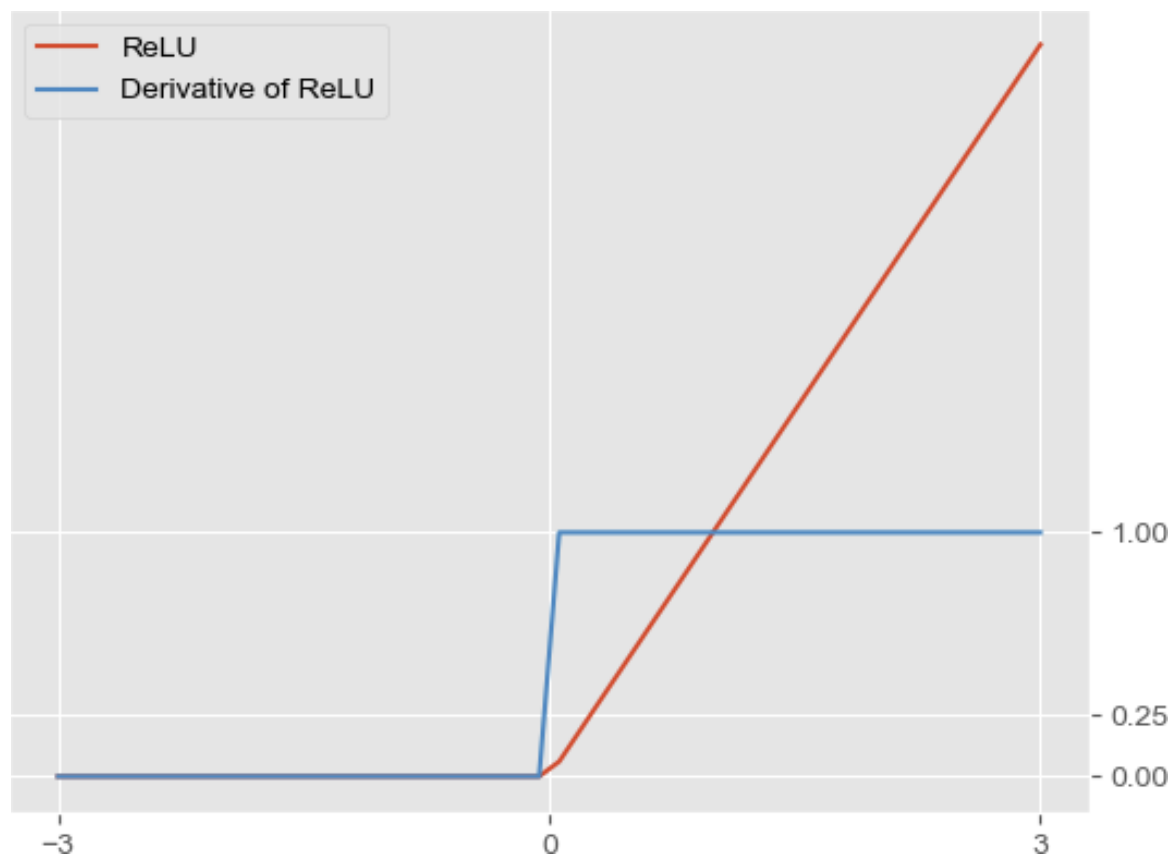
Better derivative

from a better function

- Therefore, the rectified linear unit, or ReLU, is commonly used

$$f(x) = \max(0, x)$$

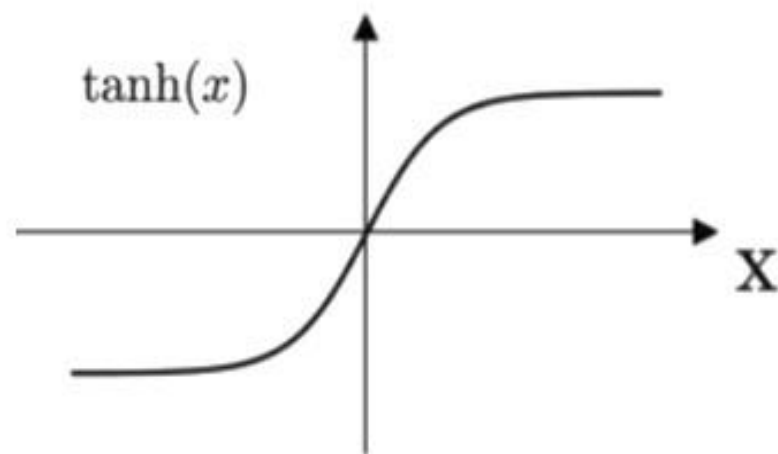
- The derivative is 1 when $x > 0$, and 0 when $x < 0$
- For active ReLU neurons, the activation derivative is 1. Therefore, ReLU does not introduce an additional shrinking factor during backpropagation
- As a result, ReLU helps earlier layers receive stronger updates and reduces the risk of vanishing gradients for active neurons



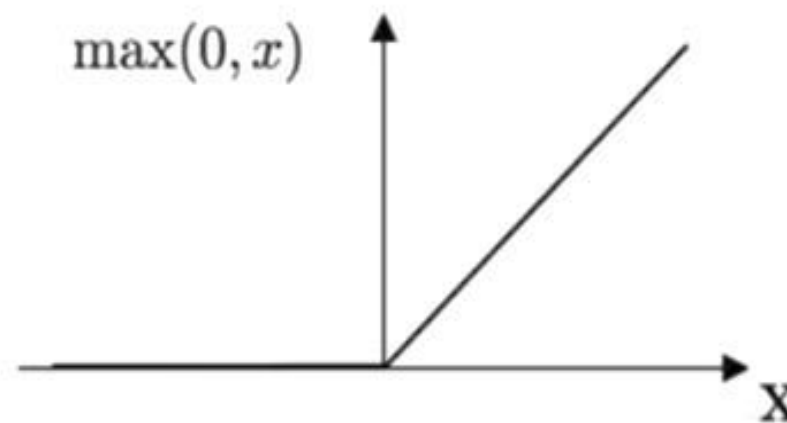
Coming late and why

- Earlier neural networks often used sigmoid or tanh because they were smooth and biologically inspired
- ReLU became popular when researchers started training deeper networks and realized that sigmoid and tanh caused serious vanishing-gradient issues.

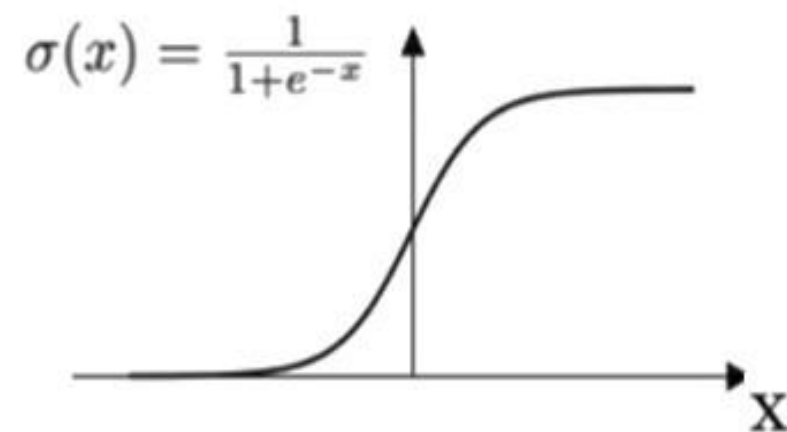
Tanh



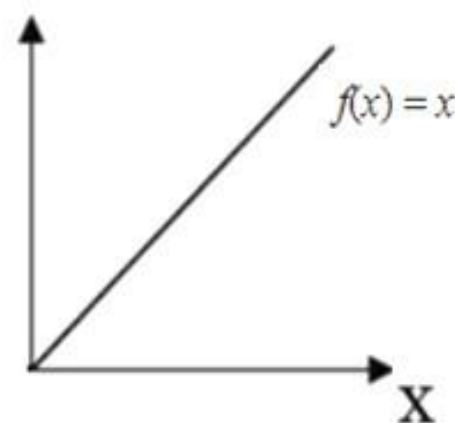
ReLU



Sigmoid



Linear

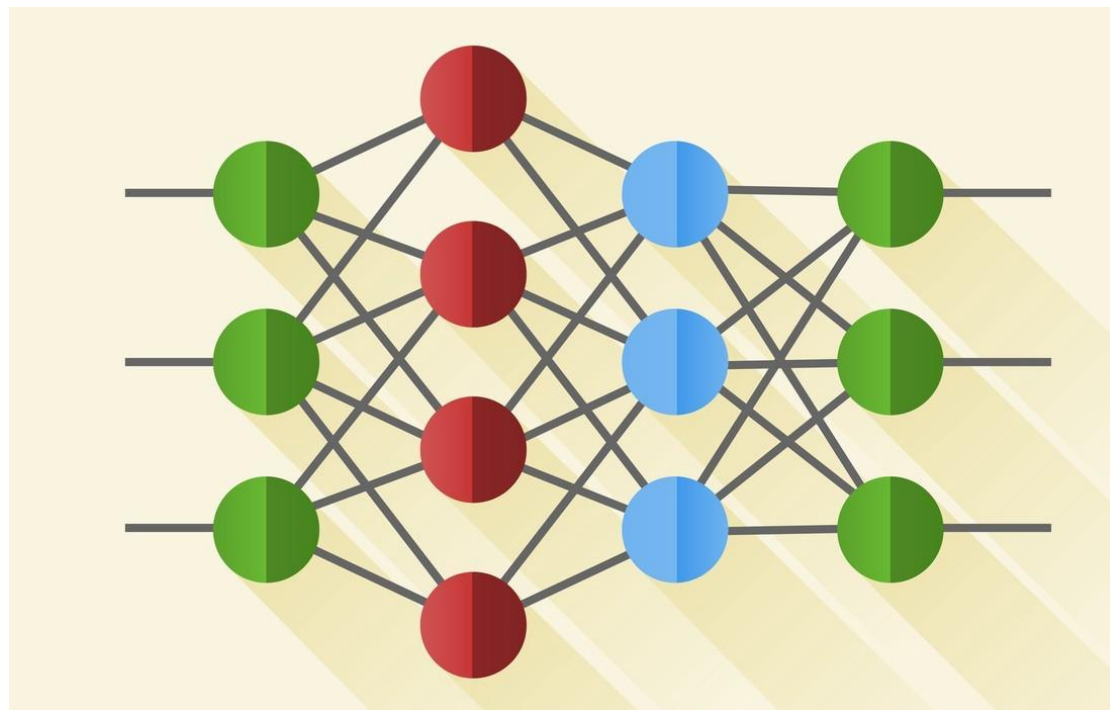


Second, initialize the weights properly

Initialization

Something to start with

- Need to give weights some values to start for the training
- Can't set all to 0, nothing will be learned in this case
- If weights are too small, signal shrinks as it passes through each layer; at later stage, neurons are 'dead', almost no activation coming out from neurons
- If weights are too large, signal grows massive as it passes through each layer; at later stage, neurons are 'saturated', network becomes unstable



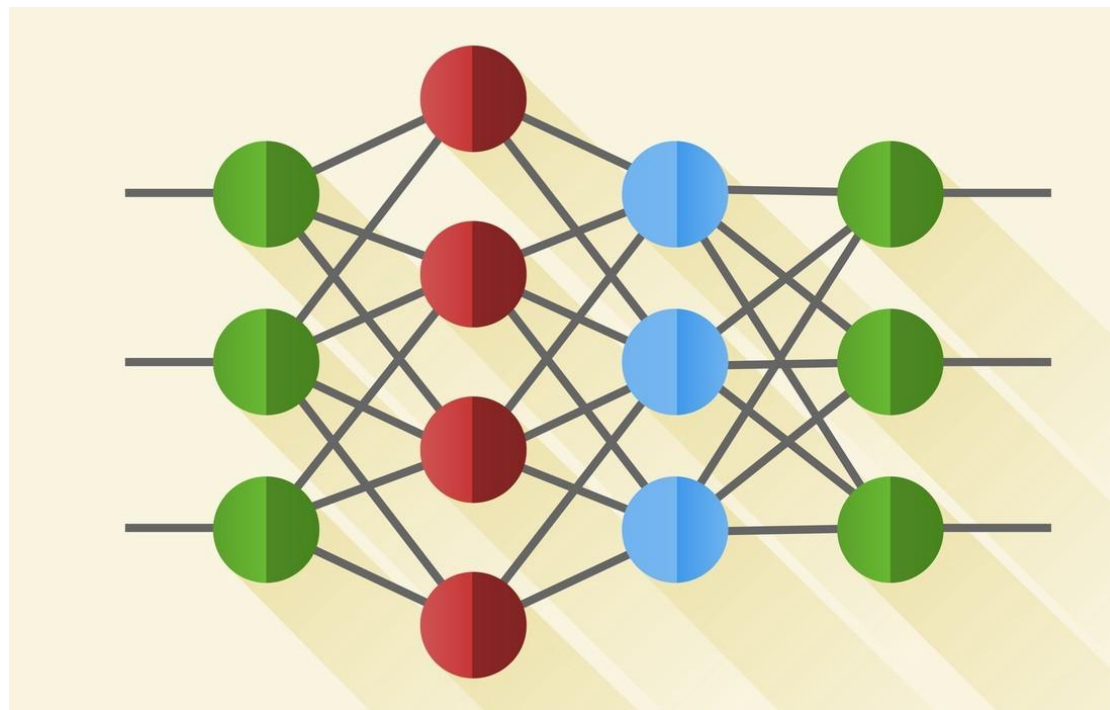
Source: <https://towardsdatascience.com/nns-aynk-c34efe37f15a>

Xavier initialization

proposed in 2010

- At that time designed for Sigmoid and tanh function
- Initialize biases to be 0, the weight of each layer is

$$w \sim U \left[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}} \right]$$



- n is the number of inputs to a neuron in a layer
- U is uniform distribution with a specified interval
- The weight is drawn randomly from the distribution

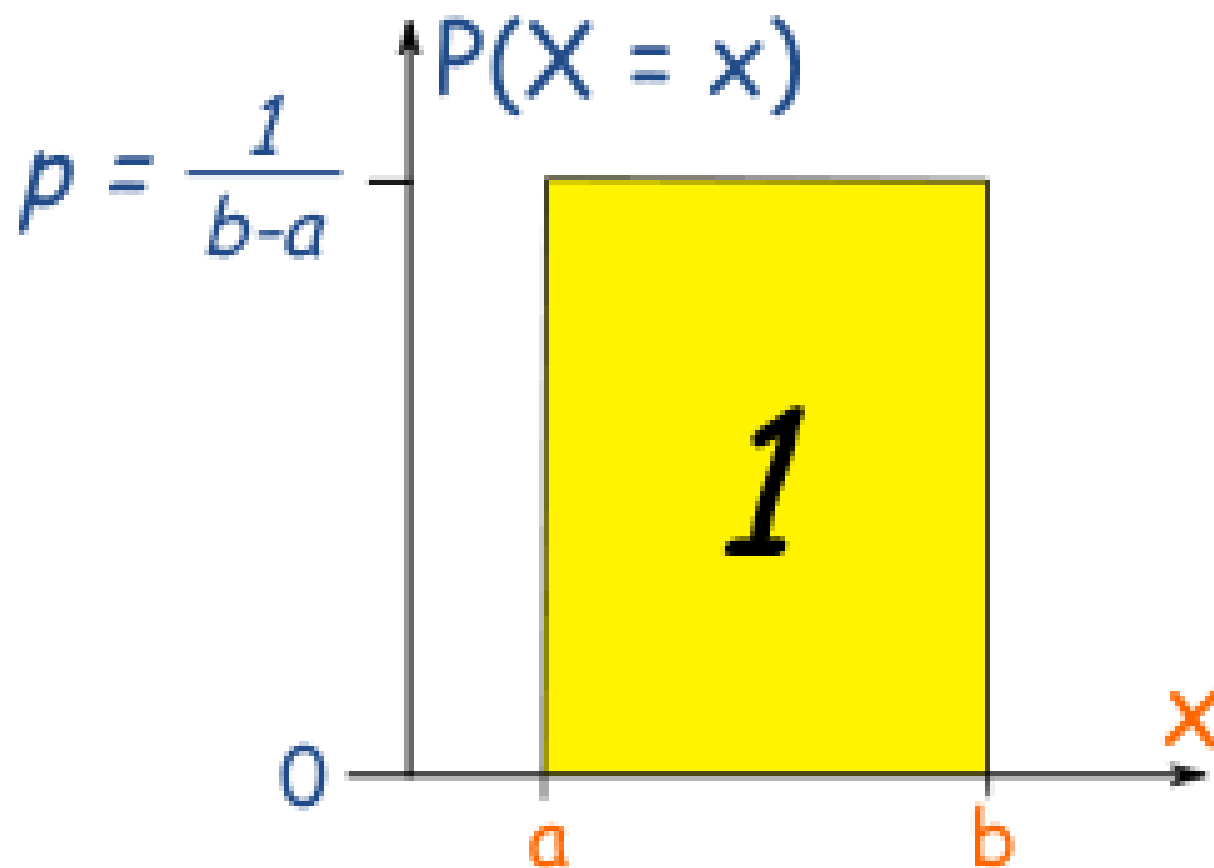
Source: <https://towardsdatascience.com/nns-aynk-c34efe37f15a>

Xavier initialization

Uniform distribution

- Uniform distribution: a distribution that has a constant probability
- In the case of Xavier initialization:

$$a = -\frac{1}{\sqrt{n}} \quad b = \frac{1}{\sqrt{n}}$$

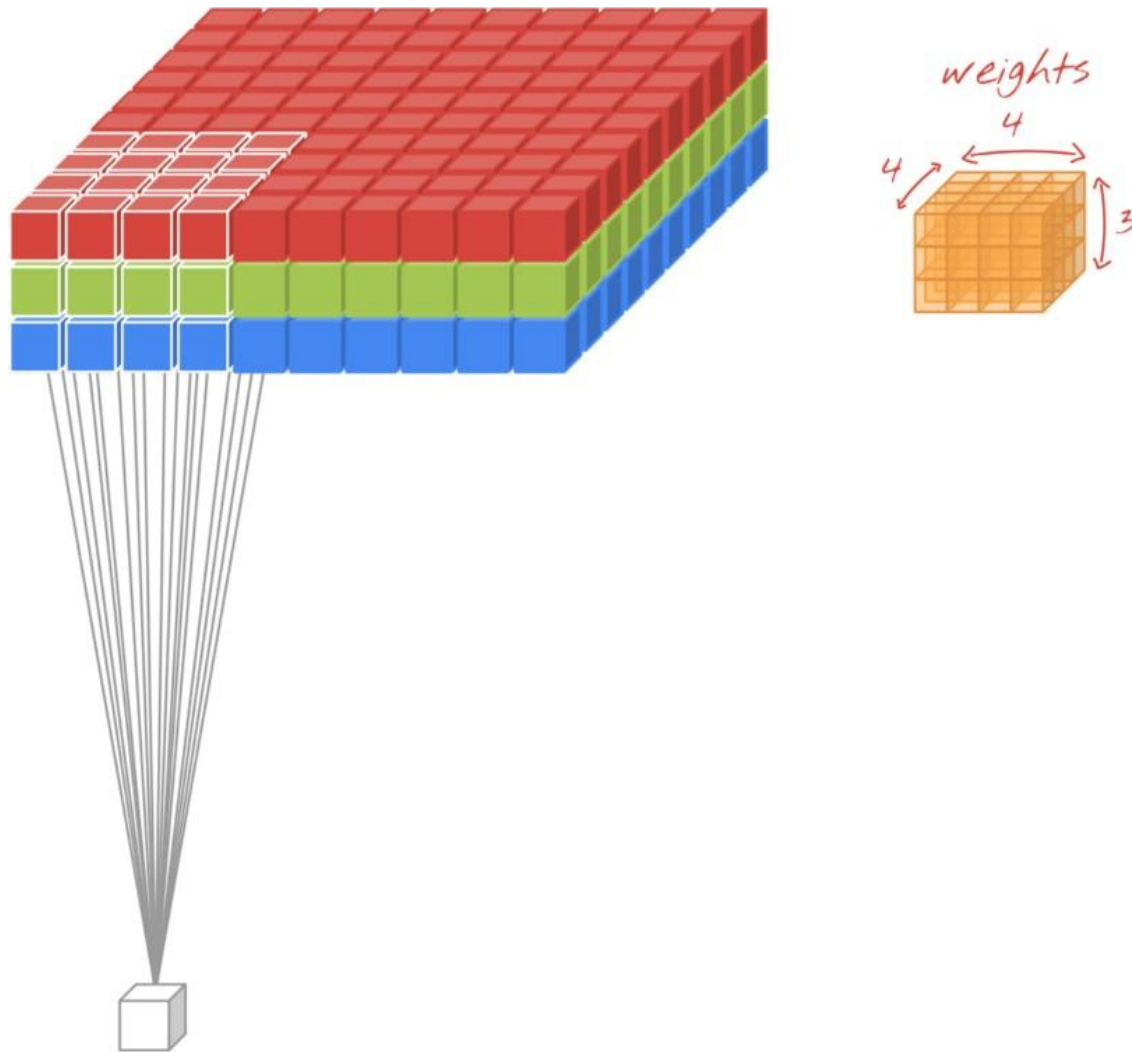


Source: <https://www.mathsisfun.com/data/random-variables-continuous.html>

The number of inputs?

To a neuron

- In literature the number of inputs to a neuron in a layer is called fan-in
- The number of inputs?

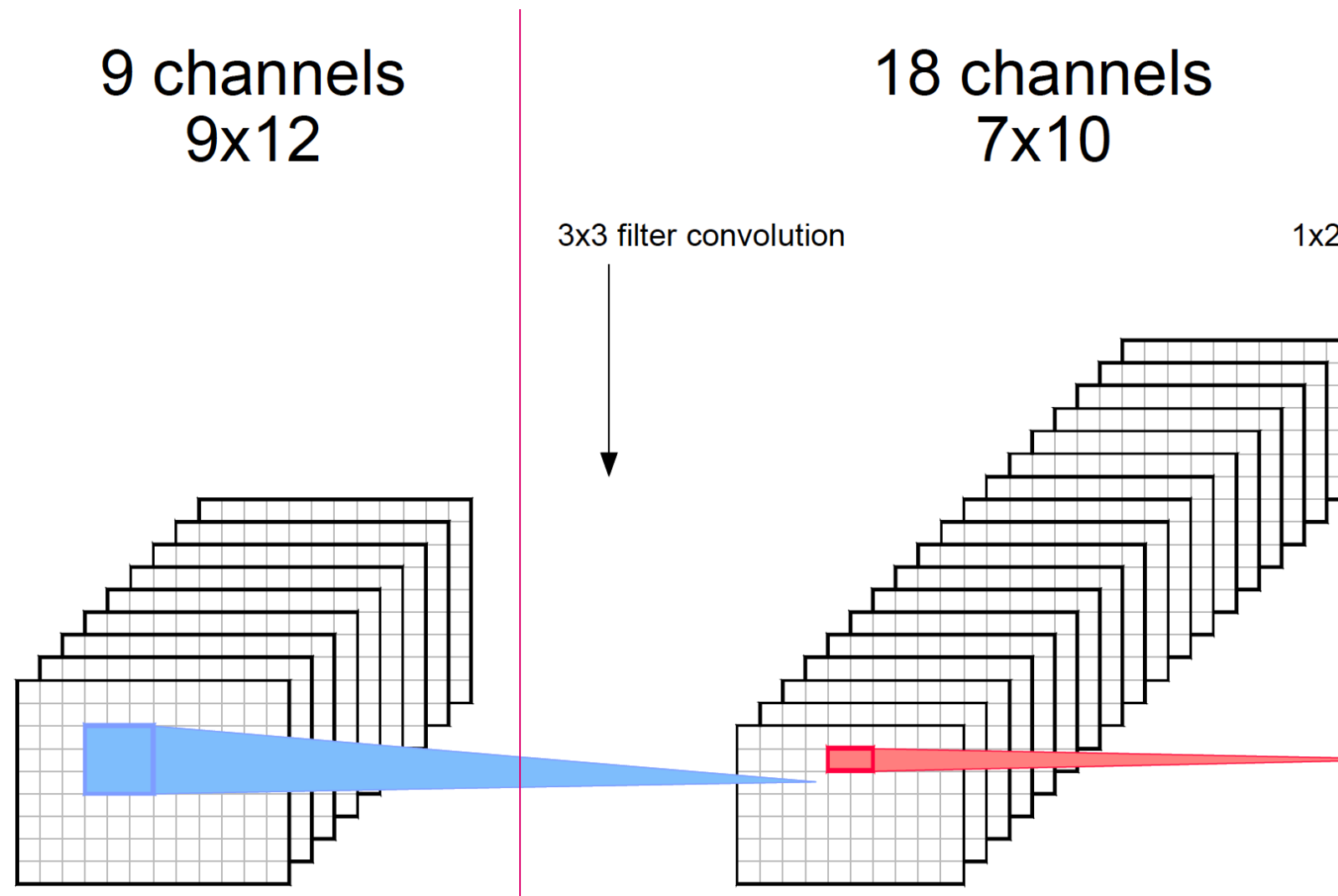


Source: https://twitter.com/martin_gorner

The number of inputs?

To a neuron

- What is the number of inputs to a neuron in the layer that has 18 channels?



Source: <https://towardsdatascience.com/nns-aynk-c34efe37f15a>

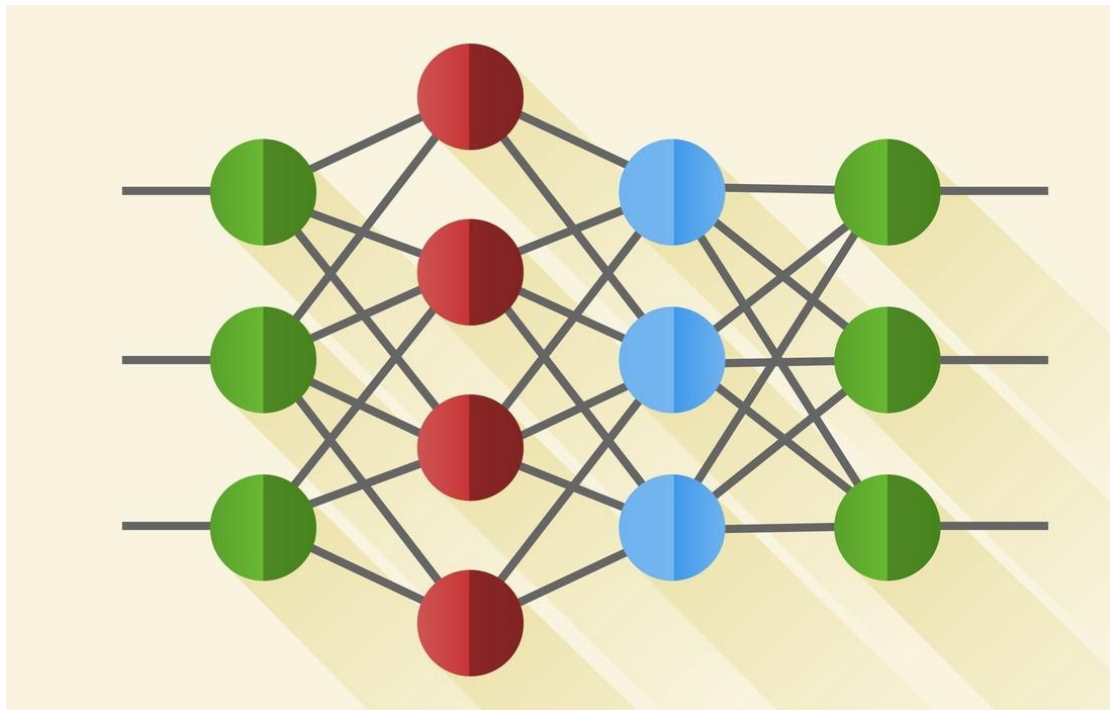
He initialization

proposed in 2015

- Designed for ReLU
- Initialize biases to be 0, the weight of each layer is

$$w \sim N\left(0, \frac{2}{n}\right)$$

- n is the number of inputs to a neuron in the layer
- N is a normal distribution with a zero mean and a variance of $2/n$

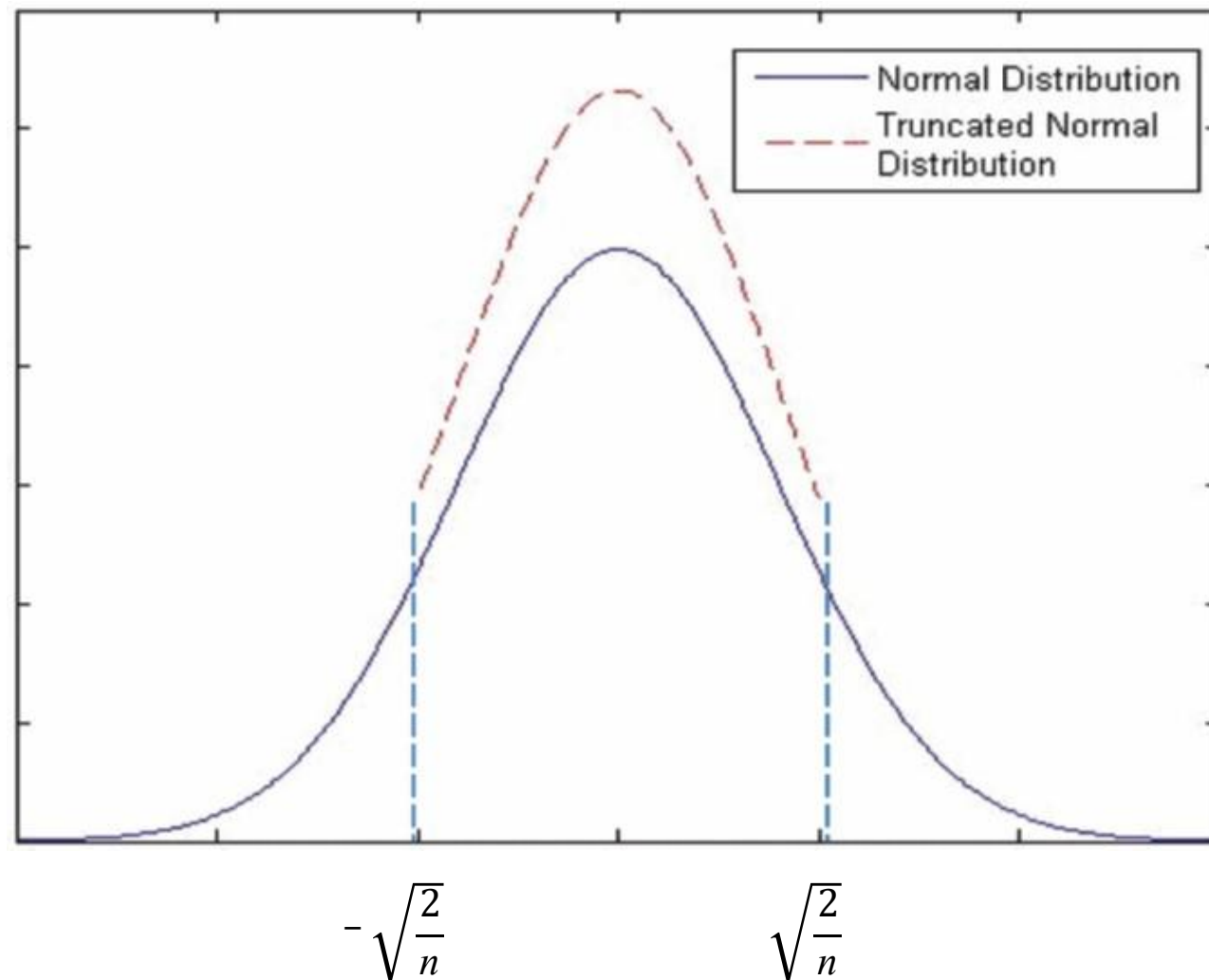


Source: <https://towardsdatascience.com/nns-aynk-c34efe37f15a>

He initialization

Truncated normal distribution

- In actual application, the normal distribution is truncated to avoid unnecessary large values

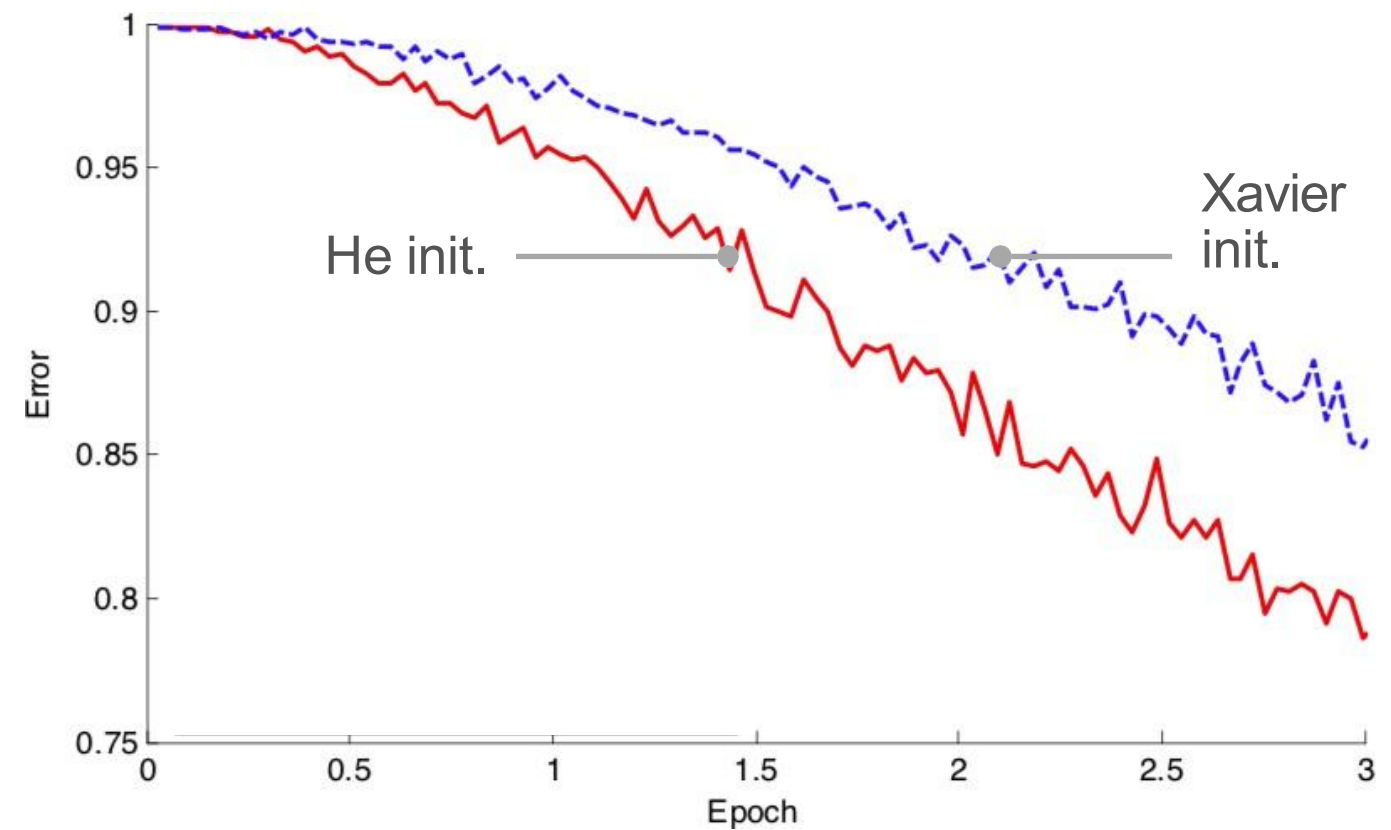


Source: doi: 10.3141/2315-07

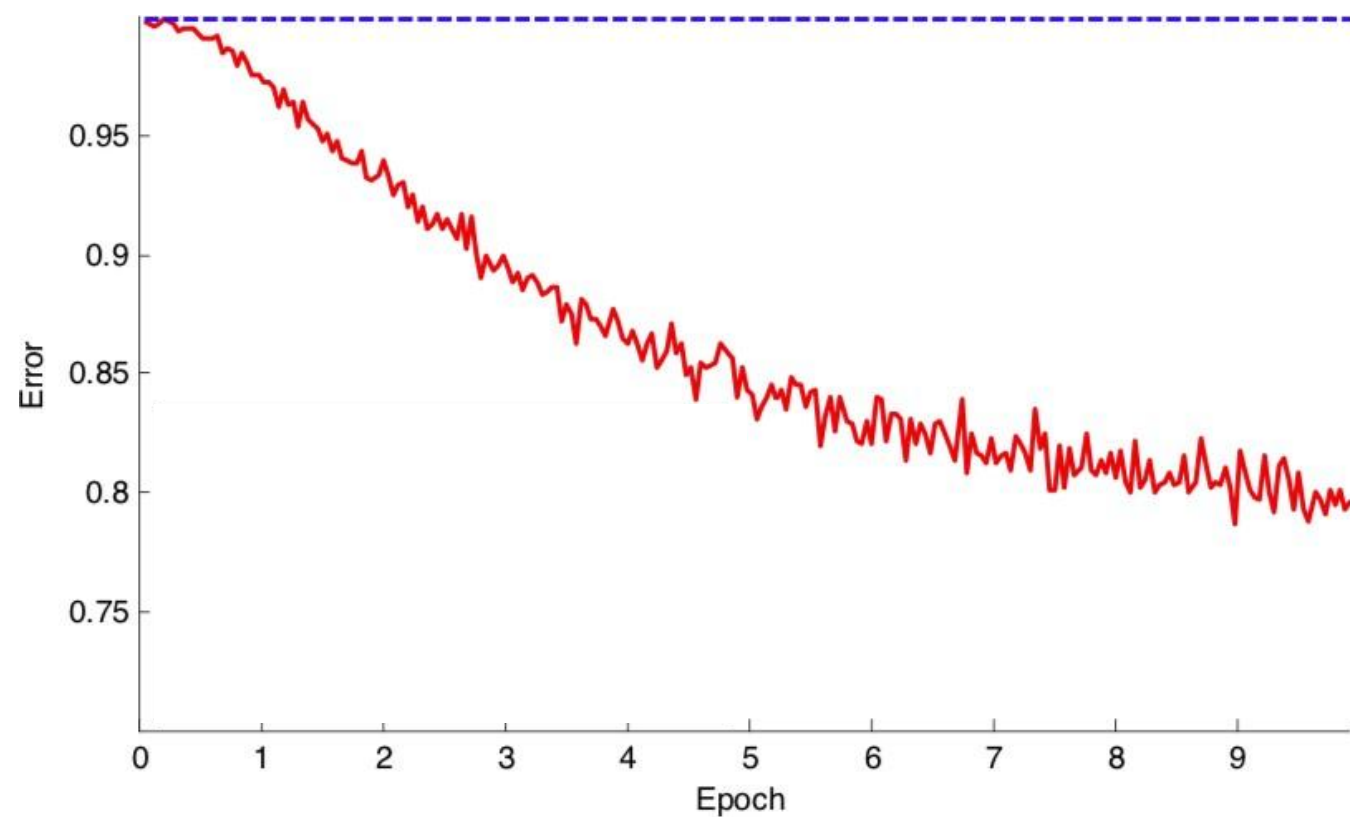
He initialization

proposed in 2015

22-layer
large model



30-layer
small model



Source: <https://arxiv.org/pdf/1502.01852.pdf>

Default in Keras

Initialization

- Keras by default uses Xavier initialization for weights, and zero initialization for biases
- See the code to make the change
- Note: Often, you need to experiment to see which setup works better; no hard rules!

```
> from tensorflow.keras.initializers import he_normal
```

```
> x = Dense(256,activation='relu',kernel_initializer=he_normal(33))(x)
```

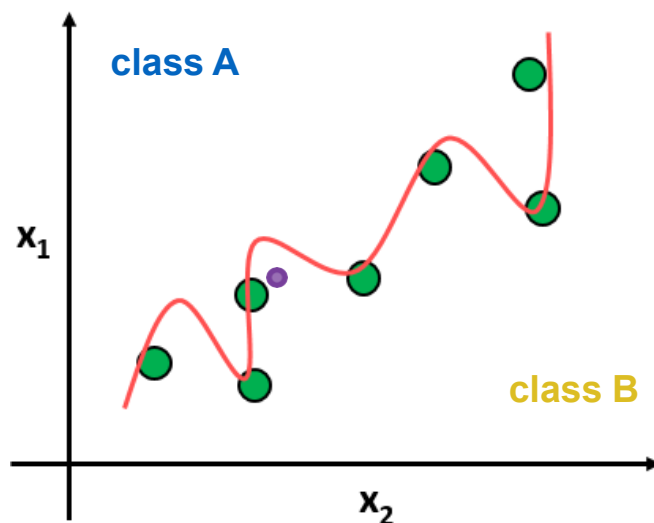
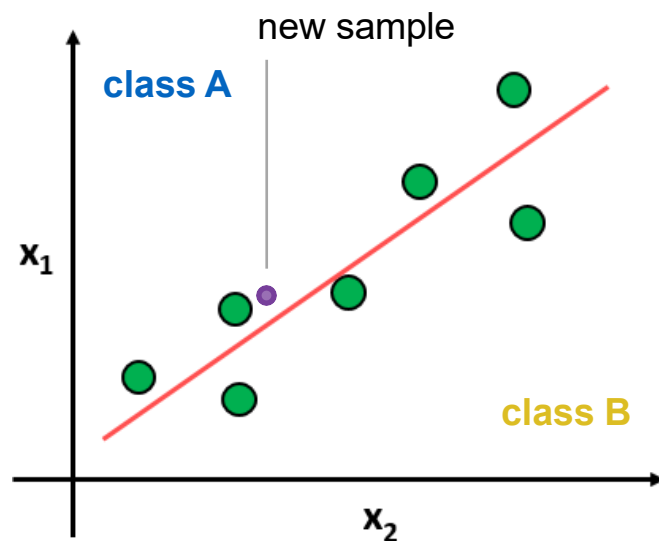
seed number

Third, restrict the weights

The problem with large values

The instability

- The longer we train, the more the weights adapt to training data
- Eventually, weights will grow in order to handle specific examples in training data
- But, large weights make networks unstable. Minor variation in input values or addition of noise to inputs will result in big differences in output
- Prefer simpler model with smaller weights

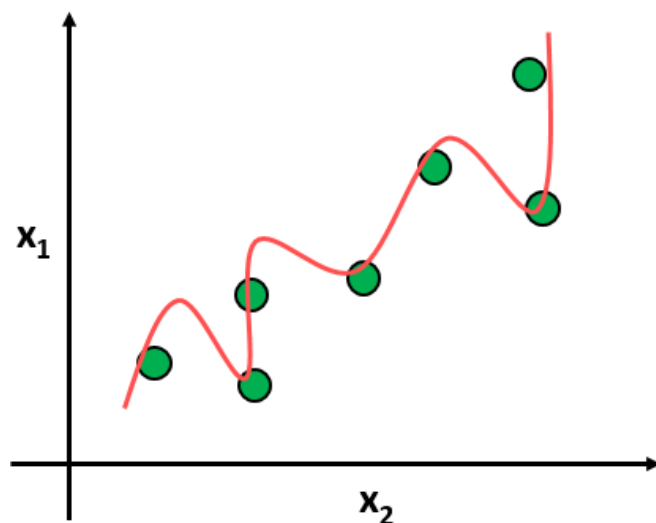
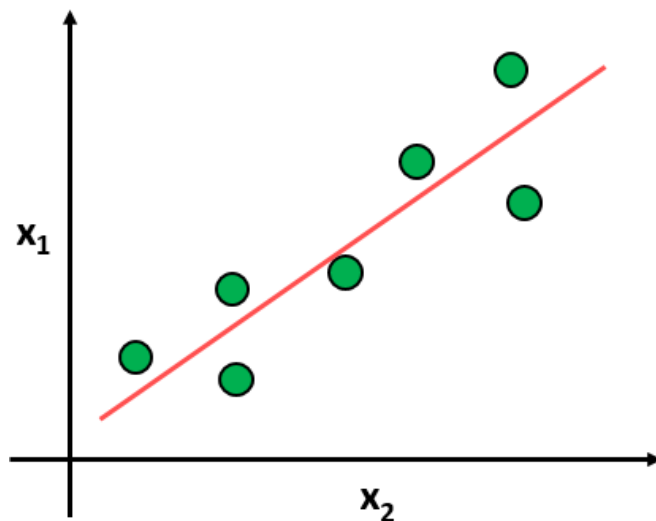


Source: <https://www.globalsoftwaresupport.com/regularization-machine-learning/>

The problem with large values

Regularization

- The magnitude of the weights must be controlled. This technique is called regularization
- Add a penalty to loss function; the penalty should be proportional to the magnitude of the weight
- L2 regularization is more often used; it calculates the sum of squared values of the weights
- L2 approach also called 'weight decay' in the field of neural networks, or 'shrinkage' in statistics



Source: <https://www.globalsoftwaresupport.com/regularization-machine-learning/>

L2 regularization

The math

- Let J be the original loss function, J_r the loss function with regularization, w the weight, η the learning rate, and α the parameter to control L2 regularization

The updated loss function

$$J_r = \frac{\alpha}{2} w^2 + J$$

The derivative

$$\frac{\partial J_r}{\partial w} = \alpha w + \frac{\partial J}{\partial w}$$

The algebraic expression to update the weights

$$w \leftarrow w - \eta \frac{\partial J_r}{\partial w}$$

The re-arrange update rule

$$w \leftarrow (1 - \alpha\eta)w - \eta \frac{\partial J}{\partial w}$$

Used in Keras

keras.regularizers

- In Keras it is easy to set up regularization
- Note: the penalty is applied on per-layer basis. Not all type of layers support regularization, and not all support it in the same way
- Dense, Conv1D, Conv2D and Conv3D have a unified API

```
> from tensorflow.keras import regularizers
```

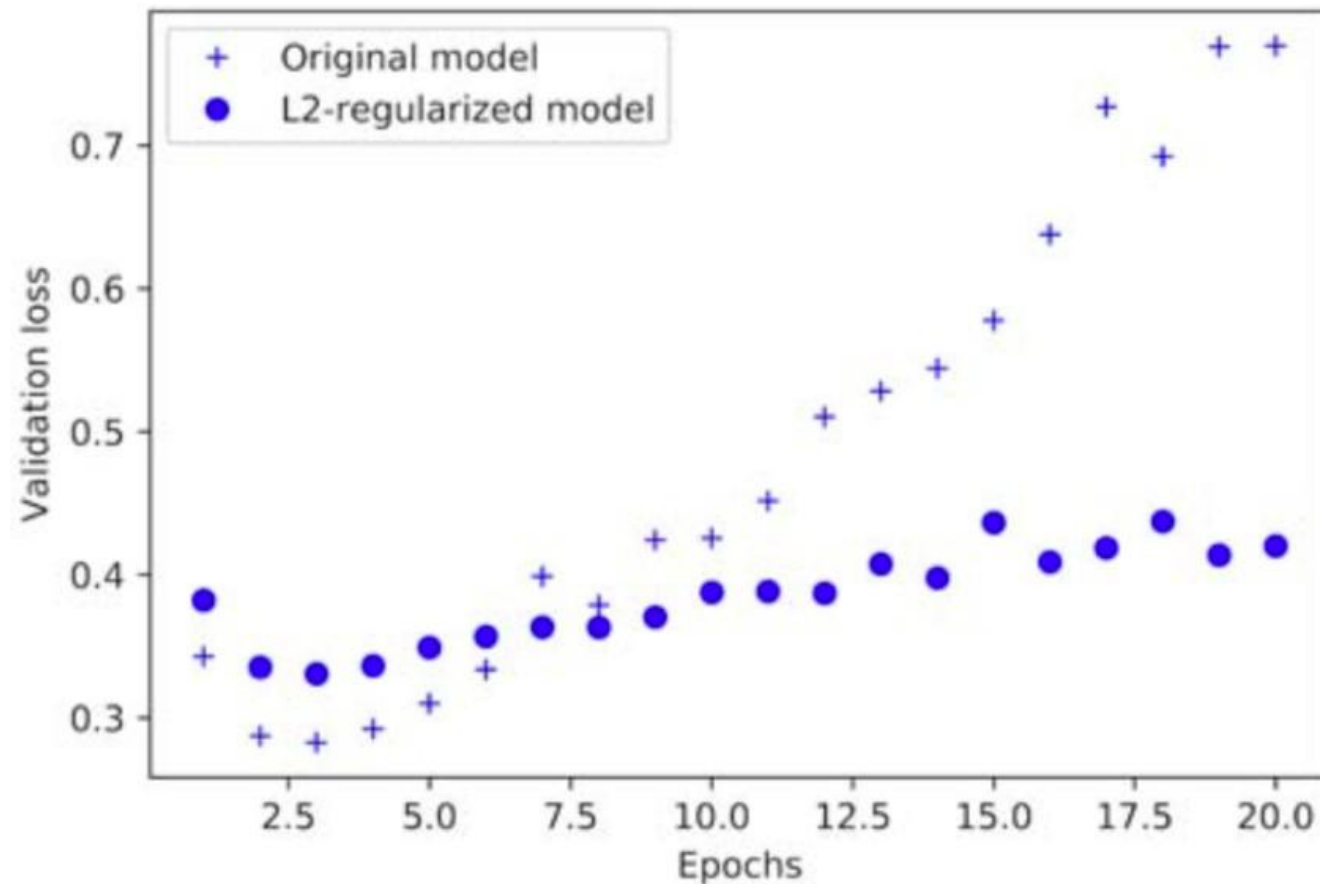
```
> x = Dense(256,activation='relu',kernel_regularizer=regularizers.l2(0.01))(x)
```

Must be a non-negative value.
Larger values apply a stronger penalty

Used in Keras

keras.regularizers

```
> from tensorflow.keras import regularizers  
  
> model = models.Sequential()  
> model.add(Dense(16,  
                  kernel_regularizer=regularizers.l2(0.001),  
                  activation='relu',  
                  input_shape=(10000,)))  
  
> model.add(Dense(16,  
                  kernel_regularizer=regularizers.l2(0.001),  
                  activation='relu'))  
  
> model.add(Dense(1, activation='sigmoid'))
```



Source: 'Deep learning with python' by Francois Chollet

**But, even with these, we still have
problems**

Overfitting

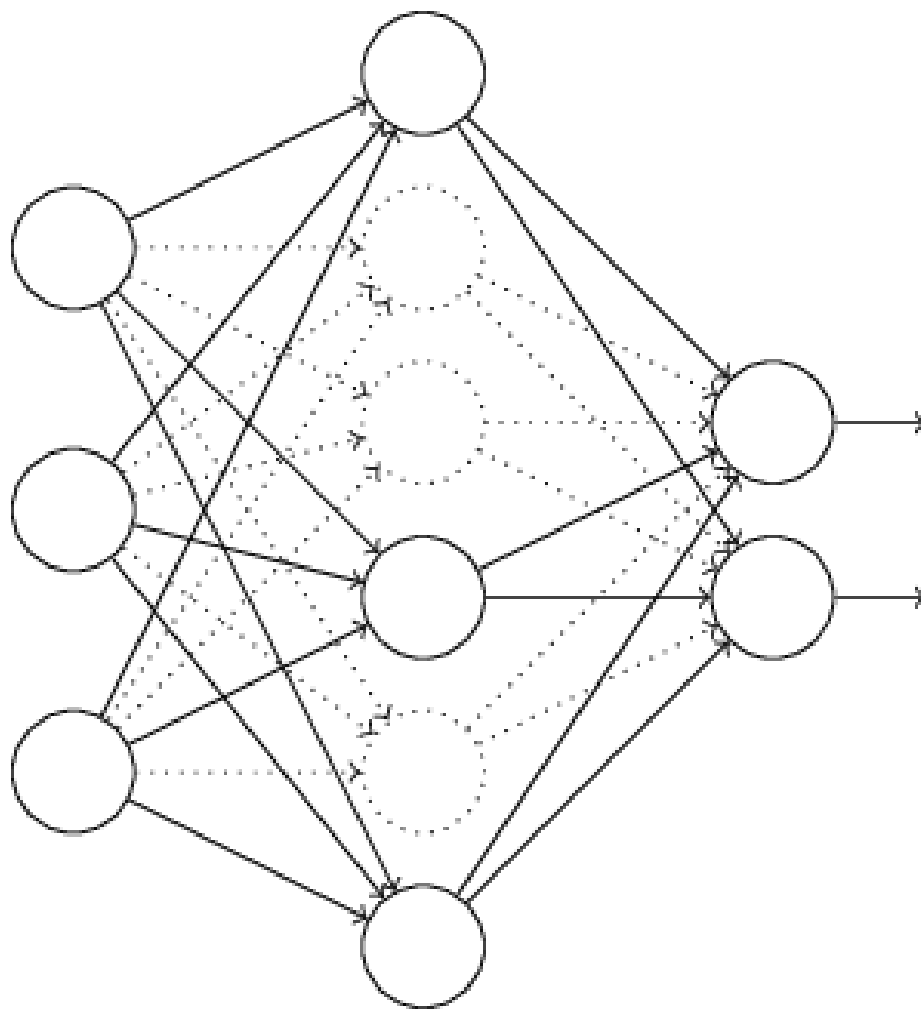
The pain, always

- Deep, large neural networks on small dataset: most of the time, they learn more on statistical noise in the data rather than key features
- Consequence: when new data come in, error increases; poor generalization
- Possible solutions: get many many neural network trained on same dataset. To perform classification, do average on prediction from each model (ensemble)
- However, not feasible in practice

Dropout

proposed in 2014

- Randomly dropping out neurons in the training phase. The 'dropping out' can be performed simply by setting the activation output from a neuron to zero



Source: <https://kharshit.github.io/blog/2018/05/04/dropout-prevent-overfitting>

Dropout

proposed in 2014

- The idea came out by Geoffrey Hinton, inspired by a fraud-prevention mechanism used by banks

- In his own words,

I went to my bank. The tellers kept changing and I asked one of them why. He said he didn't know but they got moved around a lot. I figured it must be because it would require cooperation between employees to successfully defraud the bank. This made me realize that randomly removing a different subset of neurons on each example would prevent conspiracies and thus reduce overfitting.

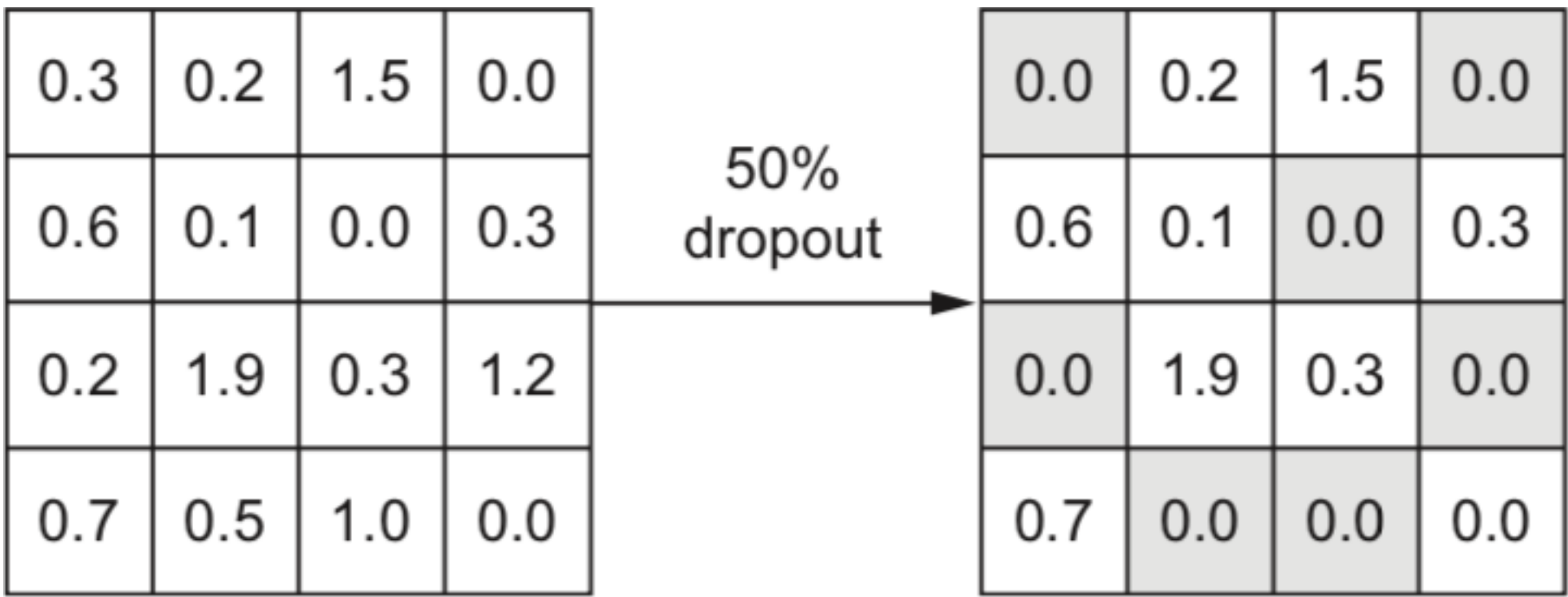


Source: <https://torontolife.com/tech/ai-superstars-google-facebook-apple-studied-guy/>

Dropout

In the form of tensor

- Note: the zeros will be applied to different neurons in different training epoch
- But the zeros are applied only during training phase, not in testing / validation



Source: 'Deep learning with python' by Francois Chollet

Dropout in Keras

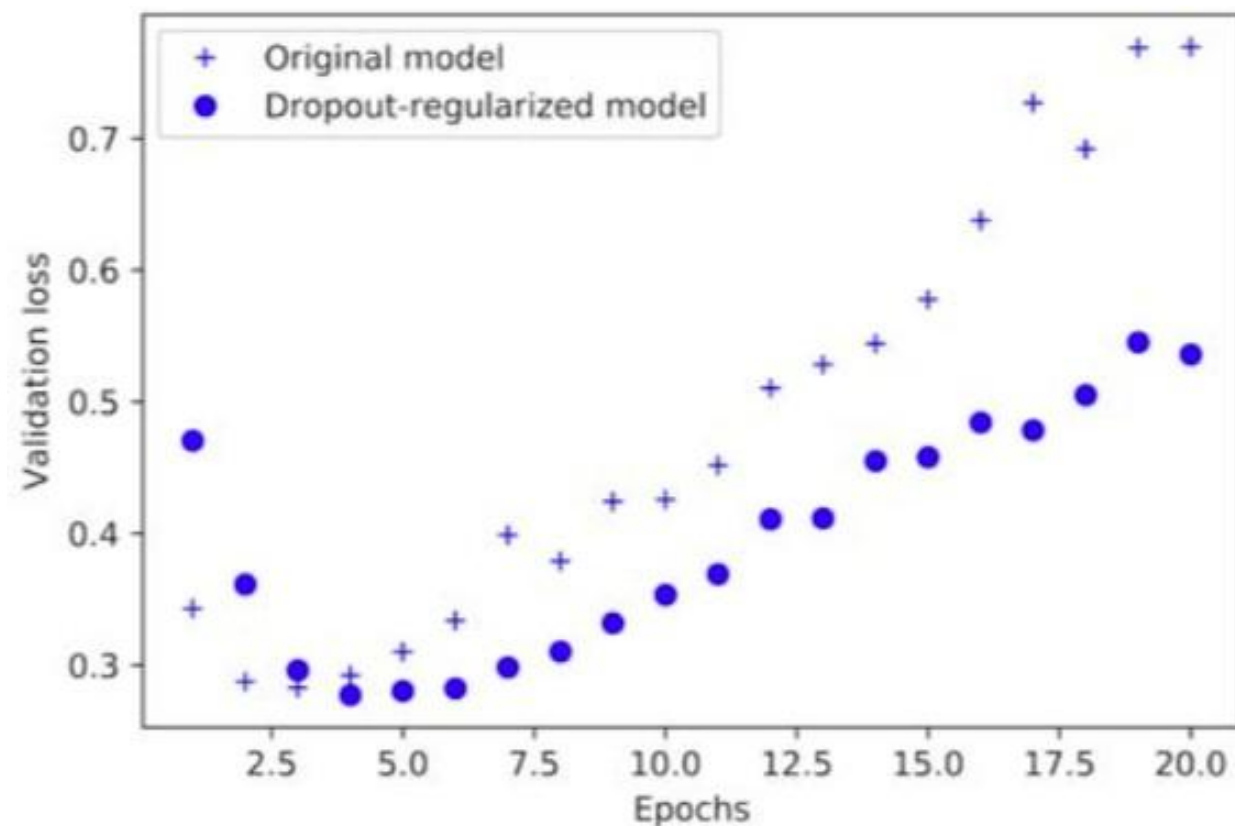
comparison

```
> from tensorflow.keras.layers import Dropout

> model = models.Sequential()
> model.add(Dense(16,
                  activation='relu',
                  input_shape=(10000,)))

> model.add(Dropout(0.5))
> model.add(Dense(16,
                  activation='relu'))

> model.add(Dropout(0.5))
> model.add(Dense(1, activation='sigmoid'))
```



Source: 'Deep learning with python' by Francois Chollet

A short summary ...

How to prevent overfitting

- The training of a deep neural network is a long march to fight overfitting
- So far we have these techniques to use:
 1. Choose the right activation function
 2. Use the suitable initialization
 3. Regularize the weights
 4. Add dropout between layers
- What else?

Batch normalization

Normalization...?

- Assume we have this five numbers

$$\mathbf{x} = [1.6, -0.5, 0.3, 0.4, 0.9]$$

- First we check the mean and the standard deviation, and get

$$\mu_{\mathbf{x}} = 0.54 \qquad \sigma_{\mathbf{x}} = 0.69$$

- For each value in the above vector, we perform

$$y_i = \frac{x_i - \mu_{\mathbf{x}}}{\sigma_{\mathbf{x}}}$$

- We get

$$\mathbf{y} = [1.53, -1.50, -0.35, -0.20, 0.52]$$



Source: <https://unsplash.com/photos/5lHz5WhosQE/>

Batch normalization

Normalization or
standardization...?

- From

$$\mathbf{x} = [1.6, -0.5, 0.3, 0.4, 0.9]$$

- with the properties

$$\mu_{\mathbf{x}} = 0.54 \qquad \sigma_{\mathbf{x}} = 0.69$$

- to

$$\mathbf{y} = [1.53, -1.50, -0.35, -0.20, 0.52]$$

- What's the big deal? Why do we do that?

- What is the mean and standard deviation of the new vector?



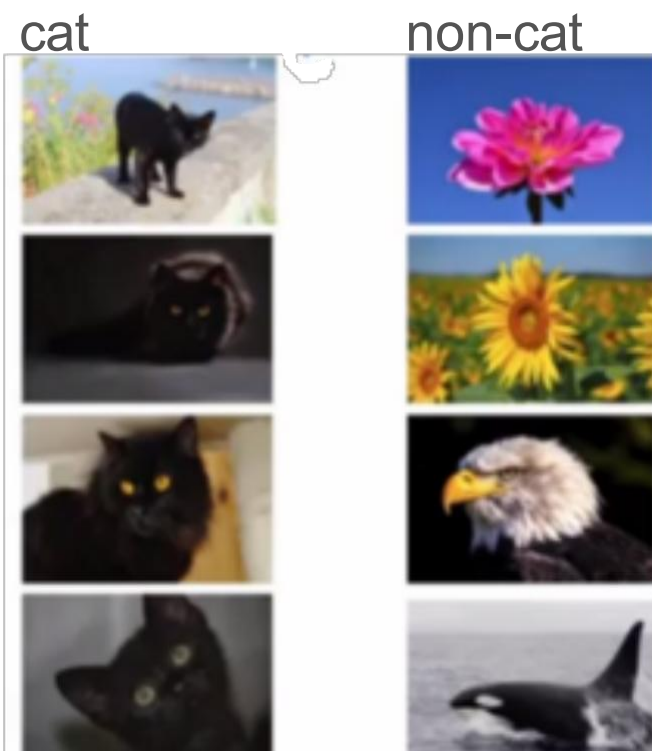
Source: <https://unsplash.com/photos/5lHz5WhosQE/>

Batch normalization

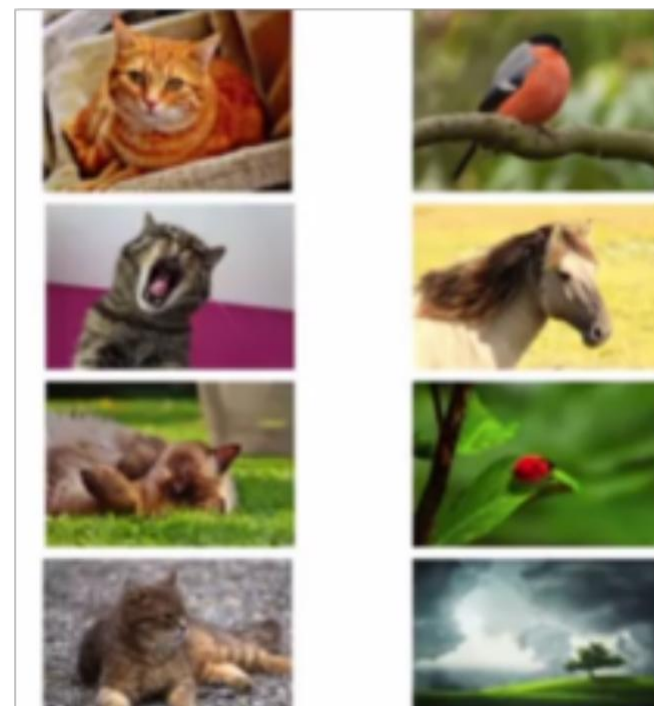
Dataset shift

- Covariate shift: The distribution of the data changes.
- Domain adaptation can help, a bit.

Train and validate on
this data set



But encounter this in actual
prediction, what would happen?

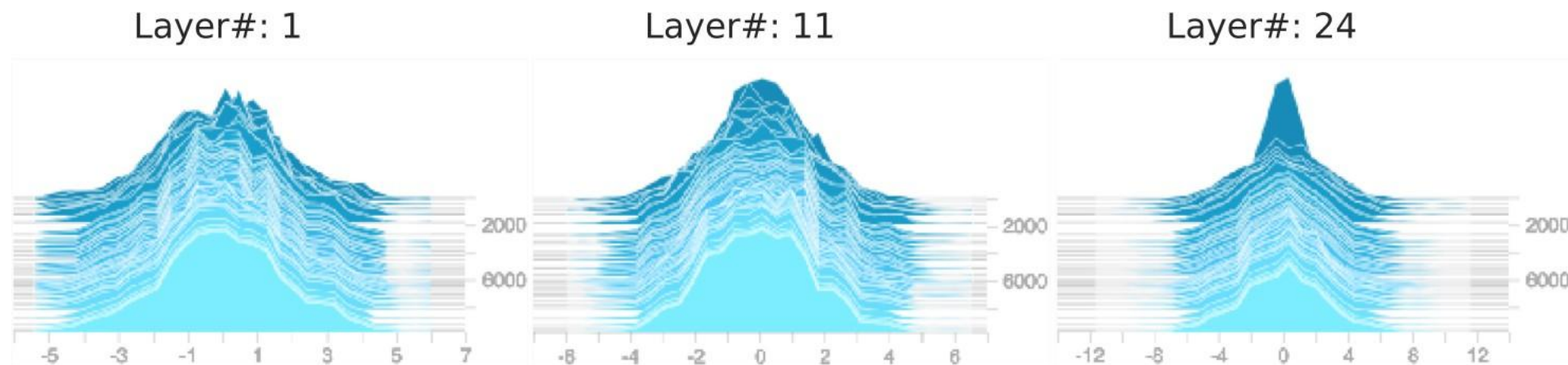


Source: <https://www.youtube.com/watch?v=nUUqwaxLnWs>

Batch normalization

Internal covariate shift

- In training deep learning model, for a layer, gradients are used to update parameters, under the assumption the other layers do not change
- In practice, all the layers are updated simultaneously
- The layers' output distribution can change as a consequence



Source: <https://arxiv.org/pdf/1805.11604.pdf>

Batch normalization

The solution

- Let v be the output from a neuron; we denote v_i as one of the neuron outputs from an input in a batch

The batch size is m

$$B = [v_1, v_2, \dots, v_i, \dots, v_m]$$

The batch mean for v

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m v_i$$

The batch variance for v

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (v_i - \mu_B)^2$$

Perform normalization on every item

$$\hat{v}_i \leftarrow \frac{v_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

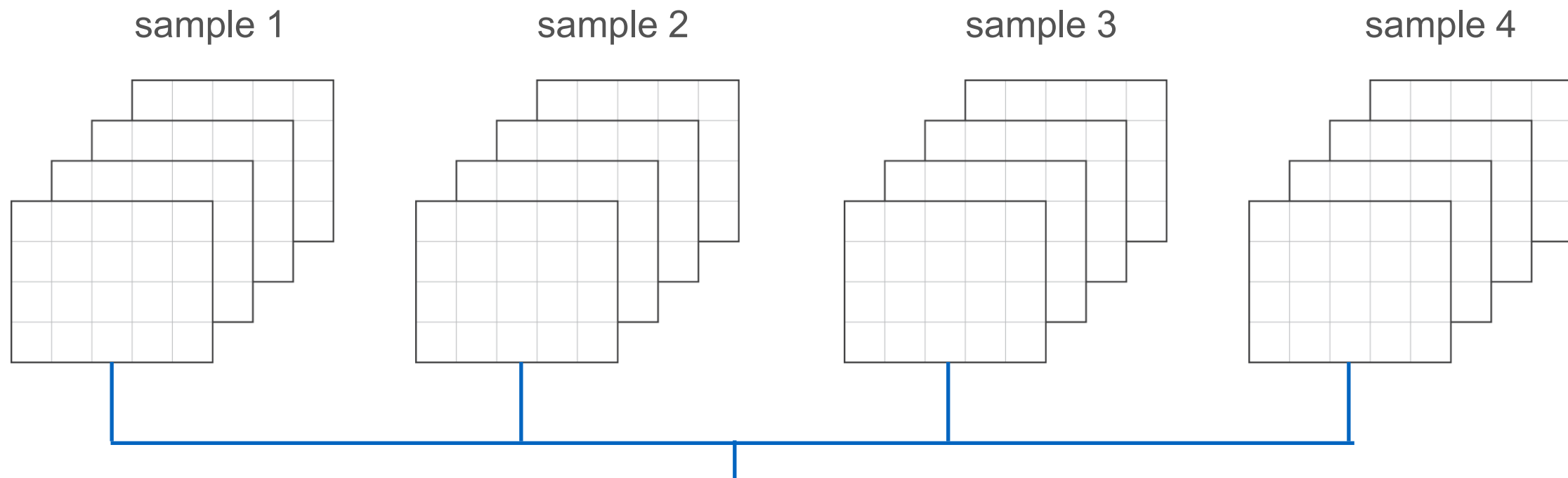
Scale and shift the output, γ and β are trainable parameters; u_i is the normalized output

$$u_i \leftarrow \gamma \hat{v}_i + \beta$$

Batch normalization

on multiple feature maps

- Assume the layer 4 of a particular network generates 4 channel (feature maps), and during training, the batch size is 4
- The batch normalization is performed channel by channel



Batch normalization is performed channel by channel. It starts with channel 1. All the elements in channel 1 (of 4 samples) form the v_i and normalized accordingly

Used in Keras

`keras.layers.BatchNormalization`

- In Keras adding batch normalization is easy
- By default normalization is performed on last axis (`axis = -1`)
- No additional argument is needed (usually)
- Note: the μ and σ are counted as non-trainable parameters, γ and β are trainable parameters

```
> from tensorflow.keras.layers import BatchNormalization
```

```
> x = BatchNormalization()(x)
```

Used in Keras

keras.layers.BatchNormalization

- In many situations, batch normalization is performed before activation

```
> from tensorflow.keras.layers import BatchNormalization
> from tensorflow.keras.layers import Conv2D
> from tensorflow.keras.layers import Activation

> x = Conv2D(32,(3,3),padding='same')(x)
> x = BatchNormalization()(x)
> x = Activation('relu')(x)
```

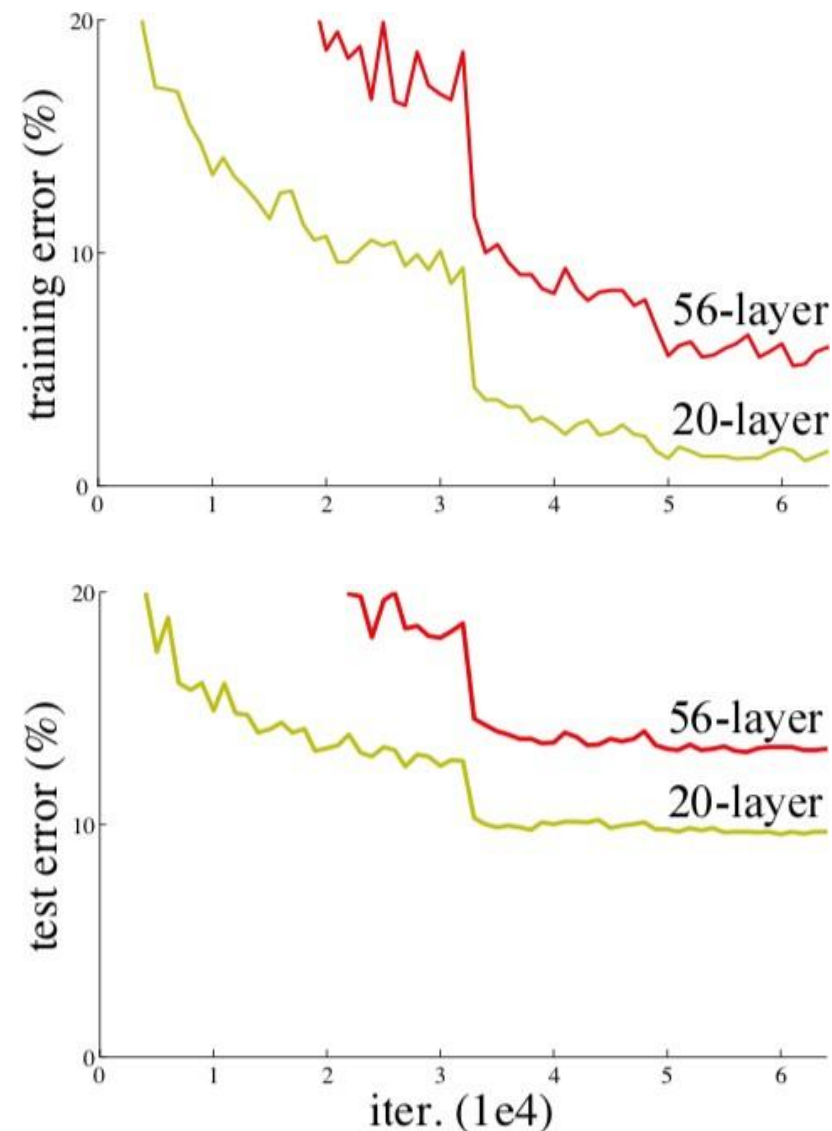
A thousand layers

Possible...?

- Techniques available:
 1. Choose the right activation function
 2. Use the suitable initialization
 3. Regularize the weights
 4. Add dropout between layers
 5. Batch normalization
- Given the available techniques, do you think we can train a 1000-layer network?
- Is vanishing gradient still an issue?

Unexpected issue

only in deep deep network

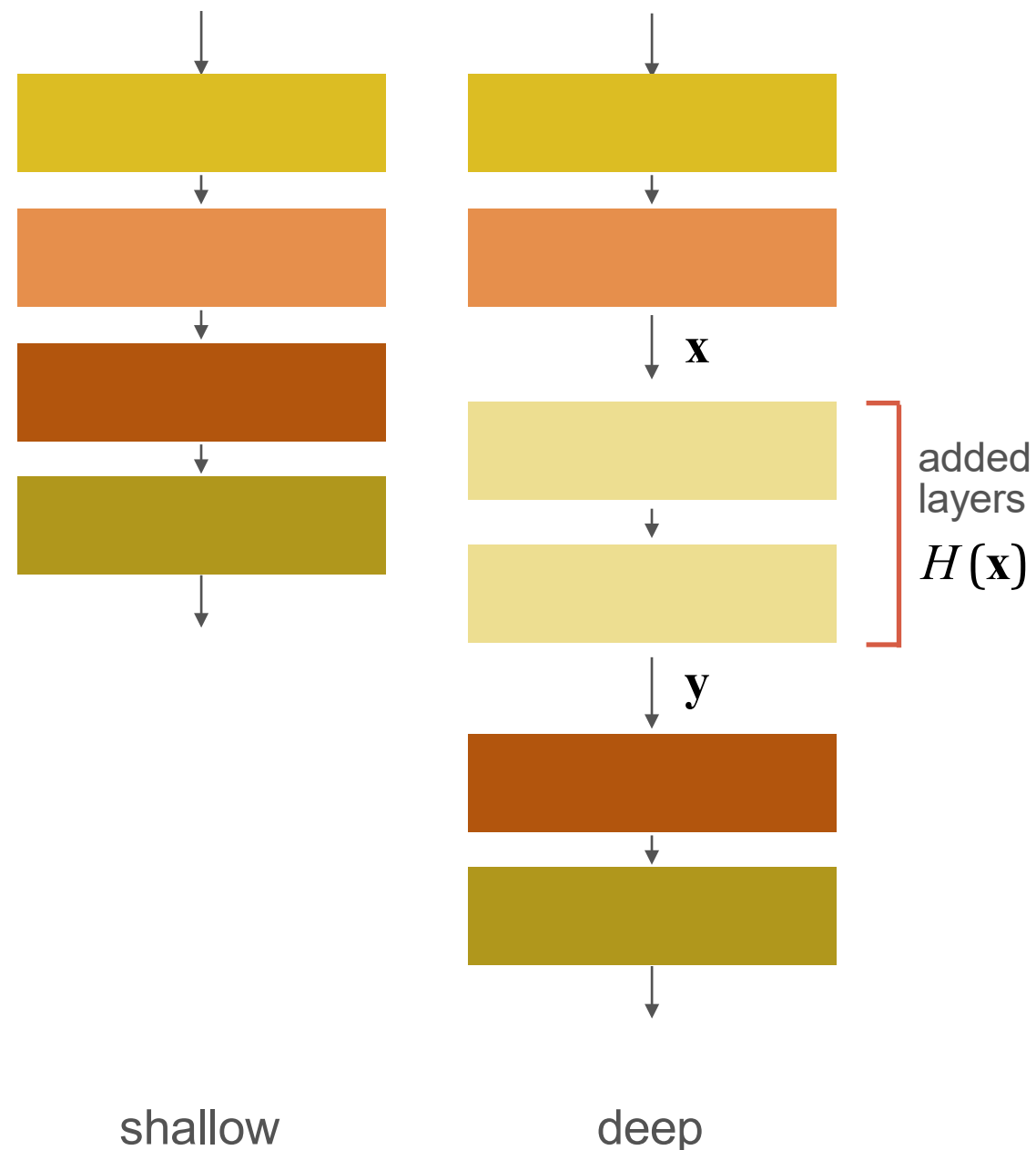


Source: <https://arxiv.org/pdf/1512.03385.pdf>

- Normalization and proper initialization have somewhat solved the problem of vanishing gradient
- But in very deep network, **degradation** happens: accuracy gets saturated and then degrades
- And, **degradation** *is not caused by overfitting*; the deep model *simply just* has higher training error
- Note: In the case of overfitting, the training error is **lower** (overfit on the training data set) yet the testing error is **higher** (cannot be generalized)

The equal

between shallow and deep

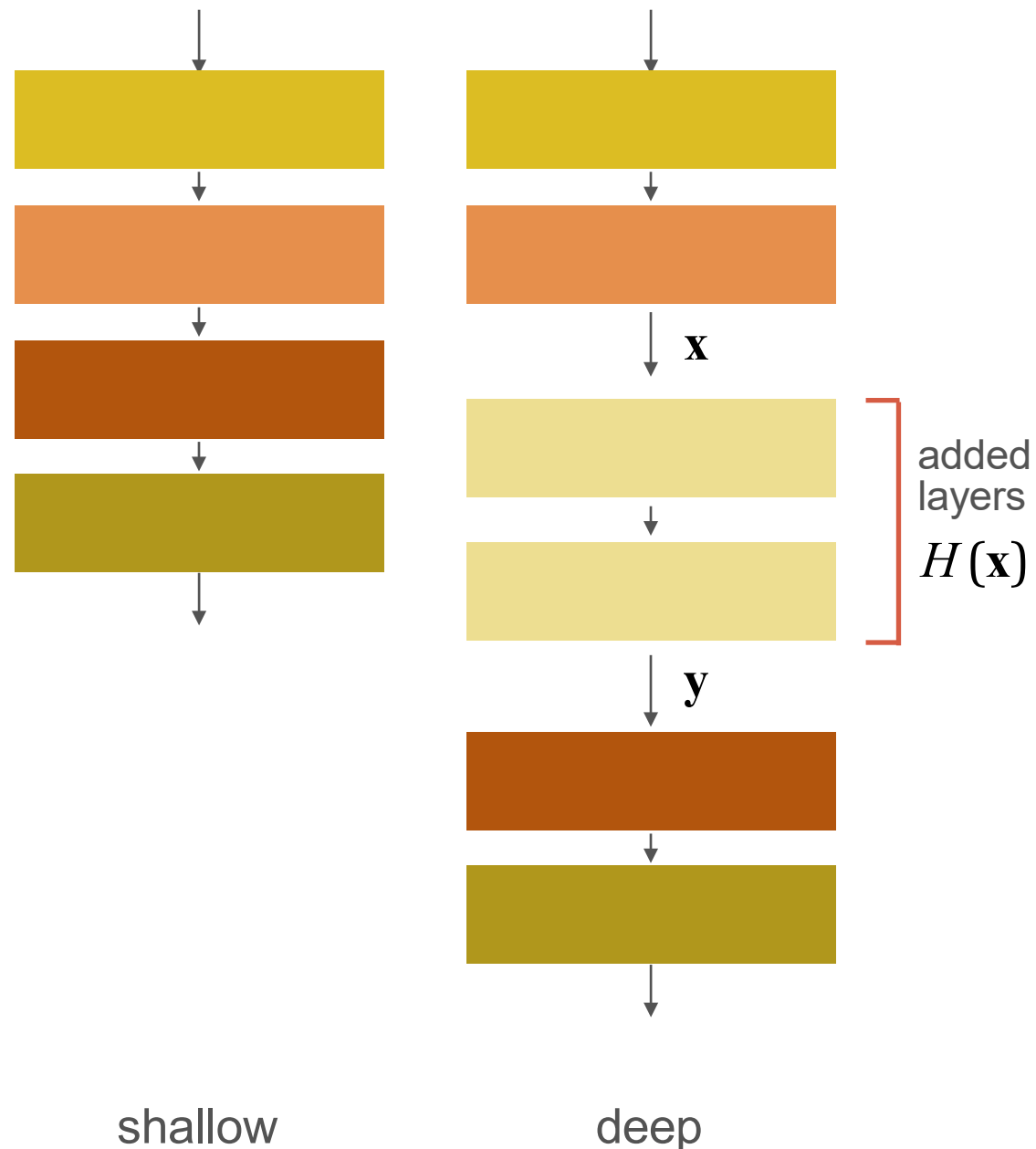


- But deep networks should not perform worse than shallow networks
- Why? Assume the added layers simply perform an **identity mapping**, then structure wise, both shallow and deep nets are **in fact the same**
- This implies the layers should perform the below

$$\begin{aligned} \mathbf{y} &= H(\mathbf{x}) \\ &= \mathbf{x} \end{aligned}$$

Conclusion?

between shallow and deep



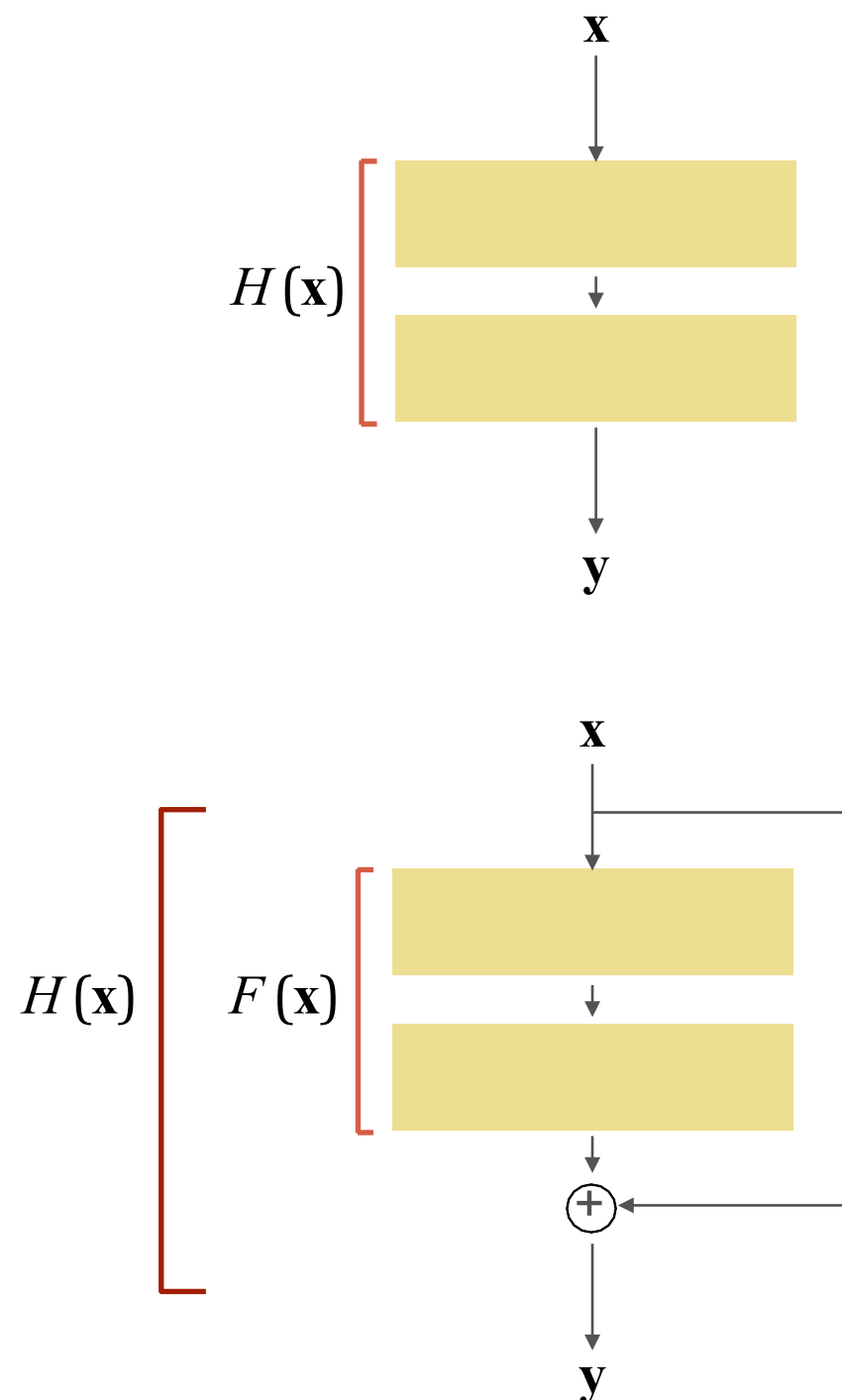
- But since the training error from a deeper network is higher, it is safe to say that, the added layers do not perform identity mapping
- Conclusion: it is very hard to get layers trained to perform identity mapping

$$\mathbf{y} = H(\mathbf{x}) \\ = \mathbf{x}$$

- But to get the network goes deeper, we need layers that can be trained to at least perform identity mapping, so that it will not perform worse than a shallow network
- A solution is needed

The solution

For the depth



- We desire to have an $H(\mathbf{x})$ that can perform identity mapping

$$\begin{aligned} \mathbf{y} &= H(\mathbf{x}) \\ &= \mathbf{x} \end{aligned}$$

- On the left, we know the top structure won't work. He et al. however, proposed that the bottom structure should work. Why?

$$\begin{aligned} \mathbf{y} &= H(\mathbf{x}) \\ &= F(\mathbf{x}) + \mathbf{x} \end{aligned}$$

- When $F(\mathbf{x})$ approaches 0, then we have the identity mapping
- It is easier to get non-linear layers pushed to values of 0 rather than values of 1 (remember vanishing gradient?)

Residual layer

The solution

- Since we have

$$\begin{aligned} \mathbf{y} &= H(\mathbf{x}) \\ &= F(\mathbf{x}) + \mathbf{x} \end{aligned}$$

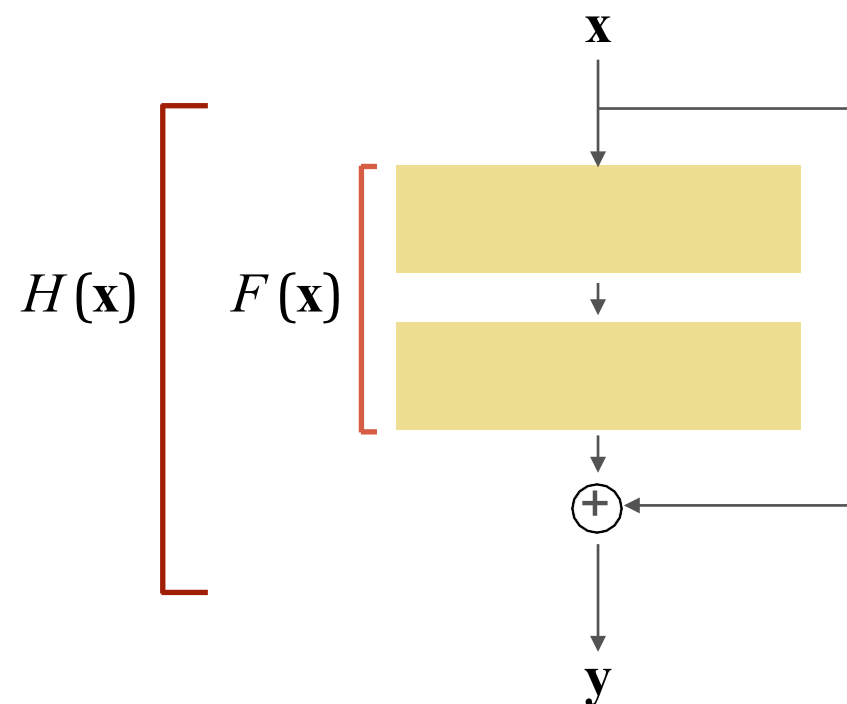
- Re-arrange the equation, we have

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

- Re-arrange the equation again and we get

$$F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$$

- This implies that $F(\mathbf{x})$ is doing a residual mapping

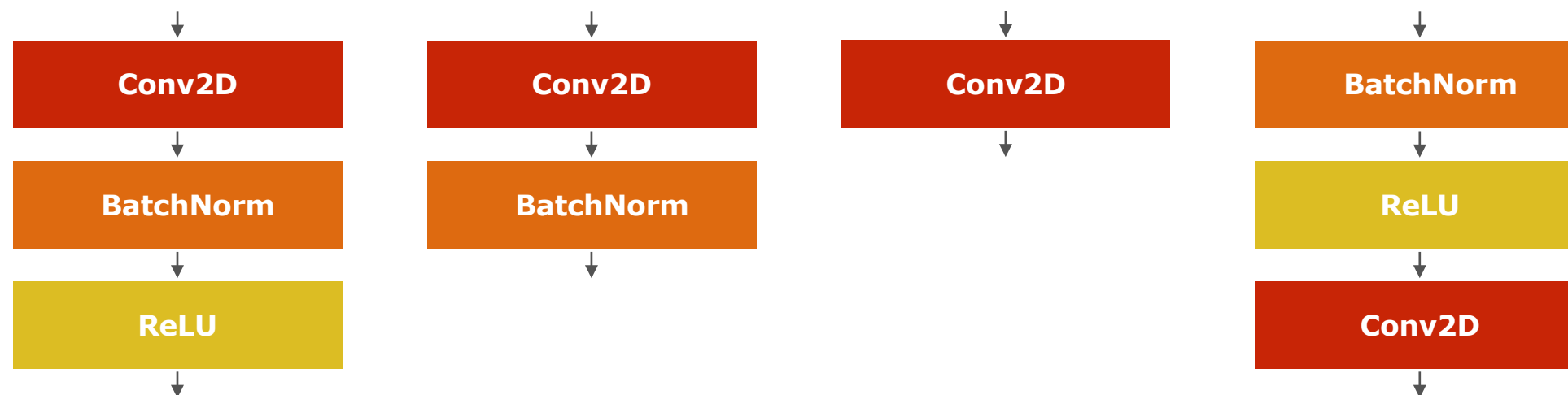


**Let's assemble the key parts
required to form a very deep neural
network!**

Components in ResNet

Some layers arrangement

- Let's define a few types of layer arrangements
- Note: the Conv2D solely performs convolution with padding, no activation function involves



Define resLyr

Create the function

- Create a function that can encompass the several layer arrangements

- Note: default kernel size is 3 x 3 for 2D convolution

```
> def resLyr(inputs,
             numFilters=16,
             kernelSz=3,
             strides=1,
             activation='relu',
             batchNorm=True,
             convFirst=True,
             lyrName=None):
    convLyr = Conv2D(numFilters,
                     kernel_size=kernelSz,
                     strides=strides, padding='same',
                     kernel_initializer='he_normal',
                     kernel_regularizer=l2(1e-4),
                     name=lyrName+'_conv' if lyrName else None)

    x = inputs
    if convFirst:
        x = convLyr(x)
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x)
        if activation is not None:
            x = Activation(activation, name=lyrName+'_'+activation if lyrName else None)(x)
    else:
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x)
        if activation is not None:
            x = Activation(activation,
                          name=lyrName+'_'+activation if lyrName else None)(x)
        x = convLyr(x)
    return x
```

if lyrName is set to 'blk', then the name of the layer will be 'blk_conv', else no name will be given to the layer

resLyr

Types of layer arrangements

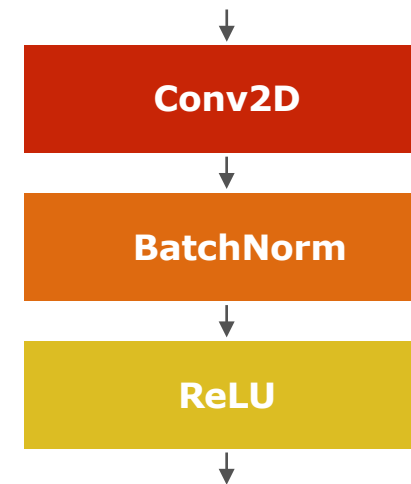
```
> def resLyr(inputs,
             numFilters=16,
             kernelSz=3,
             strides=1,
             activation='relu',
             batchNorm=True,
             convFirst=True,
             lyrName=None):
    convLyr = Conv2D(numFilters,
                    kernel_size=kernelSz,
                    strides=strides, padding='same',
                    kernel_initializer='he_normal',
                    kernel_regularizer=l2(1e-4),
                    name=lyrName+'_conv' if lyrName else None)

    = inputs

    x
    if convFirst:
        x = convLyr(x)
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,name=lyrName+'_'+activation if lyrName else None)(x)
    else:
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,
                            name=lyrName+'_'+activation if lyrName else None)(x)
        x = convLyr(x)
    return x
```

•When

convFirst is True
batchNorm is True
activation is 'relu'



resLyr

Types of layer arrangements

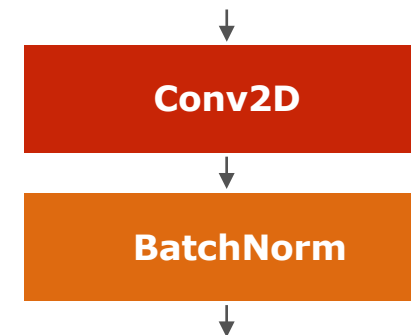
```
> def resLyr(inputs,
             numFilters=16,
             kernelSz=3,
             strides=1,
             activation='relu',
             batchNorm=True,
             convFirst=True,
             lyrName=None):
    convLyr = Conv2D(numFilters,
                    kernel_size=kernelSz,
                    strides=strides, padding='same',
                    kernel_initializer='he_normal',
                    kernel_regularizer=l2(1e-4),
                    name=lyrName+'_conv' if lyrName else None)

    = inputs

    x
    if convFirst:
        x = convLyr(x)
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,name=lyrName+'_'+activation if lyrName else None)(x)
    else:
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,
                            name=lyrName+'_'+activation if lyrName else None)(x)
        x = convLyr(x)
    return x
```

•When

convFirst is True
batchNorm is True
activation is None



resLyr

Types of layer arrangements

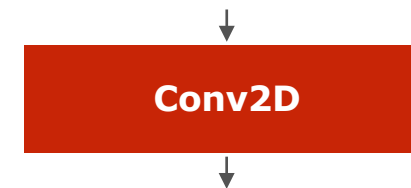
```
> def resLyr(inputs,
             numFilters=16,
             kernelSz=3,
             strides=1,
             activation='relu',
             batchNorm=True,
             convFirst=True,
             lyrName=None):
    convLyr = Conv2D(numFilters,
                    kernel_size=kernelSz,
                    strides=strides, padding='same',
                    kernel_initializer='he_normal',
                    kernel_regularizer=l2(1e-4),
                    name=lyrName+'_conv' if lyrName else None)

    = inputs

    x
    if convFirst:
        x = convLyr(x)
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
activation is not None:
            x = Activation(activation,name=lyrName+'_'+activation if lyrName else None)(x)
    else:
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
activation is not None:
            x = Activation(activation,
                        name=lyrName+'_'+activation if lyrName else None)(x)
        x = convLyr(x)
    return x
```

•When

convFirst is True
batchNorm is False
activation is None



resLyr

Types of layer arrangements

```
> def resLyr(inputs,
              numFilters=16,
              kernelSz=3,
              strides=1,
              activation='relu',
              batchNorm=True,
              convFirst=True,
              lyrName=None):
    convLyr = Conv2D(numFilters,
                     kernel_size=kernelSz,
                     strides=strides, padding='same',
                     kernel_initializer='he_normal',
                     kernel_regularizer=l2(1e-4),
                     name=lyrName+'_conv' if lyrName else None)

    = inputs

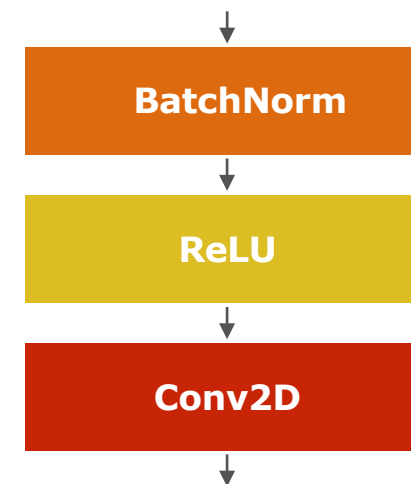
    x
    if convFirst:
        x = convLyr(x)
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,name=lyrName+'_'+activation if lyrName else None)(x)
    else:
        if batchNorm:
            x = BatchNormalization(name=lyrName+'_bn' if lyrName else None)(x) if
            activation is not None:
                x = Activation(activation,
                             name=lyrName+'_'+activation if lyrName else None)(x)
        x = convLyr(x)
    return x
```

•When

convFirst is False

batchNorm is True

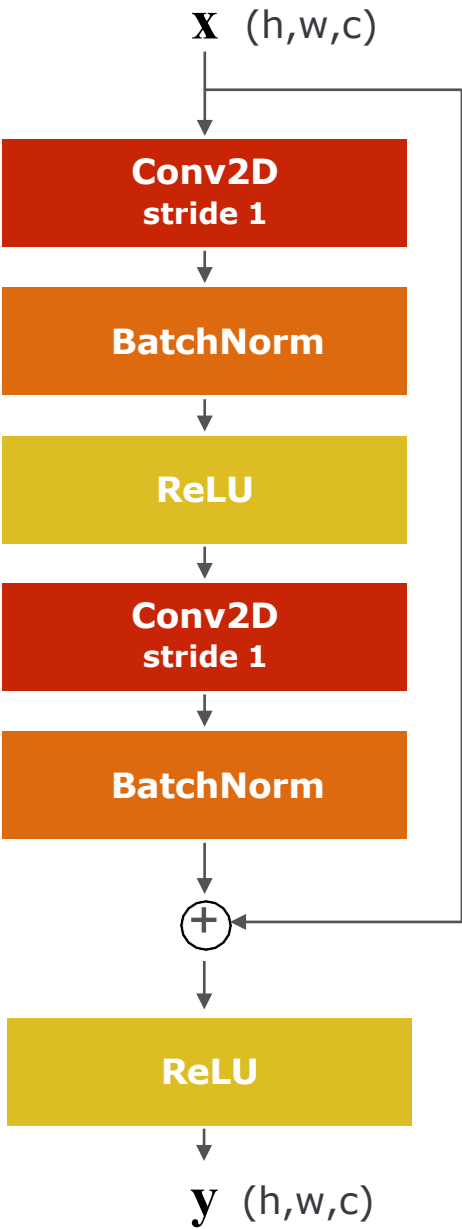
activation is 'relu'



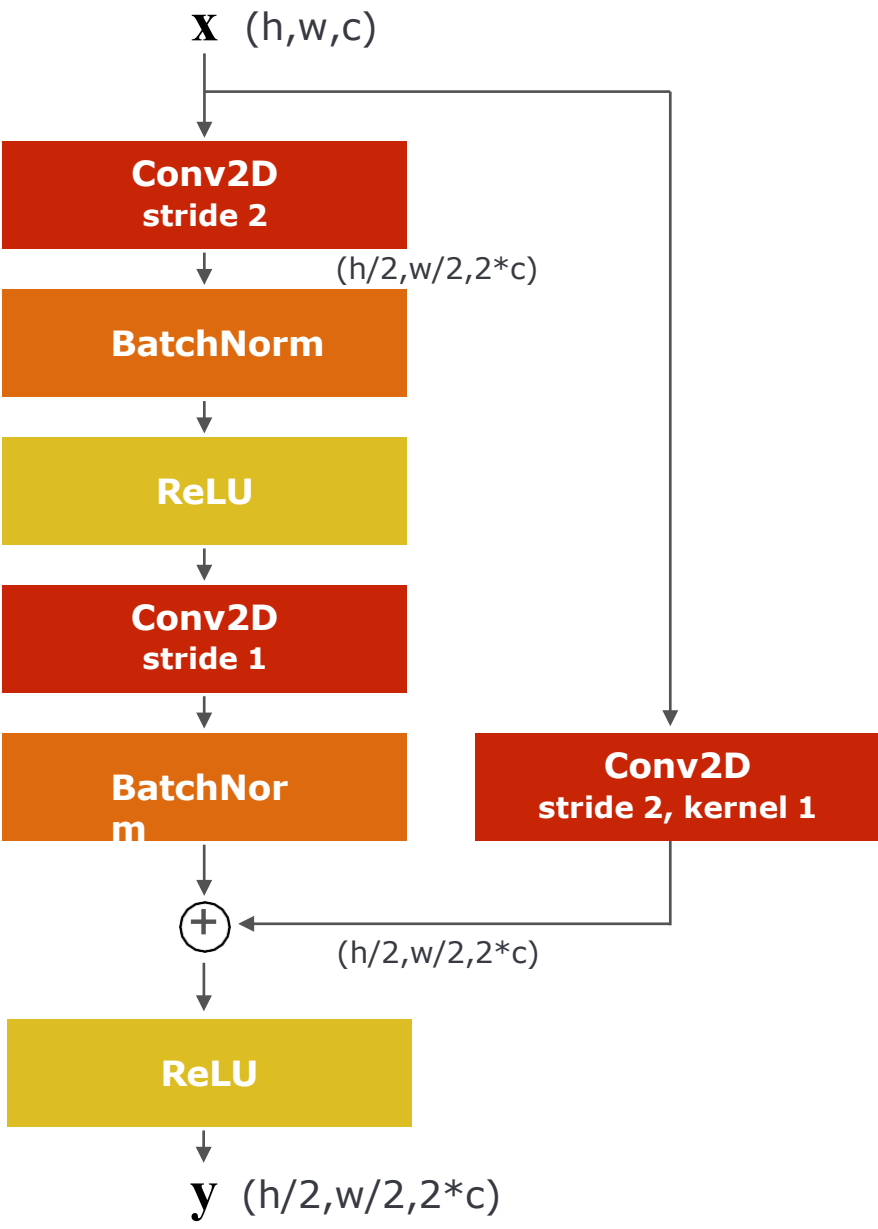
Residual blocks

Mix and match

- Two types of residual blocks in resnet v1



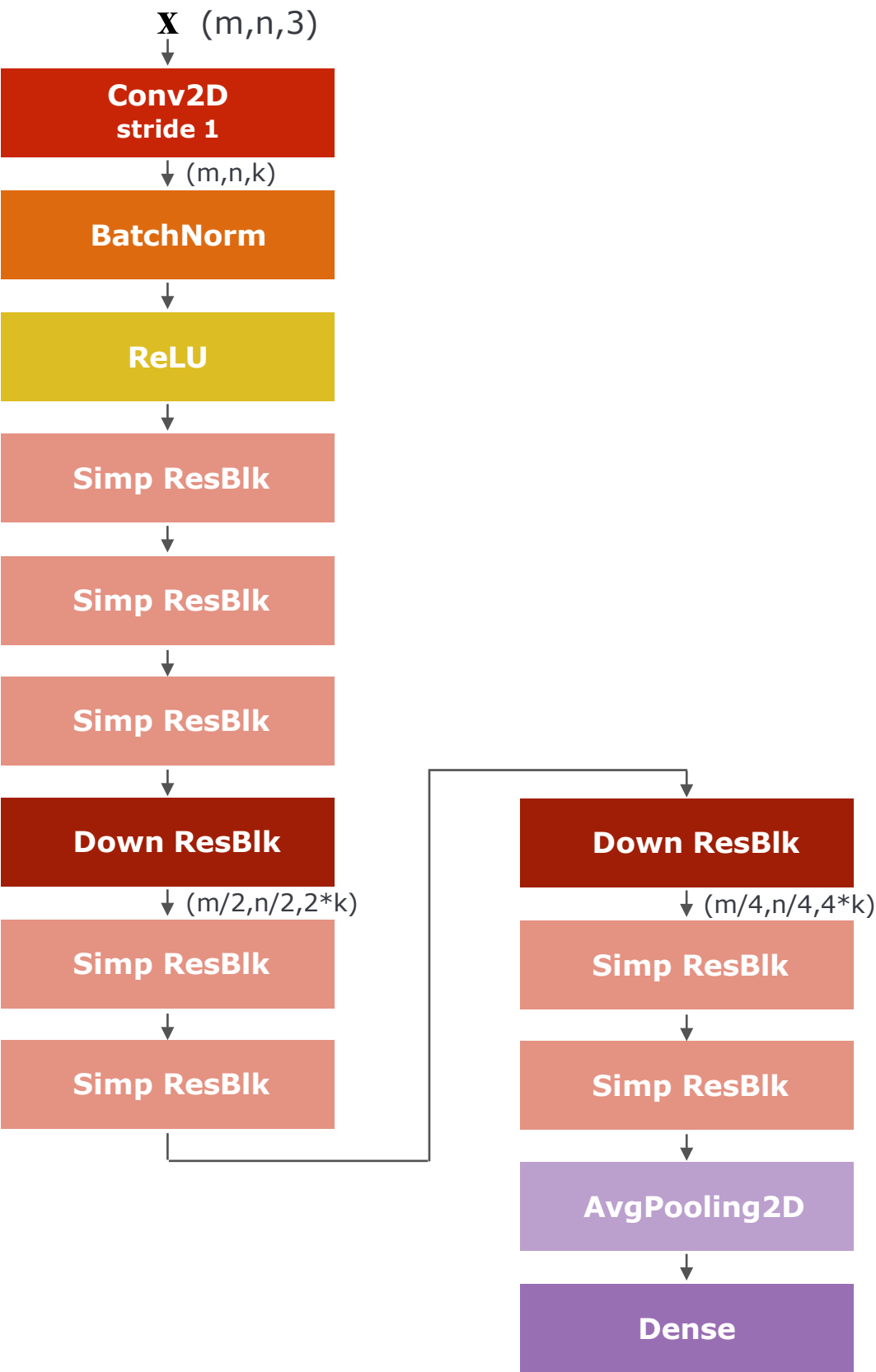
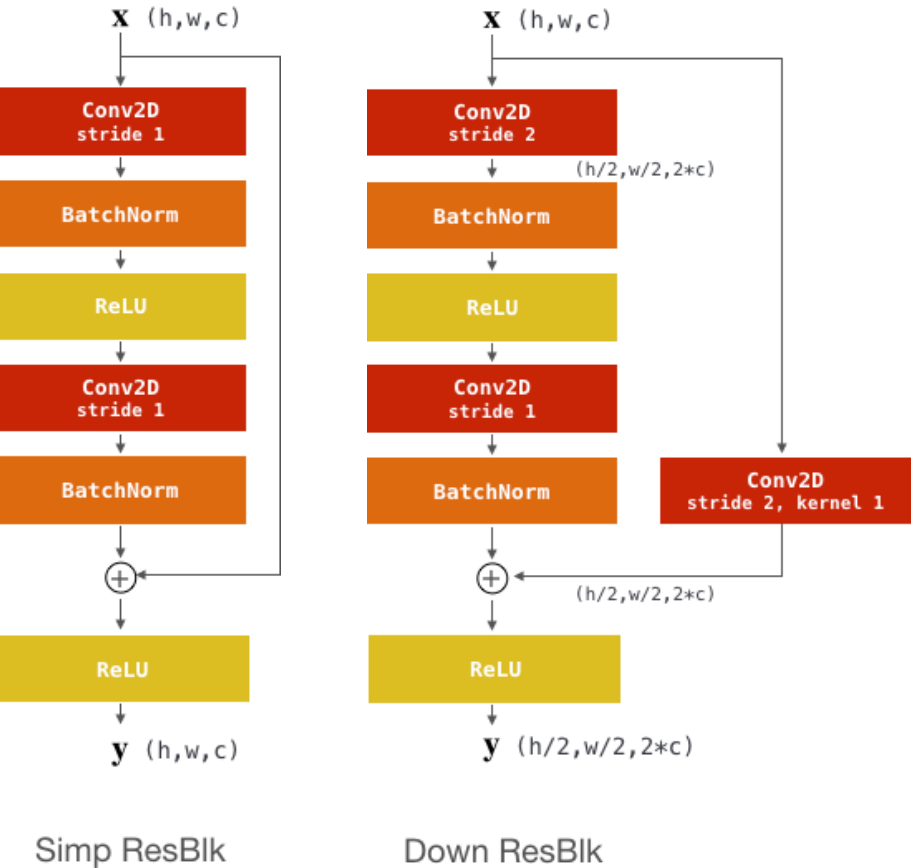
Simp ResBlk



Down ResBlk

Resnet v1

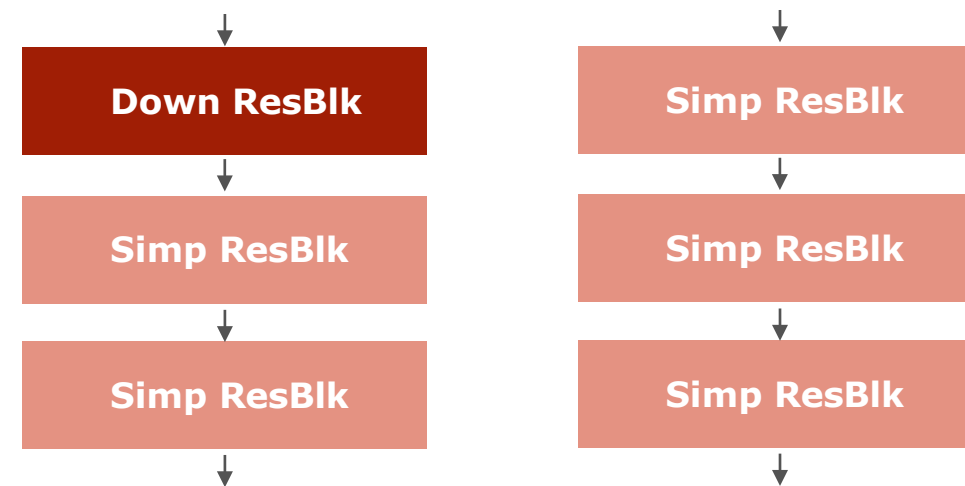
The full structure



Define resBlkV1

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=names+'_Blk'+blkStr+'_Res1' if names else None)
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=names+'_Blk'+blkStr+'_Res2' if names else None)
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=names+'_Blk'+blkStr+'_lin' if names else None)
        x = add([x,y],
               name=names+'_Blk'+blkStr+'_add' if names else None)
        x = Activation('relu',
                      name=names+'_Blk'+blkStr+'_relu' if names else None)(x)
    return x
```

- The function to produce these two sets of blocks



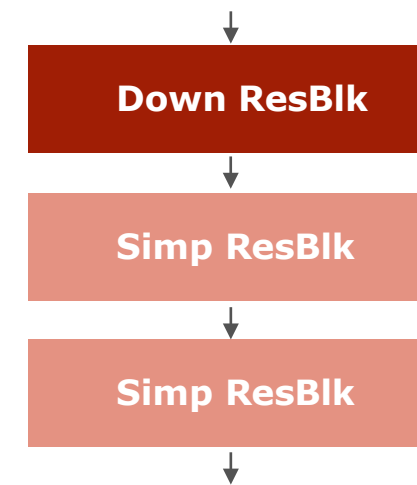
resBlkV1 configuration

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=...)
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=...)
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=...)
        x = add([x,y],
               name=...)
        x = Activation('relu',
                      name=...)(x)
    return x
```

- When

`downsampleOnFirst` is `True`
`numBlocks` is `3`

- The function creates the below 3 blocks



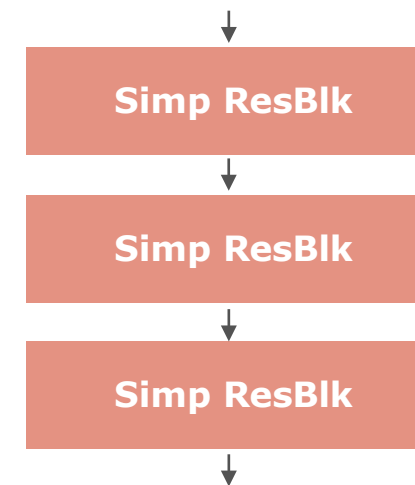
resBlkV1 configuration

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=...)
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=...)
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=...)
        x = add([x,y],
                name=...)
        x = Activation('relu',
                       name=...)(x)
    return x
```

- When

`downsampleOnFirst` is `False`
`numBlocks` is `3`

- The function creates the below 3 blocks

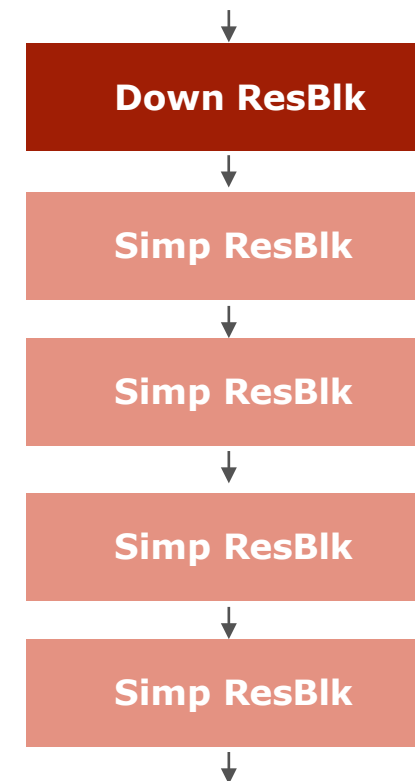


resBlkV1 configuration

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=...)
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=...)
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=...)
        x = add([x,y],
                name=...)
        x = Activation('relu',
                       name=...)(x)
    return x
```

- We can have as many blocks as we like; this is what we get when

downsampleOnFirst is True
numBlocks is 5



resBlkV1 configuration

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0, numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=... )
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=... )
        x = add([x, y],
                name=... )
        x = Activation('relu',
                       name=... )(x)
    return x
```

- The variable `run` controls the creation of blocks



resBlkV1 configuration

```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=... )
        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=... )
        x = add([x,y],
               name=... )
        x = Activation('relu',
                      name=... )(x)
    return x
```

- When **downsampleOnFirst** is set to **False**

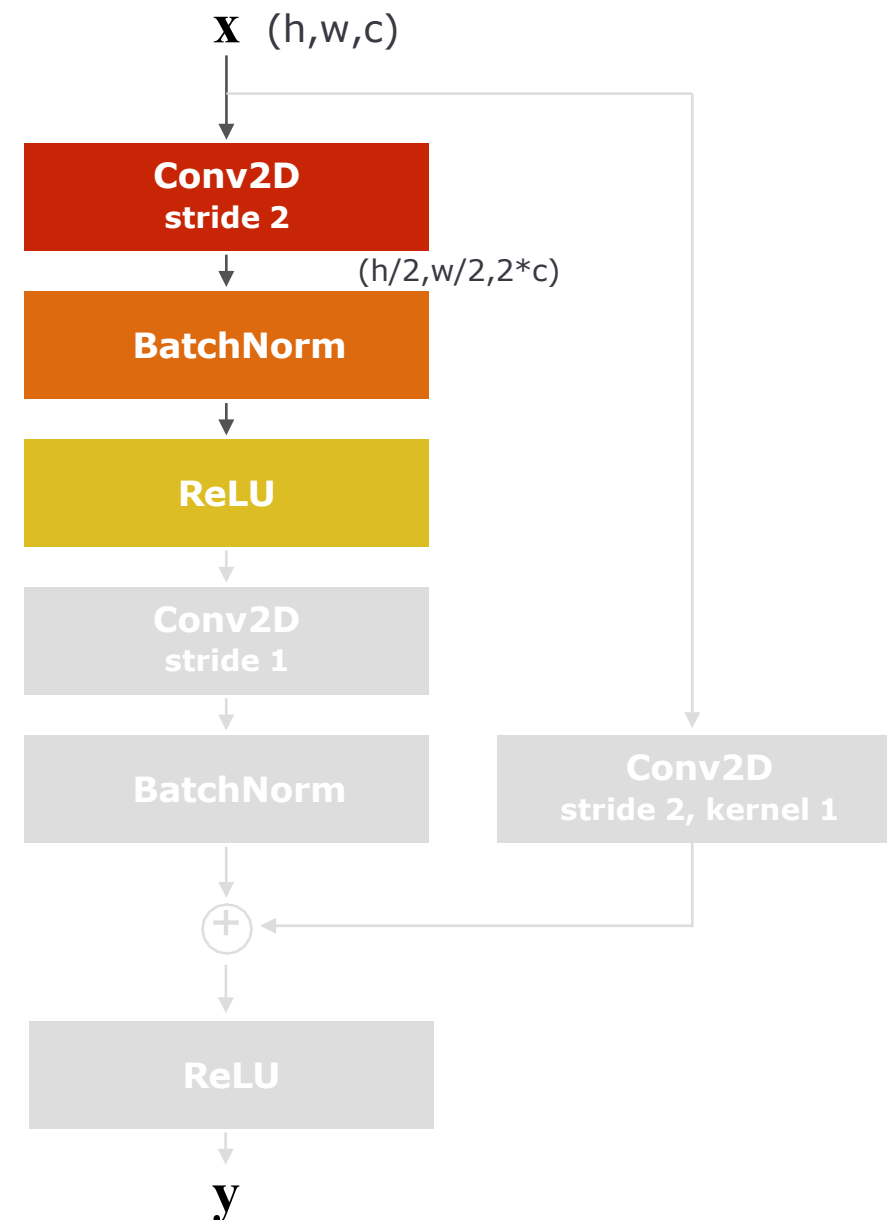


**Let's take a look at how 'Down ResBlk'
is created**

Down ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )
        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )
        x = add([x,y],
            name=... )
        x = Activation('relu',
            name=... )(x)
    return x
```

- Strides is 2 for the first part of the block:



Down ResBlk

Down ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )

        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )

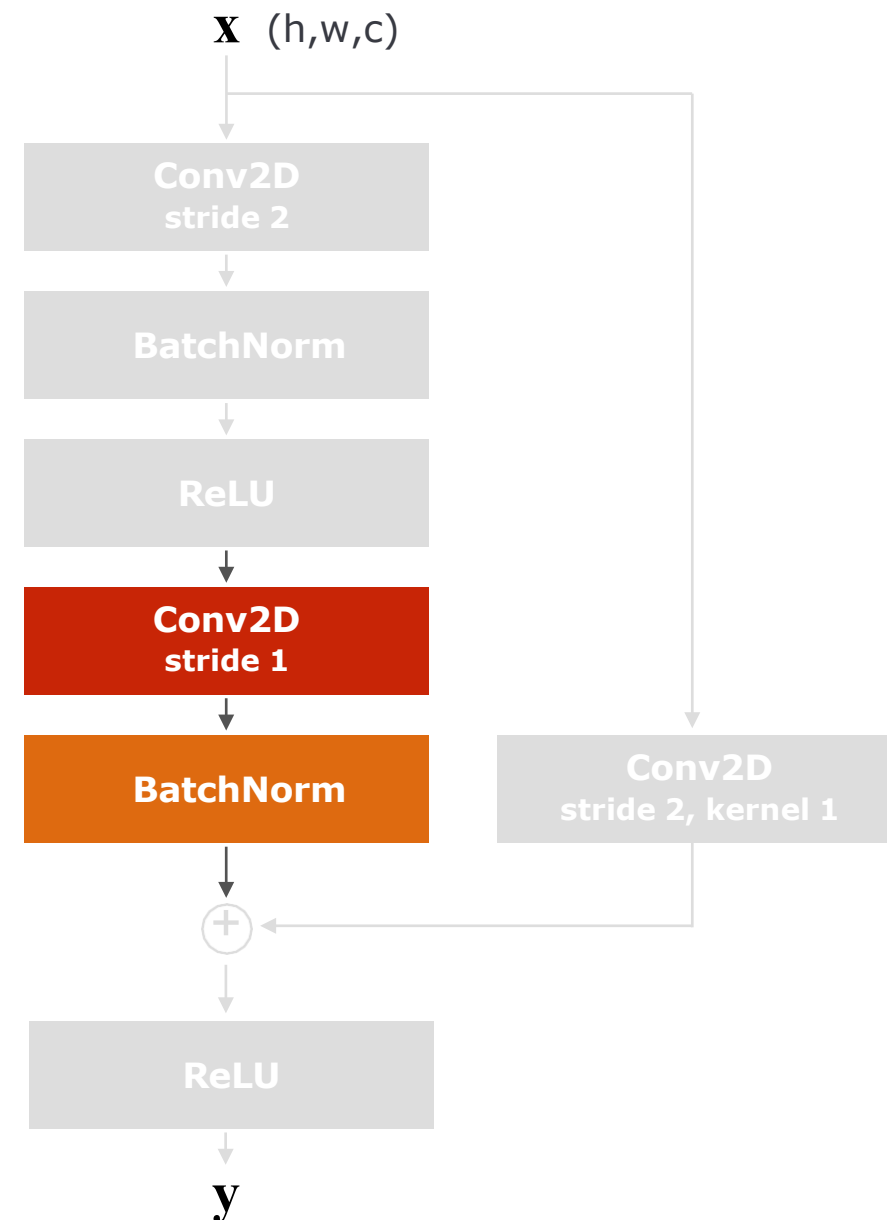
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )

        x = add([x,y],
            name=... )

        x = Activation('relu',
            name=... )(x)

    return x
```

- The second part of the block:

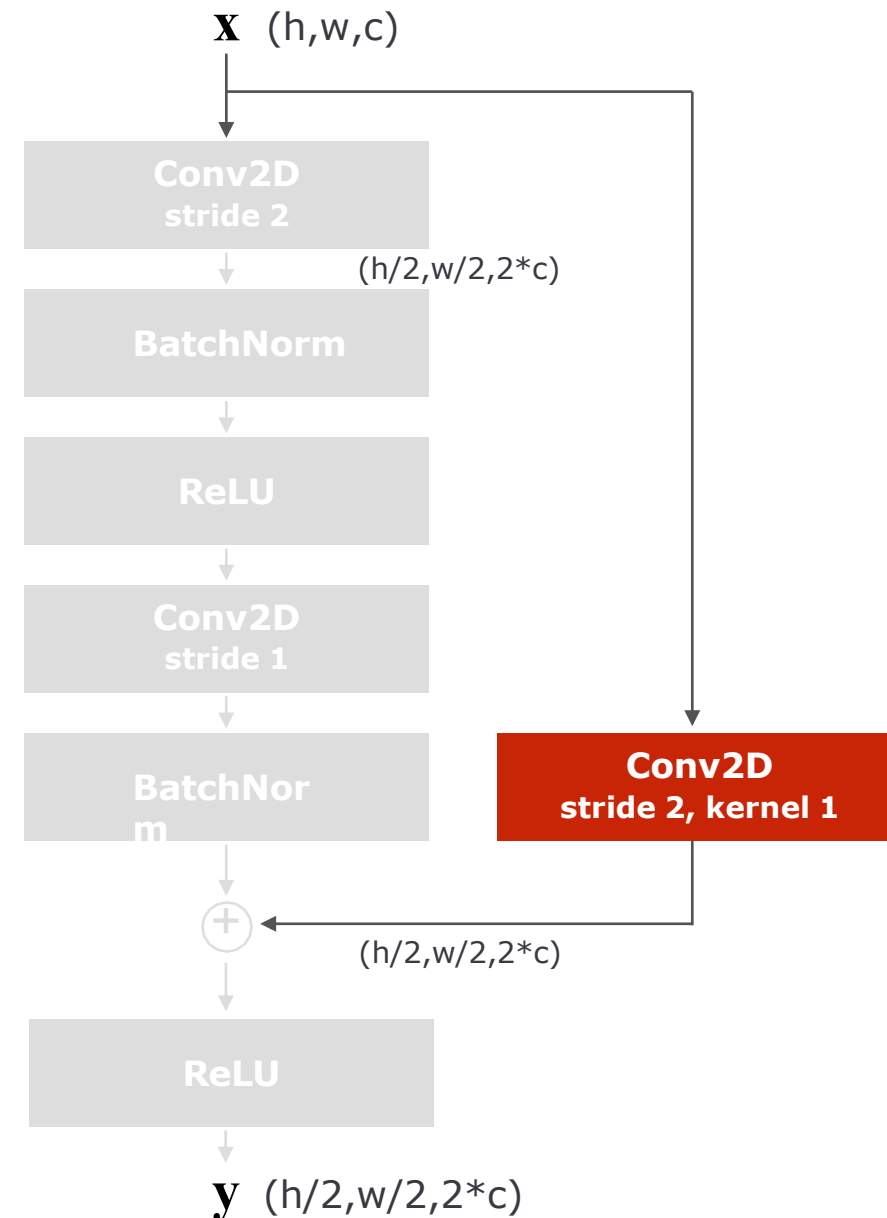


Down ResBlk

Down ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )
        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )
        x = add([x,y],
            name=... )
        x = Activation('relu',
            name=... )(x)
    return x
```

- The third part of the block:

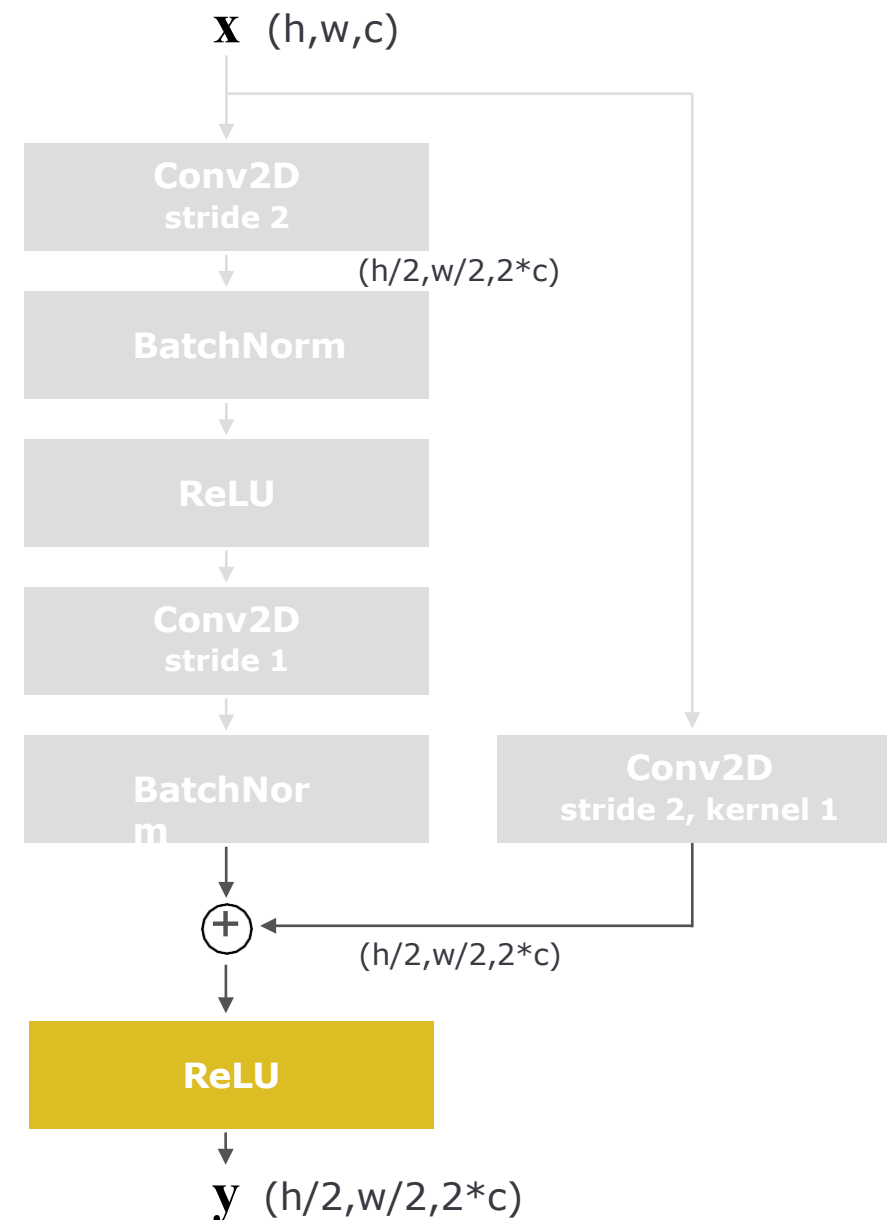


Down ResBlk

Down ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )
        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )
        x = add([x,y],
            name=... )
        x = Activation('relu',
            name=... )(x)
    return x
```

- The last of the block:



Down ResBlk

How about 'Simp ResBlk'?

Simp ResBlk

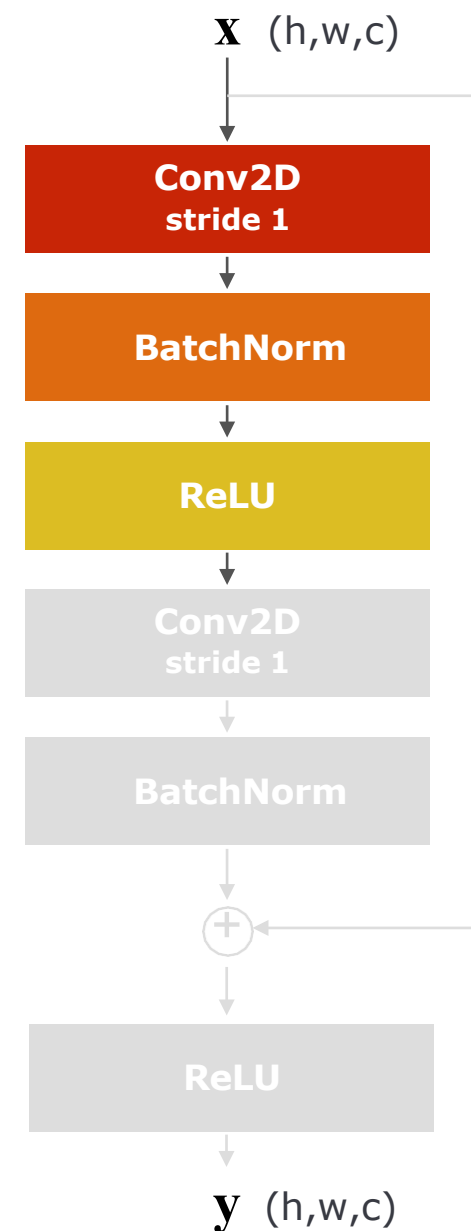
```
> def resBlkV1(inputs,
               numFilters=16,
               numBlocks=3,
               downsampleOnFirst=True,
               names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
                  numFilters=numFilters,
                  strides=strides,
                  lyrName=... .)

        y = resLyr(inputs=y,
                  numFilters=numFilters,
                  activation=None,
                  lyrName=... .)

        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                      numFilters=numFilters,
                      kernelSz=1,
                      strides=strides,
                      activation=None,
                      batchNorm=False,
                      lyrName=... .)

        x = add([x,y],
                name=... .)
        x = Activation('relu',
                       name=... .)(x)
    return x
```

- Strides is 1 for the first part of the block (no downsampling) :



Simp ResBlk

Simp ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )

        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )

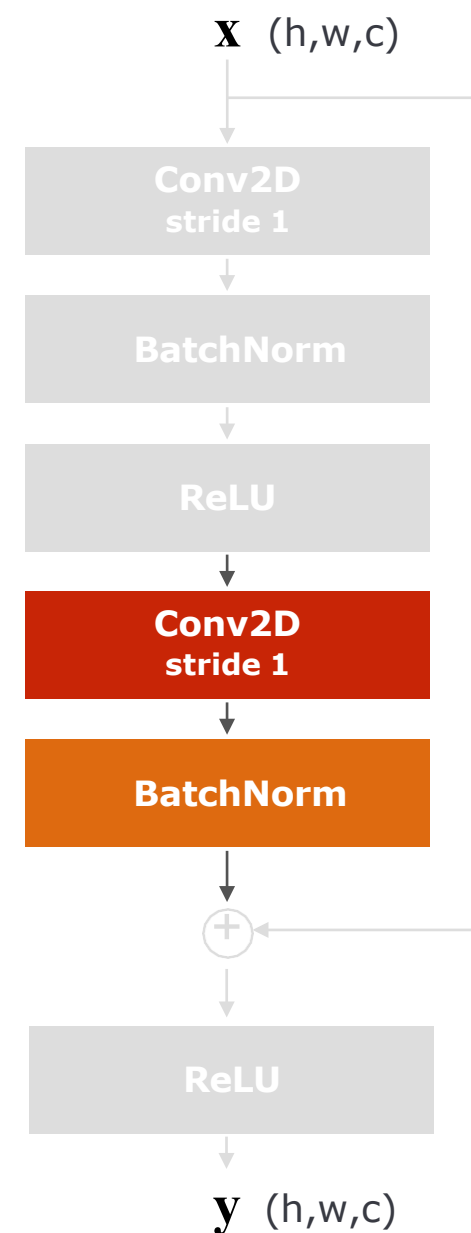
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )

        x = add([x,y],
            name=... )

        x = Activation('relu',
            name=... )(x)

    return x
```

- The second part of the block:

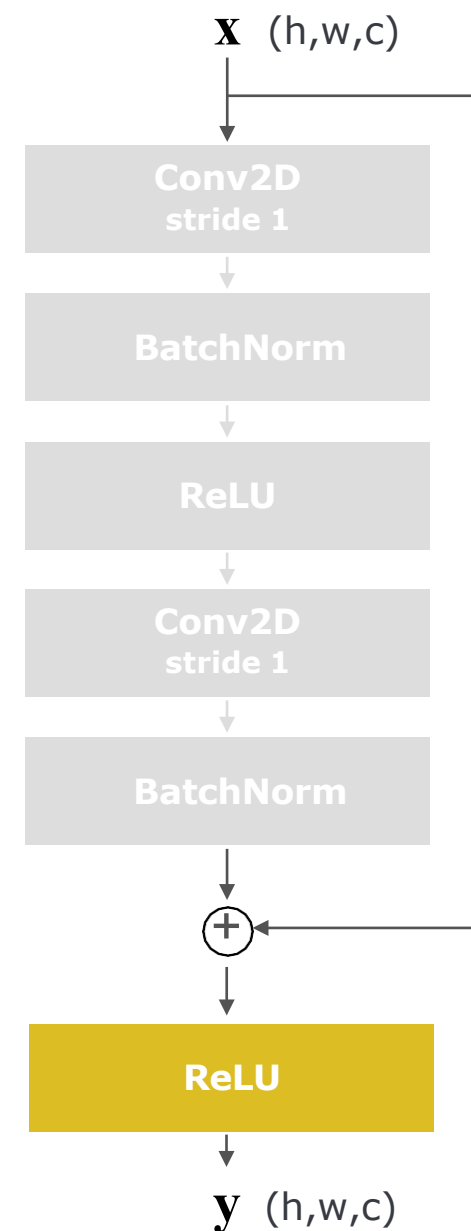


Simp ResBlk

Simp ResBlk

```
> def resBlkV1(inputs,
    numFilters=16,
    numBlocks=3,
    downsampleOnFirst=True,
    names=None):
    x = inputs
    for run in range(0,numBlocks):
        strides = 1
        blkStr = str(run+1)
        if downsampleOnFirst and run == 0:
            strides = 2
        y = resLyr(inputs=x,
            numFilters=numFilters,
            strides=strides,
            lyrName=... )
        y = resLyr(inputs=y,
            numFilters=numFilters,
            activation=None,
            lyrName=... )
        if downsampleOnFirst and run == 0:
            x = resLyr(inputs=x,
                numFilters=numFilters,
                kernelSz=1,
                strides=strides,
                activation=None,
                batchNorm=False,
                lyrName=... )
        x = add([x,y],
            name=... )
        x = Activation('relu',
            name=... )(x)
    return x
```

- The last part of the block:



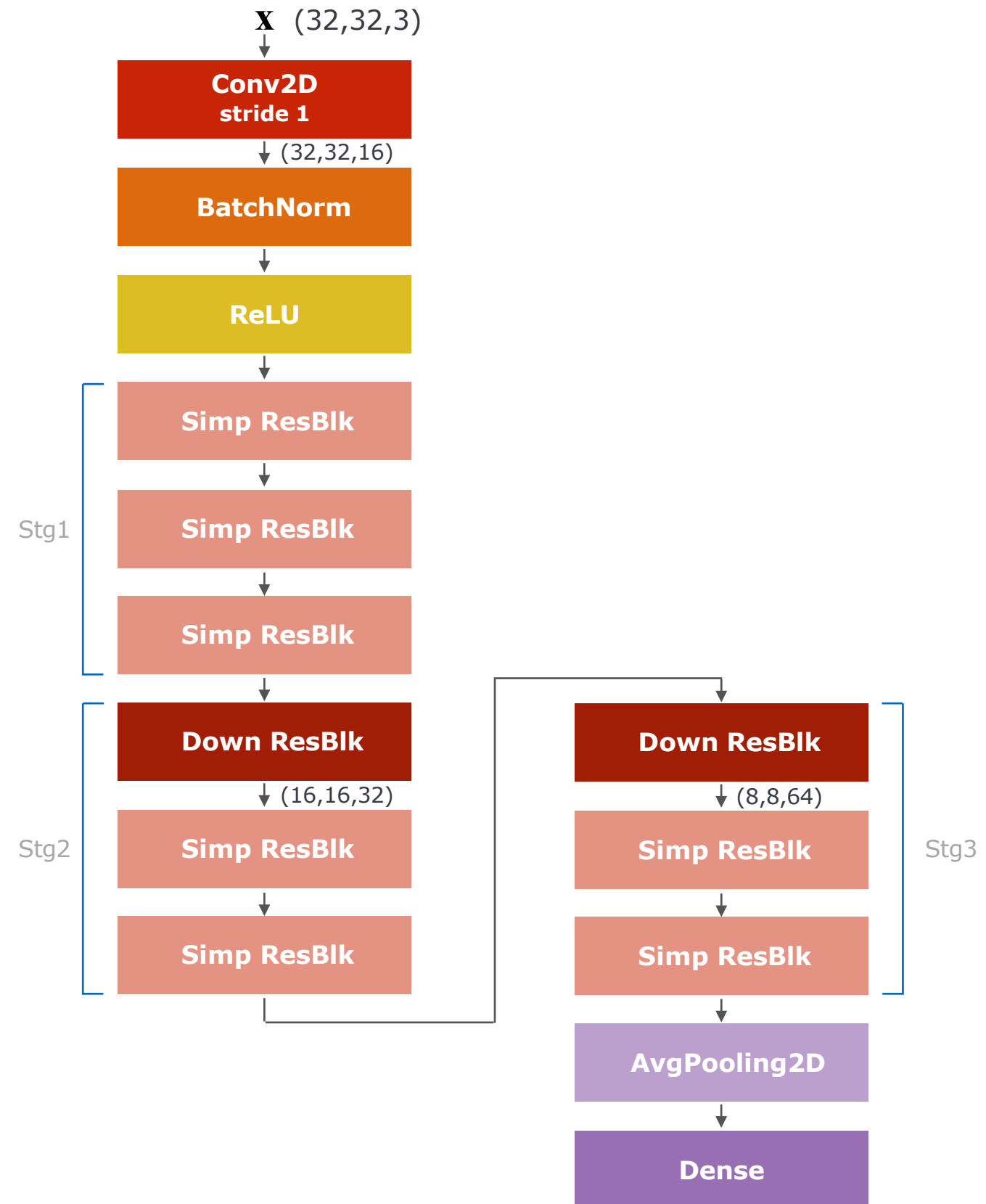
Simp ResBlk

Create full resnet v1

```
> def createResNetV1(inputShape=(32,32,3),
    numClasses=10):

1     inputs      = Input(shape=inputShape)
2     v           = resLyr(inputs,
        lyrName='Inpt')
3     v           = resBlkV1(inputs=v,
        numFilters=16,
        numBlocks=3,
        downsampleOnFirst=False,
        names='Stg1')
4     v           = resBlkV1(inputs=v,
        numFilters=32,
        numBlocks=3,
        downsampleOnFirst=True,
        names='Stg2')
5     v           = resBlkV1(inputs=v,
        numFilters=64,
        numBlocks=3,
        downsampleOnFirst=True,
        names='Stg3')
6     v           = AveragePooling2D(pool_size=8,
        name='AvgPool')(v)
    v           = Flatten()(v)
    outputs      = Dense(numClasses,
        activation='softmax',
        kernel_initializer='he_normal')(v)
    model        = Model(inputs=inputs,outputs=outputs)
    model.compile(loss='categorical_crossentropy',
        optimizer=optmz,
        metrics=['accuracy'])

    return model
```



Training

With Adam

- Use Adam as the optimizer.
- The initial learning rate is 0.001
- Set the batch size to be 128

```
> seed = 29  
> np.random.seed(seed)  
  
> optmz = optimizers.Adam(lr=0.001)  
> modelname = 'cifar10ResV1Cfg1'
```

$m = 0, \quad v = 0$ Initialization

$$m = \beta_1 \cdot m + (1 - \beta_1) \cdot dw$$

$$v = \beta_2 \cdot v + (1 - \beta_2) \cdot dw^2$$

$$\hat{m} = \frac{m}{1 - \beta_1}$$

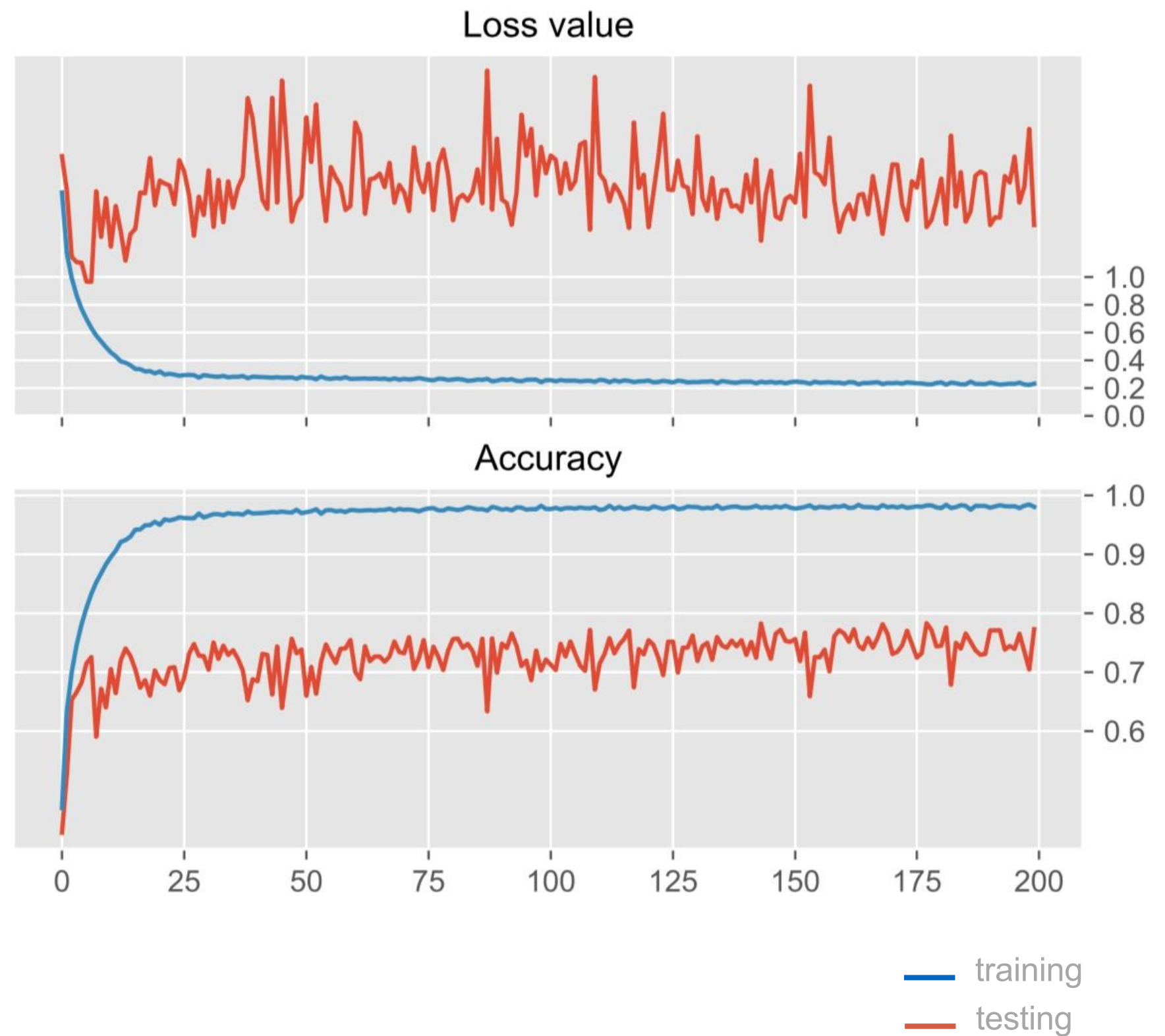
$$\hat{v} = \frac{v}{1 - \beta_2}$$

$$w = w - \alpha \cdot \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}}$$

Source: <https://arxiv.org/pdf/1412.6980.pdf>

Training

The problem?



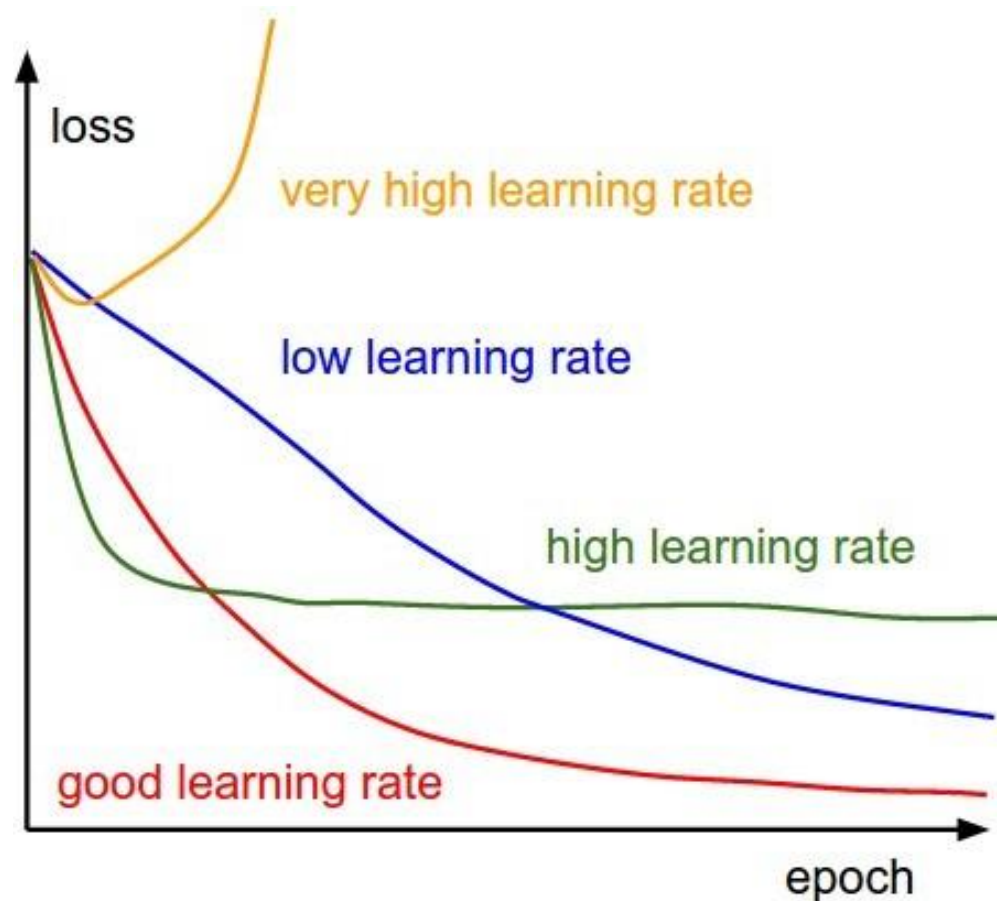
Accuracy: 78.32%

**How to unleash the power of a
deep network?**

Training

Solution to improve?

- Learning rate: the amount that the weights are updated during training, also called 'step size'
- Usually a value between 0.0 and 1.0
- Large learning rate causes model to converge too fast to a suboptimal solution
- Small learning rate gets training stuck



Source: <http://cs231n.github.io/neural-networks-3/>

Training

Solution to improve?

- Try learning scheduler

```
> from tensorflow.keras.callbacks import LearningRateScheduler
```

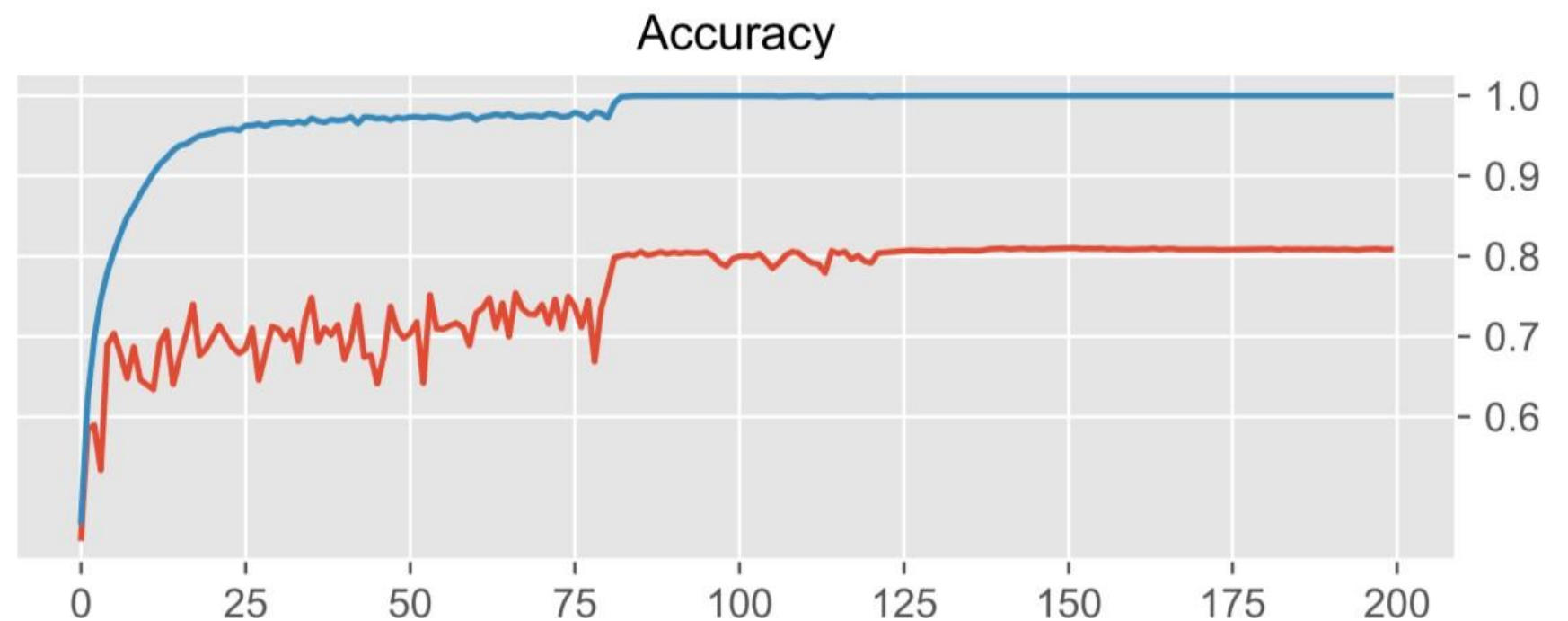
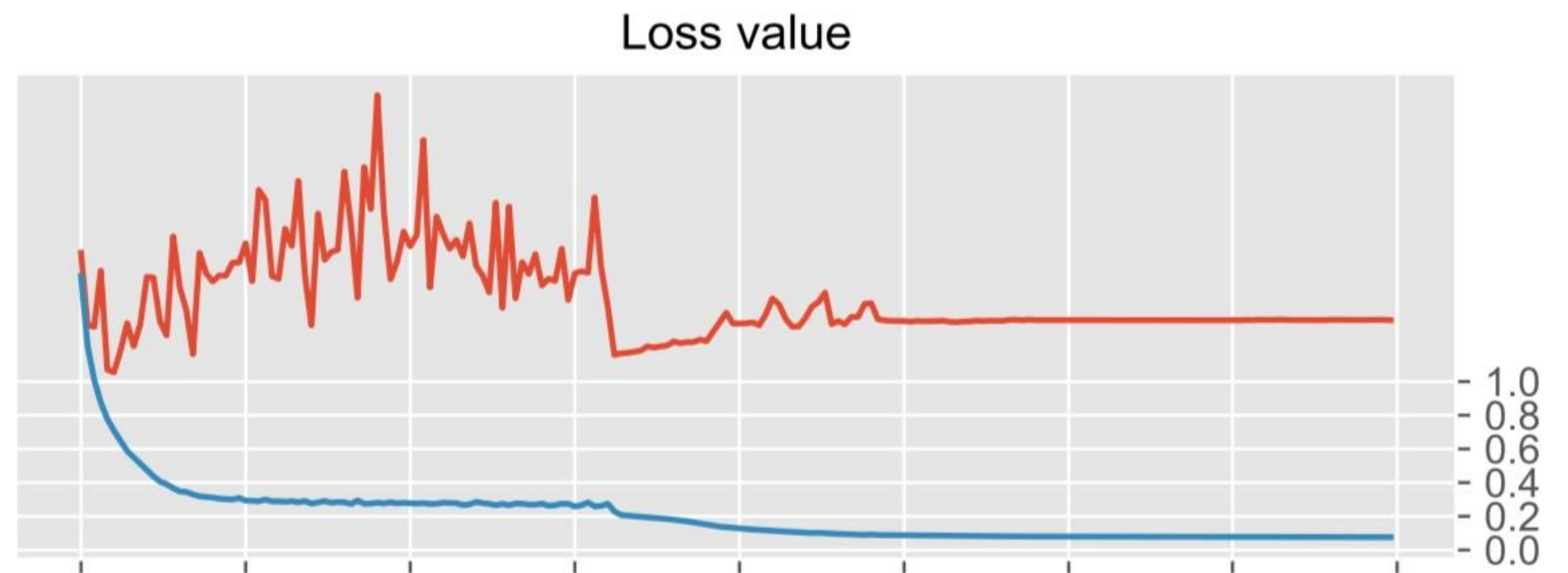
```
> def lrSchedule(epoch):  
    lr = 1e-3  
    if epoch > 160:  
        lr *= 0.5e-3  
    elif epoch > 140:  
        lr *= 1e-3  
    elif epoch > 120:  
        lr *= 1e-2  
    elif epoch > 80:  
        lr *= 1e-1  
    print('Learning rate: ', lr)  
    return lr
```

```
> LRScheduler = LearningRateScheduler(lrSchedule)
```

```
> callbacks_list = [checkpoint, csv_logger, LRScheduler]
```

Training

Better?



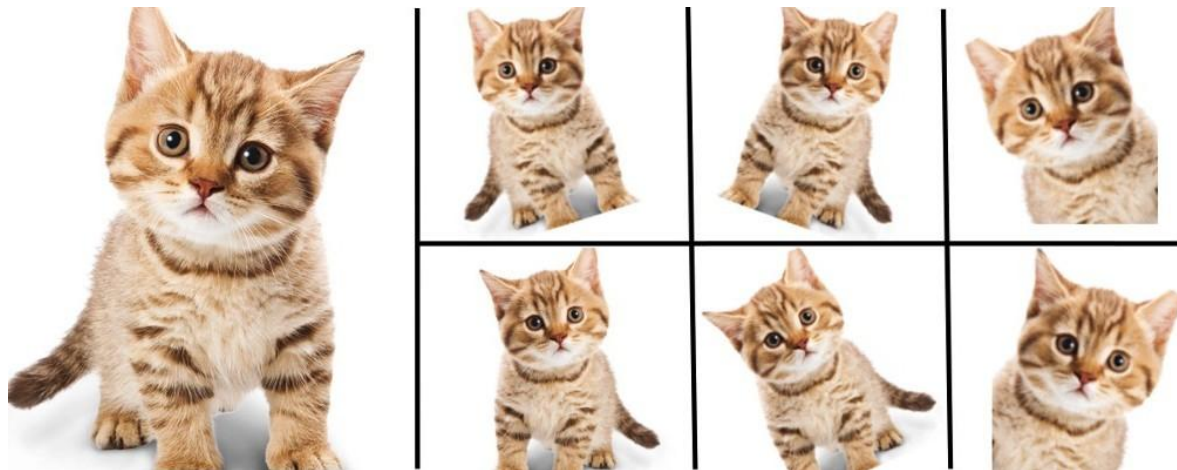
— training
— testing

Accuracy: 81.02%

Training

Solution to improve?

- Under current setup, the network sees the same set of images every epoch
- Need to create variety to force the net to learn the features that really matter for classification
- Generates randomly varied images in the beginning of every epoch, so that the network will not see the exact images twice
- This is called image augmentation



Source: <https://medium.com/nanonets/how-to-use-deep-learning-when-you-have-limited-data-part-2-data-augmentation-c26971dc8ced>

Training

Image augmentation

- Types of augmentation

Translation



Zoom



Flip



Rotate



Source: <https://medium.com/nanonets/how-to-use-deep-learning-when-you-have-limited-data-part-2-data-augmentation-c26971dc8ced>

Training

with image generator

- Build image generator and use `fit_generator` to train

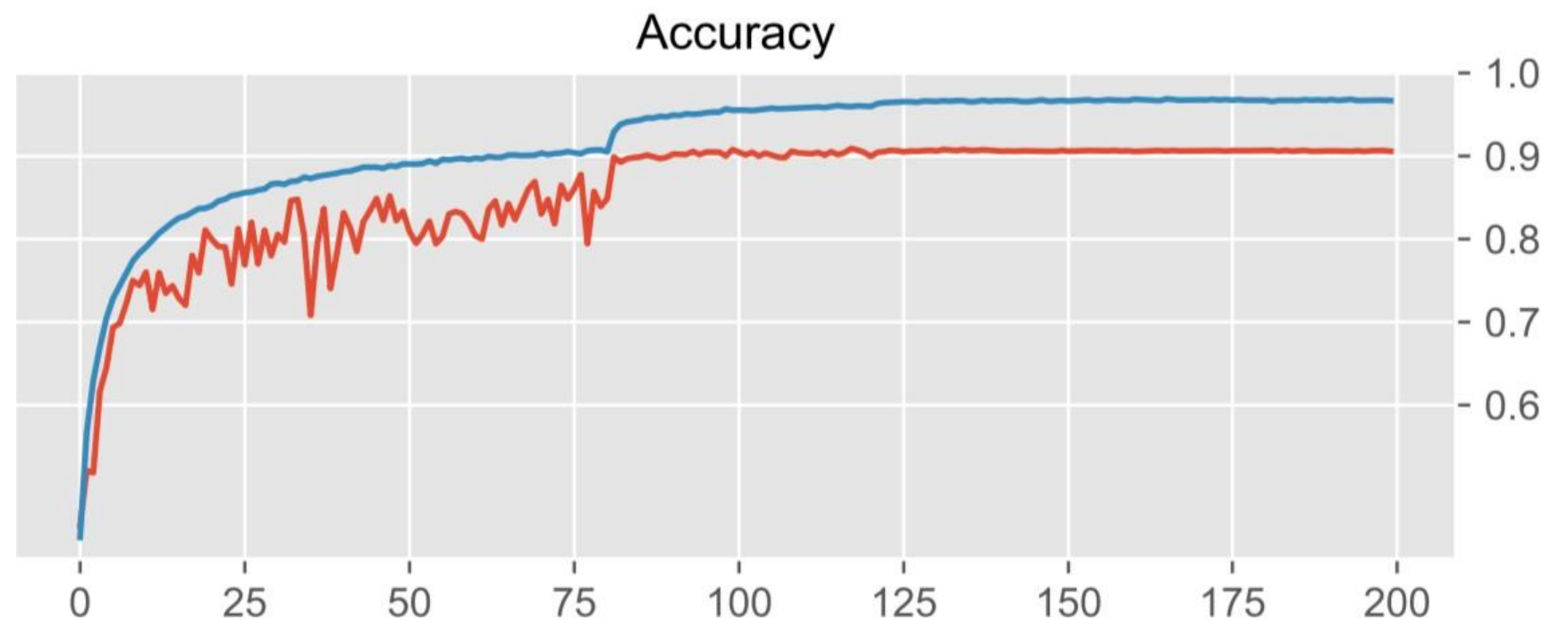
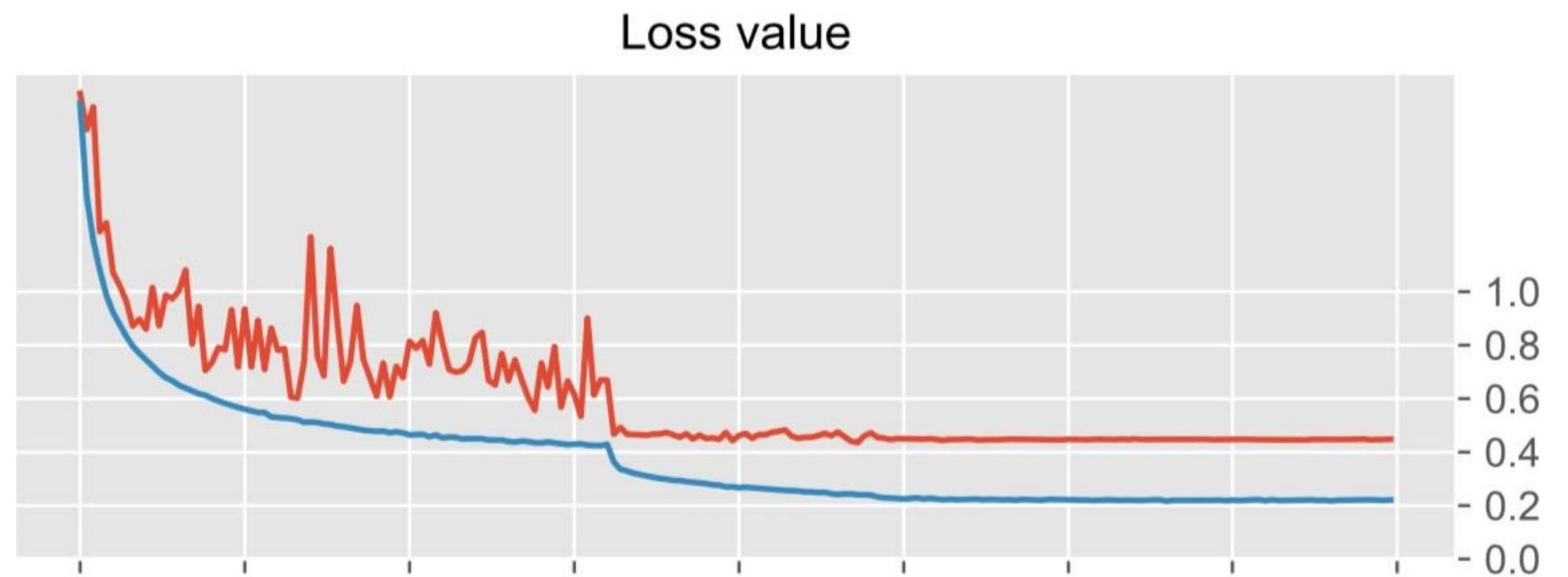
```
> from tensorflow.keras.preprocessing.image import ImageDataGenerator

> datagen = ImageDataGenerator(width_shift_range=0.1,
                                height_shift_range=0.1,
                                rotation_range=20,
                                horizontal_flip=True,
                                vertical_flip=False)

> model.fit_generator(datagen.flow(trDat, trLbl, batch_size=128),
                      validation_data=(tsDat, tsLbl),
                      epochs=200,
                      verbose=1,
                      steps_per_epoch=len(trDat)/128,
                      callbacks=callbacks_list)
```

Training

This looks good



— training
— testing

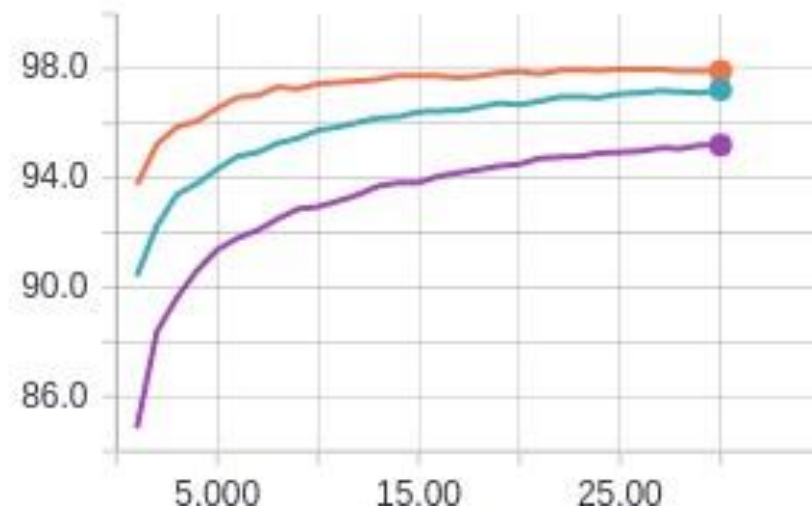
Accuracy: **90.91%**

Training

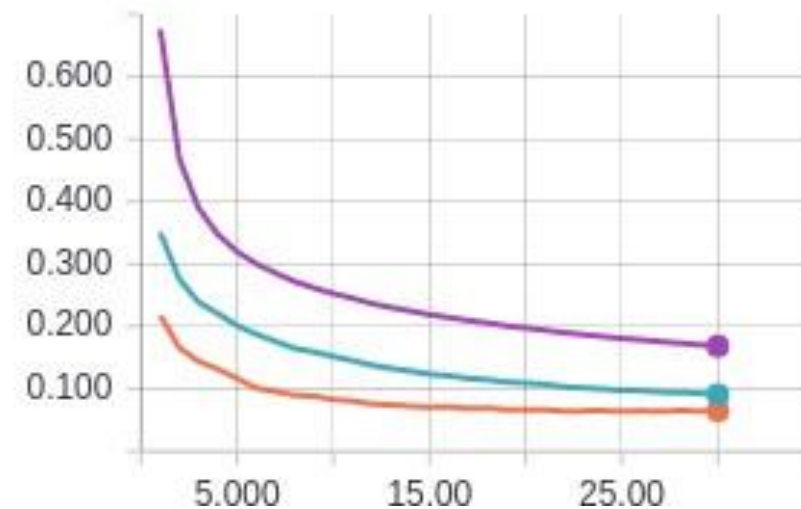
How about batch size?

- Large batch size: (much) faster training, but accuracy may suffer
- Small batch size: training gets noisy, offering regularizing effect and lower generalization
- GPU may not have sufficient memory to hold large batch of large size

test accuracy



test loss



— batch size 1024
— batch size 256
— batch size 64

Source: <https://medium.com/mini-distill/effect-of-batch-size-on-training-dynamics-21c14f7a716e>

Training

smaller batch size

- Make changes on the fit_generator

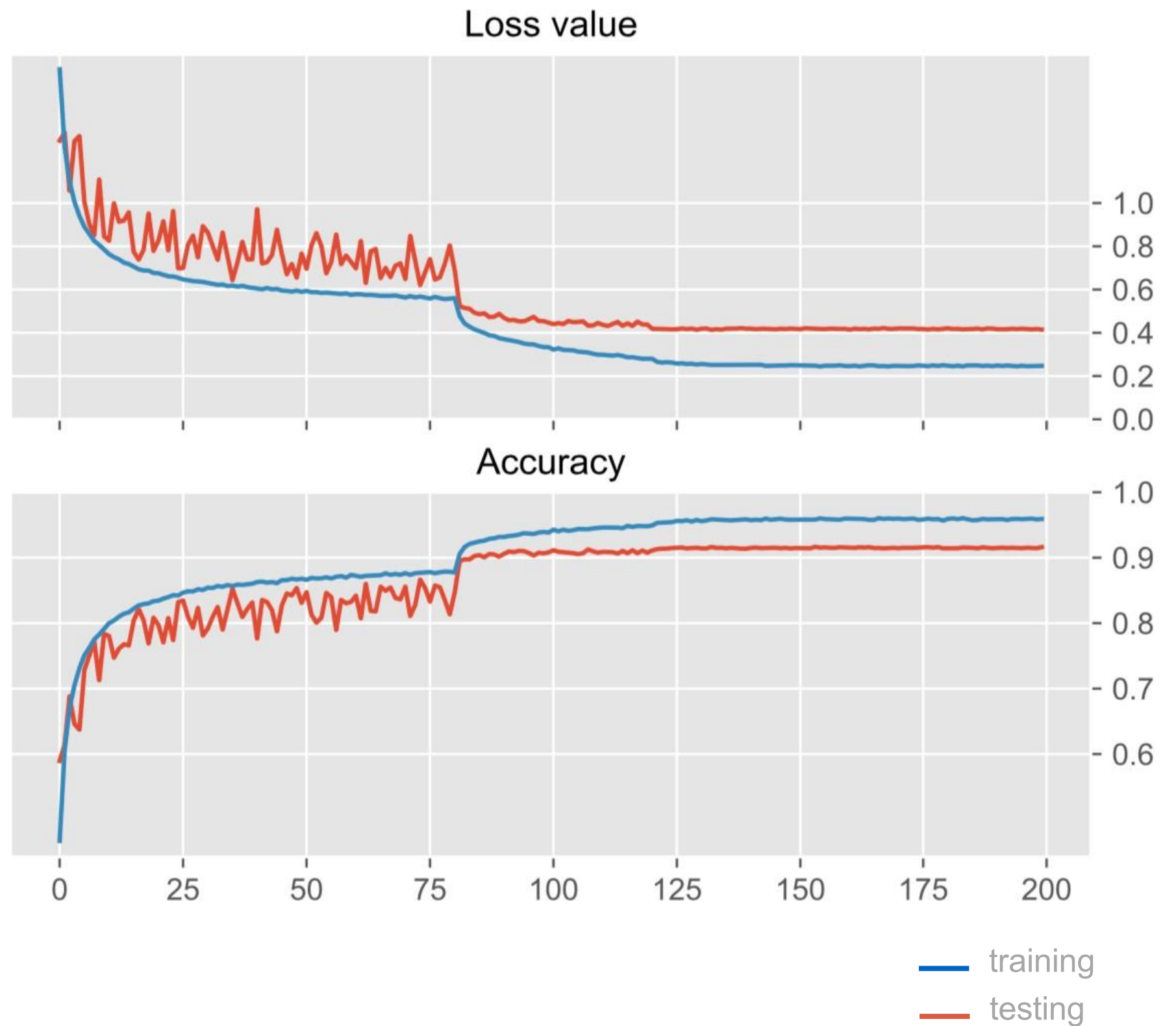
```
> model.fit_generator(datagen.flow(trDat, trLbl, batch_size=32),  
                      validation_data=(tsDat, tsLbl),  
                      epochs=200,  
                      verbose=1,  
                      steps_per_epoch=len(trDat)/32,  
                      callbacks=callbacks_list)
```



adjust
accordingly

Training

The finale



Accuracy: **91.67%**